

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

MASTER THESIS

Prokop Divín

**Combining Document Embeddings and
Knowledge Base-Enriched Entities to
Improve News Classification**

Institute of Formal and Applied Linguistics

Supervisor of the master thesis: RNDr. Jiří Hana, Ph.D.

Study programme: Informatika

Prague 2026

I declare that I carried out this master thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date

Author's signature

I would like to thank my supervisor, Jiří Hana, for his guidance throughout this thesis. He was always available, willing to help me, and enthusiastically explained every topic I needed. Furthermore, I want to thank my family and friends for their support throughout my studies. Finally, I want to thank Gemini, which consistently offered suggestions for better phrasing and was always available to correct my spelling.

Title: Combining Document Embeddings and Knowledge Base-Enriched Entities to Improve News Classification

Author: Prokop Divín

Department: Institute of Formal and Applied Linguistics

Supervisor: RNDr. Jiří Hana, Ph.D., Institute of Formal and Applied Linguistics

Abstract: Text classification into the large IPTC Media Topics taxonomy is challenging when relying solely on dense document embeddings, which often miss explicit factual context for rare named entities. This thesis investigates enhancing a multilingual, multi-label news classifier by injecting real-world entity knowledge. We propose a modular late-fusion architecture that combines standard text embeddings with structured entity representations derived from Wikidata and Wikipedia. We systematically evaluate multiple embedding sources, feature fusion techniques, and pooling strategies. The results confirm that entity-aware models consistently outperform the text-only baseline. The best configuration, which uses a learned attention mechanism to aggregate multilingual entity vectors, achieved a 1.7% improvement in Micro F_1 and a 2.4% increase in Macro F_1 . This improvement was particularly substantial in specific domains, such as sports, showing that structured entity knowledge effectively addresses data sparsity and improves practical classification performance.

Keywords: Natural Language Processing, Named Entities, NLP, Knowledge Base Enrichment, IPTC Media Topics, News Classification

Contents

Introduction	8
0.1 Context and Background	8
0.2 Motivation and Problem Statement	8
0.3 Research Objectives and Contributions	9
0.4 Thesis Outline	10
1 Background and Theoretical Foundations	11
1.1 Hierarchical Multi-label Text Classification	11
1.1.1 IPTC Media Topics Taxonomy	12
1.2 Knowledge Bases and Entities	12
1.2.1 Entities	12
1.2.2 Named Entity Recognition (NER) and Entity Linking (EL)	13
1.2.3 Wikipedia and Wikidata	13
1.3 Representation Learning	15
1.3.1 Text Embeddings	15
1.3.2 Entity Embeddings	15
1.4 Neural Network Architectures	15
1.4.1 Multilayer Perceptron (MLP)	15
1.4.2 Activation Functions	15
1.4.3 Regularization	16
1.4.4 Feature Fusion and Attention	16
1.5 Training and Evaluation	16
1.5.1 Loss Function	17
1.5.2 Evaluation Metrics	17
1.6 Statistical Testing	18
1.6.1 McNemar’s Test	18
1.6.2 False Discovery Rate (FDR) Correction	18
2 Related Work	20
2.1 Previous Experiments with NLP Models	20
2.1.1 Knowledge-Aware Models	20
2.1.2 Knowledge-Injected Models	22
2.2 Entity-Augmented Text Classification	22
2.3 Entity Embedding Sources	23
2.3.1 Wikidata Embedding Project	23
2.3.2 Wiki2vec	24
2.3.3 Wikipedia2vec	25
2.4 Research Gap	25
3 Entities and Data	27
3.1 Geneea Entities and Their Source	27
3.1.1 Linked Entity Creation Pipeline	28
3.1.2 Entity Disambiguation	31
3.2 Gold and Silver Data	31
3.2.1 Gold Dataset	31

3.2.2	Silver Dataset	32
3.2.3	Dataset Selection	32
3.3	Selected Data	32
3.4	Categories and Labels	35
3.5	Dataset Characteristics and Limitations	36
3.5.1	Strengths	37
3.5.2	Limitations and Challenges	37
3.6	Dataset Division	37
3.6.1	Evaluated Partitioning Strategies	38
3.6.2	Train Split Statistics	39
3.7	Data Format	45
3.7.1	Feature Representation in CSV	45
3.7.2	Entity Enrichment and Knowledge Base Mapping	45
4	Methodology	47
4.1	System Architecture Overview	47
4.1.1	Article Embeddings	49
4.2	Entity Embeddings	50
4.3	Evaluation Metrics	52
4.3.1	Data Distribution and Metric Constraints	52
4.3.2	Limitations of Alternative Metrics	52
4.3.3	Primary Objective: Micro F_1 -Score	53
4.3.4	Secondary Metrics Suite	53
4.4	Tuning and Model Selection	54
4.4.1	Number of Epochs and Early Stopping	56
4.5	Experiment 1: Addressing Class Imbalance	58
4.6	Baseline Classification Model	59
4.6.1	Baseline Verification and Sanity Check	60
4.7	Enhanced Entity-Aware Classifiers	61
4.7.1	Embedding Selection and Multilingual Strategy	61
4.7.2	Experiment 2: Entity Embedding Sources	62
4.7.3	Custom Sources of Entity Embeddings	64
4.7.4	Experiment 3: Entity Pooling Strategies and Attention	65
4.7.5	Experiment 4: Adding Different Languages	67
4.7.6	Experiment 5: Summation vs. Concatenation	68
4.7.7	Code	68
5	Results	70
5.1	Establishing the Baseline Model	70
5.1.1	Experiment 1: Addressing Class Imbalance	70
5.2	Choosing the Best Model	71
5.2.1	Experiment 2: Entity Embedding Sources	71
5.2.2	Experiment 3: Entity Pooling Strategies and Attention	73
5.2.3	Experiment 4: Adding Different Languages	74
5.2.4	Experiment 5: Concatenation vs. Summation	75
5.2.5	Our Choice	76
5.3	Test Set Results	76
5.3.1	Best Model vs. Baseline	77

Conclusion	85
5.4 Summary of Contributions	85
5.5 Key Findings	85
5.5.1 Architectural and Modeling Insights	85
5.5.2 Domain and Class-Level Impact	86
5.6 Limitations	86
5.7 Future Work	86
Bibliography	88
List of Figures	95
List of Tables	97
List of Abbreviations	100
A Attachments	103
A.1 Test Split Analysis	105
A.1.1 Entity Test Statistics	105
A.2 Linked Entity Creation	109
A.2.1 Entity Recognition	109
A.2.2 Span Normalization	110
A.2.3 Linking to the Knowledge Base	110
A.2.4 Disambiguation	111
A.2.5 Derived Entities	111
A.2.6 Aggregation and Relevance Scoring	111
A.2.7 Influence on Disambiguation and Relevance	112
A.3 Experiment 2: Attachments	114
A.4 Comparison of Results on Our Data Split and the Old Data Split	120

Introduction

0.1 Context and Background

Text classification is a fundamental task in Natural Language Processing (NLP), essential for organizing and navigating the large volume of news produced daily. While current generative models demonstrate strong performance on this task, they are computationally expensive and slow. Therefore, a custom classification pipeline remains necessary when processing substantial amounts of data under operational constraints.

Within the media industry, the International Press Telecommunications Council (IPTC) Media Topics taxonomy serves as the standard framework for categorizing news content. This taxonomy spans various domains, including sports, politics, and the arts. However, classifying articles into this framework is challenging due to its hierarchical structure and its extensive label space, which consists of over 1,200 distinct categories, such as soccer, elections, dance and many others.

While modern dense document embeddings effectively capture general multilingual semantic context, they often lack the explicit factual knowledge required to correctly categorize articles that rely heavily on specific, rare, or evolving named entities. For example, consider the sentence: *“Filip Jicha attended yesterday’s match to support his former club.”* The sentence itself does not reveal which sport is being discussed. Identifying the entity *Filip Jicha* as a known handball player provides the factual knowledge necessary to identify the topic as handball.

0.2 Motivation and Problem Statement

This thesis investigates the enhancement of an existing cross-lingual, multi-label news classifier by incorporating real-world knowledge of named entities. While modern text classification models achieve strong performance using contextual text embeddings alone, they typically rely on the assumption that entity representations are learned implicitly during pre-training. Consequently, these models rarely exploit structured knowledge from external knowledge bases.

We hypothesize that many rare or emerging entities remain unknown to purely text-based models. Furthermore, entity attributes and real-world contexts can change over time (e.g., a professional athlete retiring and transitioning into a public role), altering how they are framed in text. Therefore, explicit entity information can provide vital supplementary signals for classification. Structured metadata extracted from comprehensive knowledge bases, such as Wikidata or Wikipedia, can offer the explicit semantic grounding that textual embeddings miss.

The core problem addressed by this research is determining whether integrating structured entity knowledge improves the accuracy of assigning IPTC Media Topics to news articles compared to models that rely solely on textual representations. The classification system operates within a multilingual environment, processing a training corpus of more than 100,000 news articles primarily in English, German, Dutch, and French. To this end, the enhanced classifier uses three primary inputs for each article:

- The raw article text.
- Language-aligned dense document embeddings.
- Recognized named entities linked to a structured knowledge base, utilizing Wikidata identifiers and Wikipedia links where available.

0.3 Research Objectives and Contributions

To address our problem, we focus on developing a computationally efficient late-fusion architecture that uses existing Named Entity Recognition (NER) pipelines and external knowledge bases, specifically Wikidata and Wikipedia.

This thesis provides the following core contributions to the field of multi-label text classification:

- **A Modular Late-Fusion Architecture:** We propose and evaluate an architecture that successfully integrates pre-computed entity embeddings with multilingual text representations. This approach avoids the high computational overhead associated with fine-tuning entire Large Language Models.
- **ClearML Pipeline:** We implement a ClearML pipeline to log our experiments in detail and ensure their reproducibility.
- **Optimization of Entity Aggregation:** We empirically demonstrate that a learned attention mechanism is the superior strategy for pooling entity embeddings. This mechanism effectively filters semantic noise while bounding the magnitude of the resulting vector.
- **Overall Classification Improvement:** We demonstrate that explicit entity injection significantly improves classification performance, yielding a 1.7% increase in Micro F_1 score compared to the text-only baseline.
- **Mitigation of Long-Tail Sparsity:** We show that integrating entities substantially improves classification accuracy for sparse classes. Our optimal configuration achieves a 2.4% increase in Macro F_1 score overall, and a 3.6% increase specifically for small classes, compared to the text-only baseline.
- **Broad-Scale Hierarchical Evaluation:** While existing literature typically restricts evaluation to the 17 top-level IPTC categories, we evaluate our approach on 345 classes. This imbalanced setup, with test support ranging from 15 to over 3,000 samples, closely mirrors its real-world deployment in media production.
- **Domain-Specific Performance Insights:** We reveal that entity enhancement is effective for specific domains such as sports. Conversely, we identify structural limitations in ambiguous domains like conflict and war reporting.

0.4 Thesis Outline

The remainder of this thesis is organized as follows:

- **Chapter 1 - Background and Theoretical Foundations** provides the theoretical background and mathematical foundations for multi-label text classification, the IPTC taxonomy, knowledge graphs, and representation learning.
- **Chapter 2 - Related Work** reviews related work, distinguishing between knowledge-aware models, knowledge-injected pipelines, and previous approaches to entity-augmented text classification.
- **Chapter 3 - Entities and Data** outlines the data and entities, detailing the multilingual news corpora, the entity extraction pipeline, and the properties of the gold evaluation datasets.
- **Chapter 4 - Methodology** details the methodology, explaining the ClearML system architecture, feature fusion techniques, entity pooling strategies, and the design of the experiments.
- **Chapter 5 - Results** presents the evaluation results, comparing the baseline text-only model against various entity-enhanced configurations to quantify the impact of external knowledge on classification performance.
- **Conclusion** summarizes the findings, discusses limitations, and outlines directions for future work.

1 Background and Theoretical Foundations

This chapter introduces the theoretical concepts and mathematical foundations necessary for the methodology of this thesis. It begins by formalizing the multi-label text classification task and introducing the external knowledge structures used. It then details the mechanisms of representation learning, neural network architectures, and the evaluation metrics required for imbalanced datasets.

1.1 Hierarchical Multi-label Text Classification

Text classification is a foundational task in Natural Language Processing (NLP) that maps unstructured text documents to predefined categories. While classic single-label classification systems assign exactly one label y from a disjoint set of categories Y to each document, real-world applications, such as news categorization, frequently require multi-label classification (MLC). Formally, given a document x and a label space of size L , the goal of MLC is to learn a mapping to a binary label vector $\mathbf{y} \in \{0, 1\}^L$, where each element y_i indicates the presence or absence of the i -th category.[1]

In many real-world applications, target categories are naturally organized into structured taxonomies rather than flat, independent lists. Hierarchical Multi-Label Classification (HMLC) addresses the task of assigning one or more paths of a structured label hierarchy to a given document. These hierarchies are typically formalized in one of two ways:[2]

- **Tree Structures:** Each category has exactly one parent node. A real-world example is the categorization of IPTC news topics (e.g., *Sport* $\rightarrow$ *Team Sport* $\rightarrow$ *Handball*).
- **Directed Acyclic Graphs (DAGs):** A node can have multiple parent nodes. This is frequently used in bioinformatics, such as the Gene Ontology, where a specific biological function might descend from multiple broader processes.

A key constraint in HMLC is the *hierarchical consistency property*. Formally, if a document is assigned to a child label y_i , it must also be implicitly assigned to its parent label $y_{parent(i)}$:

$$y_i = 1 \implies y_{parent(i)} = 1, \quad \forall i \in \{1, \dots, L\} \quad (1.1)$$

Historically, text classification evolved from manual heuristic systems to machine learning algorithms (such as Naive Bayes and Support Vector Machines) using sparse representations like Bag-of-Words or TF-IDF. However, these traditional methods struggle to capture word order and semantic context. Modern approaches to HMLC use dense vector representations and deep neural networks to model document semantics and dependencies effectively [3, 4].

1.1.1 IPTC Media Topics Taxonomy

The target label space for our classification task is the IPTC Media Topics taxonomy. Maintained by the International Press Telecommunications Council, it is the industry-standard subject taxonomy for categorizing news content. [5]

The taxonomy is structured as a hierarchical tree with a depth of up to five levels, covering broad domains. Each domain is represented by one of the 17 top-level categories: Arts, Culture, Entertainment and Media; Conflict, War and Peace; Crime, Law and Justice; Disaster, Accident and Emergency Incident; Economy, Business and Finance; Education; Environment; Health; Human Interest; Labour; Lifestyle and Leisure; Politics and Government; Religion; Science and Technology; Society; Sport; and Weather. These domains branch into more than 1,200 specific leaf terms. Examples include musical genres (e.g., hip hop, jazz), meteorological disasters (e.g., hurricane, avalanche), physics (e.g., nuclear physics, electromagnetism), and competition disciplines (e.g., soccer, floorball) [5].

Due to this hierarchical structure, an article assigned to a specific low-level category implicitly inherits all of its parent categories. This property naturally creates a long-tailed label distribution, where top-level concepts appear frequently while highly specific leaf nodes suffer from data sparsity. [5]

1.2 Knowledge Bases and Entities

To improve standard text classification, external structured knowledge about entities can be injected into the classification model. This process requires identifying concepts within the text and linking them to a formalized knowledge base. In this thesis, we use the internal Geneea knowledge base, which allows us to link extracted entities to their corresponding Wikidata items and Wikipedia articles.

1.2.1 Entities

In NLP, an entity is a uniquely identifiable object or concept that exists in the real world or within a specific knowledge domain. Entities can represent people, organizations, locations, events, products, or abstract concepts, although the granularity of these types can vary. In text, entities are typically represented by mentions, which are specific textual expressions referring to these underlying objects. For example, consider the text: *“Apple released a new iPhone. The company reported record revenue.”* In this context, the mentions *“Apple”* and *“The company”* both refer to the same entity: Apple Inc. [6]

We can divide entities into multiple types:

- **Core (Named) Entities:** The most common predefined categories representing real-world objects, such as persons, organizations, and locations.
- **Numerical and Quantifiable Entities:** Entities representing measurable values, such as dates, times, monetary amounts, percentages, and numerical quantities.
- **Product and Object Entities:** Names of commercial products, devices, software, or other manufactured objects.

- **Domain-Specific Entities:** Specialized entity types defined for particular application domains, such as biomedical entities (e.g., diseases or medications) or legal entities (e.g., legislation or courts).
- **Structural and Syntactic Variations:** More complex entity representations, including nested entities and structured information such as contact details (e.g., “*Cambridge*” as a location inside an entity with type organization: “*University of Cambridge*”).

More information about entities used in this thesis and their source is given in Section 3.1. [7, 6]

1.2.2 Named Entity Recognition (NER) and Entity Linking (EL)

NER is the process of locating and classifying named entities within a text into predefined semantic types, such as persons, organizations, or locations. Because a single word form (e.g., “Columbia”) can refer to multiple distinct concepts (such as a river, a university, or a space shuttle), NER is typically followed by **entity disambiguation** and **(EL)**. This process maps the ambiguous textual span to a unique identifier in a target knowledge base. [8]

For example, in the sentence “*Douglas Adams’s work continues to inspire new generations of readers,*” NER detects the entity *Douglas Adams*. This entity is then mapped through EL to its corresponding Wikidata identifier, *Q42*. Using this identifier, we can query Wikidata to retrieve the entity’s description: “*British science fiction writer and humorist (1952–2001).*”

1.2.3 Wikipedia and Wikidata

In modern NLP, Wikipedia and Wikidata frequently serve as foundational knowledge bases for entity linking and semantic analysis. Together, they provide a large combination of unstructured text and structured relational data.

Wikipedia

Wikipedia is a free, multilingual online encyclopedia maintained by a global community of volunteers. As the largest reference work in history, it operates in over 340 languages and contains more than 66 million articles, with the English edition accounting for over 7 million pages [9].

Beyond its textual content, Wikipedia functions as a dense, scale-free network. Articles are interconnected via wikilinks (internal hyperlinks) that connect a mention of a concept in one article to its corresponding definition page within the same language domain. In graph terms, articles act as nodes and wikilinks serve as directed edges, providing semantic context and structural features for NLP models [10].

Wikidata

Wikidata acts as a structured, machine-readable counterpart to Wikipedia [11]. The fundamental unit of data in Wikidata is the *item*, which represents a single

topic, concept, or entity (e.g., a person, place, or historical event). Each item is assigned a unique, language-agnostic identifier known as a Q-ID (e.g., Q42 for Douglas Adams). This identifier ensures entity stability across different language editions of the Wikimedia ecosystem [12].

The characteristics of an item are described using **statements**. Each statement is structured as a semantic triple consisting of a subject (**item**), a predicate (**property**), and an object (**value**). Properties are also assigned unique identifiers, known as P-IDs, such as *educated at* (P69). Statements can be further contextualized using qualifiers, references, and ranks [12]. Table 1.1 illustrates examples of these triples.

Item (Subject)	Property (Predicate)	Value (Object)
San Francisco (Q62)	head of government (P6)	Gavin Newsom (Q461391)
San Francisco (Q62)	located in (P131)	California (Q99)
California (Q99)	located in (P131)	United States (Q30)

Table 1.1 Examples of Wikidata statements forming a relational chain.

By treating items as nodes and properties as directed edges, these statements form a massive, interconnected **knowledge graph** (KG). Figure 1.1 illustrates this network structure. This graph-structured data is highly valuable for machine learning tasks, as it can be directly processed by Graph Neural Networks (GNNs) or used to train graph embeddings [12].

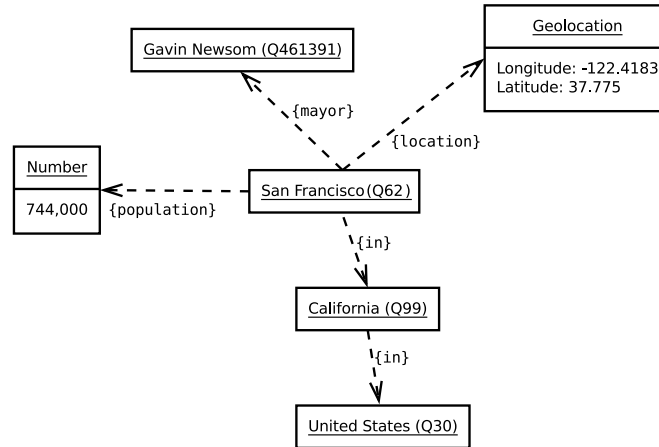


Figure 1.1 Interconnection of Wikidata items via properties within statements [13].

Connection to Wikipedia

While Wikidata items serve as abstract, language-independent representations of concepts, they are explicitly linked to external Wikimedia projects through *Sitelinks* [14]. Sitelinks function as a centralized mapping table stored within each item. This allows a single Wikidata item to act as a central hub that maps its Q-ID to corresponding articles across different language editions of Wikipedia. For example, the item for *Inflation* (Q35805) directly bridges the English (*Inflation*), German (*Inflation*), Finnish (*Inflaatio*), and Czech (*Inflace*) articles, enabling cross-lingual entity linking and alignment. [14]

1.3 Representation Learning

Machine learning models require numerical input. Representation learning transforms raw text into dense, continuous vectors of fixed dimensionality, known as **embeddings**.

1.3.1 Text Embeddings

Text embeddings provide a dense numerical representation of article content and form the primary input to the classification pipeline. This subsection summarizes how modern transformer encoders map text into fixed-dimensional vectors suitable for downstream multi-label prediction. Transformer-based architectures have revolutionized text representation. Instead of mapping individual words in isolation, models like Sentence-BERT [15] encode entire sentences or paragraphs into a single dense vector, capturing complex semantic meaning and multilingual context. For an article chunk c_i , the encoder produces an embedding vector $e_i \in \mathbb{R}^d$, where d is the embedding dimension.

1.3.2 Entity Embeddings

Similar to text, entities from a knowledge graph can be mapped into a continuous vector space. Entity embeddings can be generated purely from their structural relationships in the graph (e.g., via Graph Neural Networks), from their textual descriptions (using language models), or via joint training objectives that align both structural and textual vocabularies into a single shared space (e.g., Wikipedia2Vec[16]).

1.4 Neural Network Architectures

This section introduces the neural building blocks used in the downstream classification head. It reviews feedforward layers, non-linear activations, regularization, and mechanisms for fusing article and entity representations.

1.4.1 Multilayer Perceptron (MLP)

The Multilayer Perceptron is a class of feedforward artificial neural network. It consists of an input layer, one or more hidden layers, and an output layer. In a standard linear layer, the input vector $\mathbf{x}$ is transformed using a learned weight matrix $\mathbf{W}$ and a bias vector $\mathbf{b}$:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

1.4.2 Activation Functions

To model complex, non-linear relationships, an activation function is applied to the output of the linear transformation.

- **ReLU (Rectified Linear Unit):** The most common activation function, defined as $f(x) = \max(0, x)$. While computationally efficient, it can suffer

from the "dying ReLU" problem, where neurons permanently output zero for negative inputs [17].

- **GELU (Gaussian Error Linear Unit):** A smoother alternative to ReLU, defined as $f(x) = x \cdot \Phi(x)$, where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution [18]. GELU preserves a non-zero gradient for negative values, which can aid in learning rare features.

1.4.3 Regularization

Dropout is a regularization technique used to prevent overfitting. During training, it randomly zeroes out a fraction of the neurons in a layer with probability p . This prevents the network from relying too heavily on specific pathways, forcing it to learn redundant and robust representations.

1.4.4 Feature Fusion and Attention

When article-level and entity-level representations are available, the model must combine them into a single feature vector for classification. This subsection reviews static pooling strategies and attention-based fusion mechanisms used by the entity-aware architecture. When a model receives multiple streams of input (e.g., an article embedding and a set of entity embeddings), these features must be fused. **Pooling** operations summarize a set of vectors into a single vector of fixed size (e.g., taking the mean or sum across all entity vectors).

The **attention mechanism** was originally introduced in neural machine translation to allow models to dynamically focus on specific parts of an input sequence rather than compressing all information into a single fixed-length vector. In general, an attention mechanism computes alignment scores between a set of keys and a query, transforms these scores into a probability distribution, and computes a weighted sum over the corresponding values. [19, 20]

Several foundational types of attention exist in the literature. Additive attention (or Bahdanau attention) uses a single-hidden-layer feed-forward network to compute the relationship between vectors [20]. Multiplicative (or dot-product) attention simplifies this mechanism by computing the inner product of the vectors, reducing computational complexity [21]. Multi-Head Attention (MHA) extends this concept by executing multiple scaled dot-product attention operations in parallel, allowing the model to jointly attend to information from different representation subspaces [19].

In this thesis, we implement a custom entity attention mechanism tailored for feature fusion. Unlike the static heuristic pooling strategies described above, this attention-based variant dynamically learns to weight entity importance during training. This attention mechanism is described in more detail in Section 4.7.4 in the Methodology chapter.

1.5 Training and Evaluation

Having defined the model components, we now describe how the classifier is trained and how its predictions are assessed. Because the label space is large and

highly imbalanced, both the loss function and evaluation metrics require careful specification.

1.5.1 Loss Function

For multi-label classification, the problem is treated as a series of independent binary classification tasks. The network outputs raw logits $\mathbf{z}$, which are squashed into independent probabilities $\hat{y} \in (0, 1)$ using the **Sigmoid** function:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The model is optimized using **Binary Cross-Entropy (BCE) Loss**, which calculates the error between the predicted probabilities $\hat{y}$ and the ground truth labels $y \in \{0, 1\}$:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

1.5.2 Evaluation Metrics

Due to the extreme sparsity of multi-label datasets, accuracy is an inadequate metric. Instead, evaluation relies on True Positives (TP), False Positives (FP), and False Negatives (FN).

Precision measures the exactness of the classifier, answering what proportion of predicted positive labels were actually correct:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall measures the completeness of the classifier, answering what proportion of actual positive labels were successfully identified:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

The **F1-score** is the harmonic mean of Precision and Recall, providing a single metric that balances both concerns:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

To aggregate these metrics across hundreds of classes, two averaging strategies are employed:

- **Micro-Averaging:** Aggregates the global counts of TP, FP, and FN across all classes before calculating the final metric. This treats every individual prediction instance equally, meaning the final score is dominated by the most frequent classes.
- **Macro-Averaging:** Calculates the metric independently for each class and then takes the unweighted mean of those scores. This treats every class equally, heavily emphasizing the model's performance on rare, long-tail categories.

1.6 Statistical Testing

Point estimates of classification metrics do not show whether observed differences between models are statistically reliable. This section introduces the paired comparison and multiple-testing procedures used later to assess per-class changes.

1.6.1 McNemar’s Test

McNemar’s test is a non-parametric statistical test used to compare the performance of two classifiers evaluated on the same test set. It analyzes the individual predictions of both classifiers and determines whether the observed difference in performance is statistically significant. [22]

The test is based on the number of discordant predictions. Let n_{01} denote the number of instances correctly classified by the first classifier but misclassified by the second, and n_{10} the number of instances correctly classified by the second classifier but misclassified by the first. The null hypothesis assumes that both classifiers have the same probability of making a correct prediction. The test statistic with continuity correction is computed as

$$\chi^2 = \frac{(|n_{01} - n_{10}| - 1)^2}{n_{01} + n_{10}}, \quad (1.2)$$

which approximately follows a chi-squared distribution with one degree of freedom. A p -value below the chosen significance level indicates that the difference between the classifiers is statistically significant. McNemar’s test is commonly used in machine learning because it accounts for the paired nature of classifier predictions. In multiclass or multilabel classification, it is typically applied independently to each class using a one-versus-rest formulation. [22]

1.6.2 False Discovery Rate (FDR) Correction

Applying McNemar’s test independently to each IPTC category yields many simultaneous hypothesis tests, increasing the risk of false positives. Under a fixed significance level α (commonly 0.05), each individual test has a non-negligible chance of incorrectly rejecting the null hypothesis. Across hundreds of classes, these Type I errors accumulate, so the set of “significant” results is expected to contain many false positives if no correction is applied.

Classical multiple-testing procedures such as Bonferroni control the *family-wise error rate* (FWER), that is, the probability of making at least one false rejection. This control is often overly conservative when the number of tests is large, because it sharply reduces statistical power. An alternative is to control the *False Discovery Rate* (FDR), defined as the expected proportion of false rejections among all rejected hypotheses. [23]

The most widely used FDR procedure is the Benjamini–Hochberg (BH) method. [23] Given m hypothesis tests with p -values $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$ ordered from smallest to largest, and a target FDR level q , the BH procedure finds the largest index k such that

$$p_{(k)} \leq \frac{k}{m} q \quad (1.3)$$

and rejects the null hypotheses corresponding to $p_{(1)}, \dots, p_{(k)}$. Equivalently, each raw p -value can be converted into an FDR-adjusted p -value; a result is then declared significant if its adjusted p -value is below q . Beyond machine learning, FDR control is widely used in genomics and bioinformatics, for example when testing thousands of genes for differential expression or genetic association. [24]

In this thesis, FDR correction is applied to the per-class McNemar p -values when comparing the entity-enhanced classifier with the baseline. This allows us to identify classes with statistically significant performance changes while limiting the expected fraction of spurious discoveries among the reported significant results (see Section 5.3.1).

2 Related Work

This chapter reviews prior work on incorporating entity knowledge into NLP systems, with emphasis on approaches relevant to multilingual news classification. It surveys knowledge-aware and knowledge-injected language models, entity-augmented classification pipelines, and available entity embedding resources before identifying the research gap addressed by this thesis.

2.1 Previous Experiments with NLP Models

Current state-of-the-art embedding models, such as Qwen3-Embedding [25], Gemini Embedding [26], NV-Embed [27], and BGE-M3 [28], do not use NER or external knowledge bases to enhance text embeddings. This design choice stems from the ability of these models to learn entity relations directly from large-scale training data, as shown by [29], thereby to avoiding the computational overhead of external pipelines the NER is not used.

However, we hypothesize that a significant number of rare or emerging entities remain unknown to these models. Furthermore, entity attributes and contexts can change over time (e.g., a professional athlete retiring and transitioning into a local star), altering how they are mentioned in text. Therefore, we assume that explicit entity information can still provide valuable signals for our classification model. To test this, we focus on methods that use NER and entity linking to improve our embedding pipeline.

Pre-training a custom knowledge-injected embedding model from scratch is computationally expensive. Therefore, rather than training a new base model, this thesis adapts the practices from some of these models into the downstream classification architecture.

Previous attempts demonstrate that adding structured knowledge to pre-trained language models improves performance on knowledge-intensive tasks. These entity-aware approaches generally fall into two categories based on their inference requirements:

- **Knowledge-Aware Models:** These models internalize knowledge during training and do not require an active NER pipeline during inference.
- **Knowledge-Injected Models:** These models extract and inject structured knowledge dynamically, requiring explicit NER pipelines during both training and inference.

2.1.1 Knowledge-Aware Models

This section explores models that integrate entity knowledge strictly during the training or pre-training phase. Because these architectures do not require an explicit NER module or an external Knowledge Graph (KG) lookup during inference, they maintain the fast processing speeds of standard language models. Instead, they internalize factual knowledge into their parameters by solving multi-task objectives that include entity-centric tasks. Since this thesis focuses on generating entity representations rather than training new models from scratch,

we do not use the techniques of these specific architectures. However, they are discussed here to provide a comprehensive review of the literature.

Entity Representations During pre-training, these models process entities in several formats to learn structural relationships:

- **Knowledge Graph Triples:** Entities are represented as nodes in a graph, formatted as (h, r, t) triples (e.g., (Marie Curie, occupation, physicist)). Models use translation-based objectives to embed these relationships [30, 31].
- **Textual Spans:** Entities are treated as continuous sequences of text tokens. Knowledge is learned by masking these spans entirely, forcing the model to predict the entity based on the surrounding context [32].
- **Contrastive Anchors:** Entities serve as anchor points in contrastive learning, where the model must discriminate between true relational facts and implausible facts [33].

Common Training Tasks and Objectives

- **Masked Language Modeling (MLM):** The standard objective where a sequence of tokens X is partially masked, and the model predicts the missing tokens:

$$L_{MLM} = - \sum \log P(x_i | x_{\setminus i})$$

- **Knowledge Embedding (KE) Loss:** Often using TransE scoring, the model minimizes the distance between the head plus relation and the tail entity:

$$L_{KE} = \sum \max(0, d(h + r, t) - d(h' + r, t') + \gamma)$$

- **Entity/Relation Discrimination:** Contrastive tasks where the model differentiates between correct entity connections and negative samples.

Important Knowledge-Aware Models

- **ERNIE (Baidu) [32]:** ERNIE masks entire entities and phrases as single units. By predicting the complete entity from context, it implicitly learns relational knowledge without needing external graph vectors.
- **ERICA [33]:** Enriches Pre-trained Language Models (PLMs) via contrastive learning using distant supervision and two auxiliary tasks: entity discrimination and relation discrimination.
- **KEPLER [30]:** Optimizes the MLM and KE objectives simultaneously using a shared RoBERTa encoder, aligning textual semantics with graph structures.
- **JAKET [31]:** Jointly models the KG and language model. The language model provides context for the KG, while the KG provides factual grounding for the language model.

- **K-Adapter [34]:** Solves forgetting by keeping the base language model’s parameters frozen. It introduces small, independently trainable neural network modules (adapters) for distinct types of information.

2.1.2 Knowledge-Injected Models

In contrast to knowledge-aware architectures, knowledge-injected models require an active NER and entity linking (EL) pipeline during both the training and inference phases. These models extract structured knowledge directly from external knowledge bases and inject it into the language model to provide factual grounding.

A primary limitation of traditional knowledge-injected models is the computational overhead introduced by explicit NER and EL, which typically require multi-stage inference pipelines.

Combination and Fusion Mechanisms To merge textual context with structured entity representations, models employ various architectural techniques:

- **Concatenation or addition at the Input Layer (e.g., E-BERT [35]):** Pre-trained entity vectors are mapped into the same vector space as the model’s native word embeddings and concatenated or added at the initial input layer of the BERT model.
- **Direct Injection and Structural Alignment (e.g., K-BERT [36]):** Knowledge graph triples are explicitly injected into the input text, transforming the linear sentence into a “sentence tree.”
- **Entity-Aware Self-Attention (e.g., LUKE [37]):** Computes attention scores using distinct weight matrices based on the types of interacting tokens (e.g., word-to-entity, entity-to-entity).
- **Multi-Stream Fusion Aggregators (e.g., ERNIE Tsinghua [38]):** Processes text and knowledge in parallel, periodically aggregating hidden states via Information Fusion layers.
- **Dynamic Knowledge Attention and Recontextualization (KAR) (e.g., KnowBERT [39]):** Acts as a mid-network fusion strategy, dynamically retrieving relevant entity vectors via attention and updating textual representations.

Explicit entity representations consistently improve tasks requiring factual knowledge or structured semantic understanding (e.g., relation extraction, entity typing). However, they provide little benefit on general language understanding benchmarks (e.g., GLUE [40]), where contextual language models already capture sufficient lexical information.

2.2 Entity-Augmented Text Classification

This section summarizes prior work that incorporates entities strictly to improve classification pipelines, rather than to train entirely new entity-aware embedding models.

- **Entity Replacement for Privacy-Preserving Text Classification**

K. Mohammed [41] propose a pipeline that uses NER to preserve privacy and reduce feature space dimensionality. Original entity names are replaced with their categorical labels (e.g., John becomes PERSONentity). The text is then embedded using a bag-of-words (BoW) feature extractor and classified using a Support Vector Machine (SVM). The impact on classification accuracy varied by domain: it improved accuracy in legal texts but hurt performance in subjective contexts such as movie reviews.

- **Entity-Augmented Convolutional Neural Networks**

Bie. Qingyuan [42] introduced EntCNN, designed for short text classification. The model concatenates pre-trained word vectors with pre-trained entity vectors (retrieved via Wikipedia2Vec or Doc2Vec). The combined embeddings are fed into a single-layer CNN. Adding entities was particularly beneficial for multi-class categorization tasks, though it yielded minimal improvement on sentiment-based datasets.

- **Neural Attentive Bag-of-Entities (NABoE)**

Y. Ikuya [43] introduced NABoE, leveraging pre-trained entity embeddings and an attention mechanism. The model computes a word-based representation and an entity-based representation, combining them via simple addition. The model achieved state-of-the-art results on factoid question answering and standard text classification

2.3 Entity Embedding Sources

After an entity is detected and linked, its embedding is typically generated using two primary knowledge bases: Wikidata and Wikipedia. Because Wikidata items are often explicitly connected to their corresponding Wikipedia articles, both resources can be used to create entity representations. This section first describes these two foundational sources, followed by a description of two key projects that process them to generate entity embeddings.

2.3.1 Wikidata Embedding Project

The Wikidata Embedding Project is an open-source data infrastructure initiative led by Wikimedia Deutschland in collaboration with Jina AI and DataStax. [44] Officially launched on October 1, 2025, the project’s primary objective is to enhance the query capabilities of Wikidata by integrating vector-based semantic search. By converting structured graph items into dense vector spaces, the project aims to support the open-source artificial intelligence community in building applications such as Retrieval-Augmented Generation (RAG) systems, allowing large language models to interact directly with Wikidata’s multilingual knowledge base. [44]

Methodologically, this framework generates text-based embeddings of knowledge graph entries. It converts structured Wikidata items (identified by unique QIDs) into text representation by concatenating their official labels, textual descriptions, and associated language aliases. These synthesized strings are then

embedded using Jina AI’s default recommended engine, `jina-embeddings-v3`. Released as a state-of-the-art multilingual embedding model, `jina-embeddings-v3` natively supports over 100 languages and features an extended context window of up to 8,192 tokens. Because it is an infrastructure project rather than a novel neural architecture, its primary contribution is data accessibility rather than a new machine learning algorithm.[45, 46]

The official project ecosystem provides out-of-the-box configuration files, data pipelines, and pre-computed vector databases hosted via DataStax, primarily for English (`en`) and German (`de`) dumps. However, because the underlying embedding engine is multilingual, the pipeline can be programmatically extended to other languages. For this thesis, custom extraction and ingestion scripts were implemented to generate corresponding item embeddings for Czech (`cs`), Dutch (`nl`), Spanish (`es`), and French (`fr`) [47].

An example text representation for an entity is:

```
Douglas Adams, English writer and humorist, also known as
Douglas Noël Adams, DNA. Attributes include:
- instance of: "human"
- occupation: "writer",
  "screenwriter"
- country of citizenship: "United Kingdom"
- date of birth: "11 Mar 1952"
- date of death: "11 May 2001"
```

2.3.2 Wiki2vec

The tool `wiki2vec` is an open-source utility developed by the software company idio, originally released around 2015 and archived on GitHub as a read-only repository in February 2021 [48]. The purpose of this project was to simplify the generation of continuous word and entity vectors directly from raw Wikipedia XML dumps.

The tool operates by parsing the markdown text of Wikipedia articles and identifying hyperlinked anchor text. It automatically substitutes these hyperlinks with their corresponding DBpedia resource identifiers; for example, a link pointing to the article about Prague is converted into the explicit string token `DbpediaID/Prague`. Once the entire text dump is flattened and transformed into a token stream where text words and entity IDs coexist, the corpus is passed directly into standard word embedding algorithms, specifically the Skip-gram or Continuous Bag-of-Words (CBOW) architectures implemented in the Gensim library. [48]

This implementation represents a legacy approach and does not reflect the current state of the art. It treats entities identically to standard vocabulary words, completely ignoring the structural topology of the underlying knowledge graph or page-link hierarchies. No official pre-trained models are maintained for this repository. It is included in this literature review solely to establish a clear distinction from the more advanced `wikipedia2vec` framework.[48]

2.3.3 Wikipedia2vec

Developed and maintained by Studio Ousia, Wikipedia2Vec is an open-source toolkit designed to learn joint embeddings of words and entities from Wikipedia within a unified continuous vector space. First proposed by Yamada et al. [49] in 2016 and expanded as an optimized toolkit in the Wikipedia2Vec article [16] in 2020, the framework bridges the gap between unstructured textual representations and structured encyclopedic concepts.

Unlike standard word embedding tools or legacy pipelines that treat entity tokens as flat text, Wikipedia2Vec uses an extended skip-gram architecture that simultaneously optimizes three distinct sub-objectives during training:

1. **Word-based Skip-gram:** Learns standard linguistic contexts by predicting neighboring words given a target word within a sliding text window.
2. **Wikipedia Link Graph:** Learns entity-to-entity structural relationships by treating Wikipedia articles as nodes and hyperlinks as edges, optimizing the model to predict neighboring entities in the graph topology.
3. **Word-Entity Co-occurrence:** Aligns both vocabularies into a single shared space by predicting neighboring text words given a target entity anchor, and vice versa.

The project provides officially distributed, pre-trained binary and text embedding files for 12 major languages, including English, German, French, Dutch, and Spanish. However, official pre-trained models are not provided for languages with smaller text footprints, such as Czech (`cs`). For these unsupported languages, the project provides an executable command-line interface that allows users to train custom models from scratch using raw Wikipedia database dumps. [50]

While standalone static embeddings are no longer the absolute state of the art compared to deep contextual language models (e.g., BERT[51] or RoBERTa[52]), Wikipedia2Vec remains a highly efficient baseline for knowledge-injection and text classification. The utility of this model has been verified across several closely related projects. For instance, B. Qingyuan [42] introduced a neural short-text classification framework utilizing the “Words are not enough!” paradigm, combining standard textual vectors with entity embeddings derived from Wikipedia2Vec to resolve contextual ambiguities. Furthermore, Nishikawa and Yamada et al. [53] extended this concept into the cross-lingual domain by proposing a Multilingual Bag-of-Entities (M-BoE) architecture. By mapping language-specific Wikipedia entities back to language-agnostic Wikidata identifiers, they demonstrated that sharing these underlying entity spaces significantly improves zero-shot cross-lingual document classification.

2.4 Research Gap

In this chapter we focus on models that uses NER systems to inject structured knowledge directly into the internal architecture of neural networks. Notable examples include architectures such as LUKE [37], KnowBERT [39], and K-BERT [36], which modify the internal transformer layers or attention mechanisms

to align textual representations with corresponding entity vectors from knowledge bases.

In contrast, contemporary Large Language Models (LLMs) used for generating text embeddings do not employ these explicit injection mechanisms. Instead, they rely on implicitly learning semantic relationships between entities from large text corpora during pre-training. This paradigm requires substantial computational resources and often underperforms in domain-specific tasks due to the lack of explicit semantics.

To address these limitations, recent approaches have attempted to integrate entity knowledge directly into classification pipelines without the need for computationally expensive fine-tuning of the entire language model. However, existing methodologies exhibit several gaps:

- **Lack of Application to large hierarchical multilingual classification:** Current methods do not address complex categorization schemes such as the standardized media topics defined by the IPTC(Section 1.1.1), which are essential for global media monitoring and news analysis.
- **Absence of Synergy Between Modern Embeddings and Integrated Knowledge:** Existing systems fail to effectively combine state-of-the-art embedding models with deeply interconnected semantic entity knowledge. Instead, they are often limited to replacing words with generalized labels (e.g., `PERSON_entity`) in bag-of-words representations or employing simple ensembling of NER tag frequencies with text predictions.
- **Unexplored Potential of the Wikidata Embedding Project:** None of the current approaches have evaluated the utility of the recently launched Wikidata Embedding Project, despite evidence from projects like Wikidata5M demonstrating that linking textual descriptions with Wikidata embeddings significantly enhances semantic accuracy in tasks such as link prediction. [30]
- **Insufficient Research on Entity Influence Across Media Topics:** There is a total absence of systematic testing in current literature regarding how explicit entity representations affect classification accuracy across diverse news and media domains (e.g., sports, culture, politics) within a multilingual environment.

This Master’s thesis directly addresses these gaps by proposing and evaluating a modular late-fusion architecture that integrates pre-computed entity embeddings into a multilingual text classification pipeline. The research focuses on isolating feature fusion and pooling strategies (including advanced attention mechanisms) from heavy text-encoder layers. This approach enables the model to benefit from rich entity semantics effectively, completely avoiding the computationally expensive updating of entire language models.

3 Entities and Data

This chapter describes the data and entities used in our experiments, along with their preprocessing. First, we define the entities and explain how they are detected. Next, we outline the characteristics of the collected news articles, including their origins, structural differences, and preprocessing. The resulting dataset contains over 110,000 articles from 17 corpora, written in 6 languages, and published between 1969 and 2023.

The data for this project was provided by Geneea. For legal reasons, the full dataset cannot be published. However, we provide a data sample to demonstrate that the processing and classification pipelines function correctly, along with a description of the data in this chapter.

3.1 Geneea Entities and Their Source

This section defines the entity inventory used in the experiments and describes how Geneea detects, classifies, and links mentions to its knowledge base. It first introduces the entity types and linking outcomes before detailing the extraction pipeline. As described in Section 1.2.1, an entity is a uniquely identifiable object or concept that exists in the real world or within a specific knowledge domain. In this thesis, we categorize entities into the following types:

- **Person** – A named individual, real or fictional.
Examples: Barack Obama, Marie Curie, Sherlock Holmes.
- **Location** – A geographical place, region, or physical landmark.
Examples: Prague, Mount Everest, the Sahara Desert.
- **Product** – A commercial item, software, or branded offering.
Examples: iPhone, ChatGPT, Coca-Cola.
- **Event** – A named occurrence, happening, or occasion.
Examples: World War II, the 2024 Olympics, Woodstock Festival.
- **Organization** – A company, institution, government body, or group.
Examples: United Nations, Google, Czech Technical University.
- **General** – A catch-all category for named entities that do not fit the above types, such as works of art, laws, diseases, or abstract concepts.
Examples: COVID-19, GDPR, Mona Lisa.

Entity extraction is performed using internal Geneea processes, which include NER and entity disambiguation. We classify the detected entities into two primary groups based on whether they link to the Geneea Knowledge Base (GKB). The first group consists of **linked entities**, which successfully map to a unique entry in the GKB. This study uses only these linked entities, which are further divided by their origin into two subtypes:

- **Direct Entities:** These entities are explicitly identified within the source text and are assigned specific semantic types, such as person, location, organization, product, event, or general.
- **Derived Entities:** These entities do not appear directly in the text. Instead, they are inferred from direct entities using knowledge base relations. For example, mentioning the city “Zlín” triggers the automatic inclusion of “Zlínský kraj” (Region) and “Czech Republic” (Country) to construct a geographic hierarchy (see Figure 3.1). Our dataset includes approximately 3,500 articles containing such geographical derived entities, along with 188 articles containing derived entities that indicate industry types (e.g., finance, mass media, logistics).

The second group consists of **unlinked entities**. These entities do not correspond to any specific item in the GKB. Frequent examples include cardinal numbers (e.g., “one”, “two”), temporal references (e.g., “2020”, “PAST_REF”), and ordinal values. Because these entities primarily represent numerical or temporal noise and cannot be linked to the knowledge base, they are excluded from our experiments.

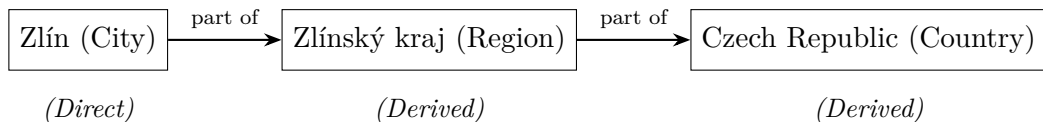


Figure 3.1 Example of a derived entity chain within the geographic hierarchy. Identifying a city explicitly triggers the inclusion of its higher-level administrative regions.

3.1.1 Linked Entity Creation Pipeline

The process of transforming raw text into structured, linked entities is a multi-stage pipeline (see Figure 3.2). It progressively moves from surface-level pattern matching to semantically grounded entities with calculated relevance scores.

Consider the following input sentence:

“Elon Musk announced that Tesla will open a new Gigafactory near Berlin, Germany, in 2026.”

The entity linking pipeline transforms this raw text into structured, knowledge-base-linked entities through several consecutive stages.

Stage 1 — Entity Recognition. The first stage identifies text spans that potentially refer to real-world entities and assigns them coarse semantic types. For the given example, the recognition model extracts the following mentions:

Detected Span	Entity Type	Example
Elon Musk	Person	Person entity mention
Tesla	Organization	Company mention
Gigafactory	Product	Product/object mention
Berlin	Location	Geographic entity mention
Germany	Location	Country mention
2026	General	Date

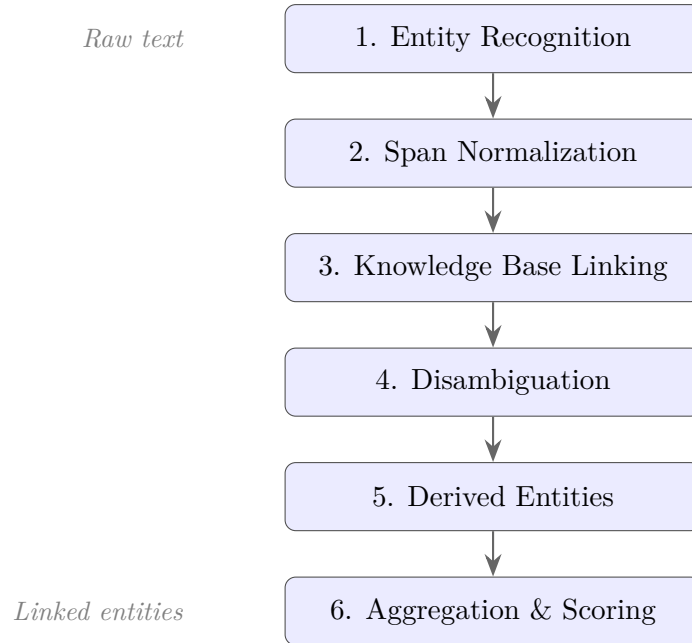


Figure 3.2 Overview of the entity creation pipeline.

Stage 2 — Span Normalization. Detected mentions are converted into canonical forms that better match the representations stored in the knowledge base. In this example, the surface form *Tesla* is normalized to *Tesla, Inc.*, and *Gigafactory* is normalized to *Tesla Gigafactory*. Other mentions, such as *Elon Musk*, *Berlin*, and *Germany*, already correspond to their canonical forms.

Original Mention	Normalized Form
Elon Musk	Elon Musk
Tesla	Tesla, Inc.
Gigafactory	Tesla Gigafactory
Berlin	Berlin
Germany	Germany

Stage 3 — Knowledge-Base Linking. Each normalized mention is matched against candidate entities stored in a knowledge base such as Wikidata. The linker returns possible candidates with associated confidence values. For example, the mention *Tesla, Inc.* may be linked to the Wikidata entity Q478214, while *Berlin* may produce several candidates representing different locations.

Mention	Example Knowledge Base Entry
Elon Musk	Q317521 (entrepreneur)
Tesla, Inc.	Q478214 (automaker)
Berlin	Q64 (capital of Germany)
Germany	Q183 (country in Europe)
2026	NIL (no matching entity)

Stage 4 — Disambiguation. When multiple knowledge-base candidates exist, contextual information is used to select the correct entity by computing a disambiguation score. The entity with the highest disambiguation score wins. In the

example, the mention *Berlin* is resolved to Q64 because the surrounding context contains *Germany*, boosting Q64 over Q1086827 (which is in the US). Similarly, *Tesla* is linked to the automobile company rather than other entities with the same name because it co-occurs with *Elon Musk* and *Gigafactory*.

Stage 5 — Derived Entities. After linking, additional entities can be obtained from relations stored in the knowledge base. For example, from *Tesla, Inc.* the system can derive the entity *United States* through the company-country relation. Similarly, *Berlin* can provide the related entity *Brandenburg* through its administrative-region relation.

Derived Entity	Source Entity	Relation
Brandenburg	Berlin	Located in administrative region

Stage 6 — Aggregation and Relevance Scoring. Finally, all mentions referring to the same entity are merged and assigned a relevance score representing their importance within the document. For the example article, *Tesla, Inc.* and *Elon Musk* receive higher scores because they are central to the article, while derived entities receive lower scores because they are only indirectly mentioned.

Entity	Type	Example Score
Tesla, Inc.	Organization	92
Elon Musk	Person	89
Berlin	Location	74
Germany	Location	61
Brandenburg (derived)	Location	21

The system assigns a *relevance score* ranging from 1 to 100, which quantifies the entity’s prominence within the article. This score is a weighted combination of several factors:

- **Frequency and Position:** Mentions in the title or lead paragraph are weighted heavily, whereas body mentions score lower. Derived entities receive substantially reduced weights.
- **Entity Type and Popularity:** Persons and events generally receive slight scoring bonuses compared to locations or products. General popularity metrics (e.g., Wikipedia prominence) also provide minor boosts.
- **Knowledge Base Presence:** Entities lacking a direct KB link are heavily penalized.
- **Topic Context:** Entities typical for the article’s classified topic receive a boost, while atypical ones are penalized.

The raw weighted sum is normalized using a logarithmic function:

$$s_{\text{norm}} = \log_2 \left(1 + \frac{s_{\text{raw}}}{D} \right) \quad (3.1)$$

where s_{raw} is the raw score and D is the sum of all positive entity scores in the document (computed only over entities with multiple mentions for longer texts). A final score above 80 indicates a central theme, a score of 20 to 80 indicates a standard role, and a score below 20 suggests a peripheral mention. In our experiments, this relevance attribute can be used as a weighting factor for entity embeddings.

3.1.2 Entity Disambiguation

Several factors influence the disambiguation score:

- **Entity Type Matching:** The alignment between the entity type predicted by the machine learning recognizer and the type defined in the GKB.
- **Graph and Frequency Metrics:** Features such as link frequency (the prior probability of linking to a given entity), span source ranking (how well the span was detected as a mention), and link source ranking (how strong the connection to the GKB entity is).
- **Knowledge Base Properties:** Metrics derived from the GKB, including popularity or ranking, context rules, and pre-calculated disambiguation scores.
- **Resolution Mechanics:** Internal mechanisms such as score averaging (how scores are unified across mentions), the handling of overlapping spans (which span wins when they overlap), and penalization for orphan coreferences (entities that are never directly mentioned elsewhere in the article).
- **IPTC Media Topic:** The IPTC media topic classification interacts with the entity pipeline during disambiguation. Specifically, topic classification helps resolve ambiguous entity names by incorporating predicted categories into the scoring rules. For example, in sports-related articles, these rules help the system identify “Munich” as a sports club rather than a geographical region.

3.2 Gold and Silver Data

The current IPTC classifier uses two types of data: the *gold* dataset and the *silver* dataset. These categories are distinguished by their annotator and their role in the development of the classification model.

3.2.1 Gold Dataset

After the data cleaning described in Section 3.2.3, the gold dataset consists of 110,138 news articles manually annotated by humans. These documents have served as the basis for the classifier since its initial version, ensuring a balanced representation of sources and preventing the model from overfitting to specific publication styles.

3.2.2 Silver Dataset

To address the sparsity of some labels in the gold data and to provide more training examples, a silver dataset of almost 80 thousand articles was introduced. These articles were annotated by the existing classifier but with more advanced OpenAI embeddings from either the *Ada* or *Davinci* families [54]. The exact name has not been preserved. Although these more complex embeddings provide higher-quality representations, they cannot be used in production due to their high computational cost and latency. Incorporating the silver data has historically improved the performance of the current model, but the reliability of these automated labels remains a concern.

The silver-annotated corpora consistently show a lower average number of labels per document than their gold counterparts. On average, silver documents contain 0.96 fewer labels than gold documents. The mean number of labels in the silver dataset is 4.95, compared to 5.91 in the gold dataset, a difference of approximately 16.2%.

3.2.3 Dataset Selection

The primary objective of this thesis is to investigate whether integrating Wikidata-structured knowledge can enhance classifier performance. To maintain high experimental integrity, we use only the gold dataset for our experiments. By excluding the silver dataset, we avoid potential label noise caused by incorrectly predicted labels. This ensures that any performance changes are directly due to our methodology, rather than inconsistent labeling. Furthermore, if the inclusion of entities improves overall prediction, we can regenerate the silver data in the future to potentially achieve even better results.

We also excluded the Finnish corpus, which consists of 9,932 articles, due to the poor quality of its extracted entities. Because of the lack of analysis tools for Finnish, the entity analysis yielded a very low success rate, identifying an average of only 2.7 linked entities per article, compared to the overall average of 24.4. Since we cannot rely on the accuracy of these entities, we prioritize data quality to observe the effect of Wikidata features. We therefore excluded the Finnish corpus from the training set to avoid introducing noisy data into the model.

Even after excluding the silver dataset and the Finnish corpus, the remaining dataset still contains 110,138 human-annotated articles. This volume provides a solid foundation for training and evaluating our models, allowing us to determine whether incorporating structured entities is a viable research strategy.

3.3 Selected Data

The list of corpora used in this thesis is shown in Table 3.1. Our dataset consists of articles from 17 distinct corpora across six languages: Czech (cs), German (de), English (en), Spanish (es), French (fr), and Dutch (nl). This section presents the foundational characteristics of each corpus. Because not all data are available, a more detailed analysis is presented in Section 3.6.2.

Table 3.2 provides a quantitative summary of the dataset. There is considerable variance in data volume across sources: the largest corpus, *NoordHB (NL)*,

Shortcut	Description
Mafra (CS)	Articles from Czech news outlets owned by the MAFRA media group [55, 56, 57].
APA (DE)	Articles from Austria Presse Agentur, Austria’s national news agency [58].
DPA (DE)	Articles from Deutsche Presse-Agentur, Germany’s largest news agency [59].
Es (DE)	Articles from the German edition of the international sports media platform [60].
BBC (EN)	Articles from the British Broadcasting Corporation news service [61].
CNN (EN)	Articles from the American cable news network, focused on low-coverage IPTC categories [62].
DW (EN)	Articles from Deutsche Welle, Germany’s international broadcaster publishing in English [63].
Es (EN)	Articles from the English edition of the international sports media platform [60].
Mediahuis (EN)	Articles from the Belgian-Dutch media group operating newspapers and digital platforms across Europe [64].
Es (ES)	Articles from the Spanish edition of the international sports media platform [60].
RTS Ing (FR)	Instagram posts from Radio Télévision Suisse, the French-language public broadcaster of Switzerland [65].
RTS (FR)	Articles from Radio Télévision Suisse, the French-language public broadcaster of Switzerland [65].
Es (NL)	Articles from the Dutch edition of the international sports media platform [60].
DNA (NL)	Articles from EventDNA, a Dutch event-based news corpus with tokenized texts [66].
Mediahuis (NL)	Articles from Mediahuis Dutch-language publications [64].
NoordHB (NL)	Articles from the Dutch regional newspaper owned by Mediahuis [67].
NU (NL)	Articles from the most-read online news platform in the Netherlands [68].

Table 3.1 Overview of the news corpora used in this work.

comprises approximately 25.5% of the total articles in the collection. Conversely, the smallest corpora, *Mediahuis (NL)* and *Mediahuis (EN)*, each contribute less than 1% to the overall dataset. Text length also varies considerably. For instance, the English corpora *DW (EN)* and *CNN (EN)* contain the longest articles on average (883 and 844 tokens, respectively), whereas *DNA (NL)* contains the shortest, with a mean of only 60 tokens per article.

The entity extraction statistics are summarized in Table 3.3. We observe a positive correlation between article length and the absolute number of detected entities: the English *DW (EN)* and *CNN (EN)* corpora have the highest entity counts, averaging over 45 entities per article. In contrast, the *DNA (NL)* corpus contains the fewest entities on average (7.4) but has the highest mean relevance score (55.8). This suggests that the entities identified in these shorter texts are more closely connected to the core topic of the document. However, the relationship between entity linking success and relevance scores is not consistently

Corpus	Articles	Mean Article Token Count				
		Title	Lead	Body	Total	$\pm$ Std.
Mafra (CS)	2,082	13	48	451	512	$\pm$ 347
APA (DE)	14,293	8	9	236	253	$\pm$ 184
DPA (DE)	3,047	8	6	292	305	$\pm$ 251
Es (DE)	3,630	13	55	329	397	$\pm$ 330
BBC (EN)	10,879	9	0	364	373	$\pm$ 429
CNN (EN)	5,974	14	34	796	844	$\pm$ 535
DW (EN)	9,987	8	27	847	883	$\pm$ 765
Es (EN)	4,630	16	55	568	639	$\pm$ 568
Mediahuis (EN)	783	11	27	544	583	$\pm$ 450
Es (ES)	1,687	14	54	375	443	$\pm$ 270
RTS Ing (FR)	1,132	25	0	238	263	$\pm$ 80
RTS (FR)	15,458	11	27	382	421	$\pm$ 264
Es (NL)	1,054	13	52	232	298	$\pm$ 272
DNA (NL)	1,764	0	0	60	60	$\pm$ 22
Mediahuis (NL)	689	10	0	300	309	$\pm$ 287
NoordHB (NL)	28,065	14	40	360	414	$\pm$ 306
NU (NL)	4,984	9	30	276	315	$\pm$ 213
Mean	6,884	12	30	392	430	$\pm$ 331
Total	110,138	—	—	—	—	—

Table 3.2 Quantitative overview of the corpora, detailing the total number of articles and the average token counts for the title, lead, and body text segments.

negative across the remaining corpora.

Corpus	Entities	Linked		Relevance	
		$\bar{x}$	$\bar{x}$ σ	$\bar{x}$ σ	
Mafra (CS)	32.9	28.3	17.4	18.3	27.4
APA (DE)	20.7	17.0	13.4	29.4	31.1
DPA (DE)	26.9	25.0	25.4	21.9	28.4
Es (DE)	25.1	22.6	17.9	24.4	28.8
BBC (EN)	21.7	19.5	17.7	23.1	30.9
CNN (EN)	45.1	42.0	23.0	14.0	23.1
DW (EN)	47.3	43.6	30.4	<i>13.5</i>	<i>22.7</i>
Es (EN)	28.8	25.4	19.5	22.6	28.4
Mediahuis (EN)	32.7	27.1	24.7	19.4	27.3
Es (ES)	23.1	16.1	11.7	30.8	28.4
RTS Ing (FR)	24.2	21.6	8.7	24.6	31.6
RTS (FR)	32.5	29.3	16.6	18.8	27.9
Es (NL)	19.9	17.4	14.2	25.6	31.9
DNA (NL)	<i>7.4</i>	<i>6.7</i>	<i>2.9</i>	55.8	37.1
Mediahuis (NL)	21.4	17.5	14.1	25.9	31.6
NoordHB (NL)	25.5	20.5	11.7	23.3	30.4
NU (NL)	22.2	19.7	12.0	24.6	31.1
Mean	26.9	23.5	16.5	24.5	29.3

Table 3.3 Named entity statistics per corpus, detailing the average number of detected entities, the subset successfully linked to Wikidata, and their associated relevance scores.

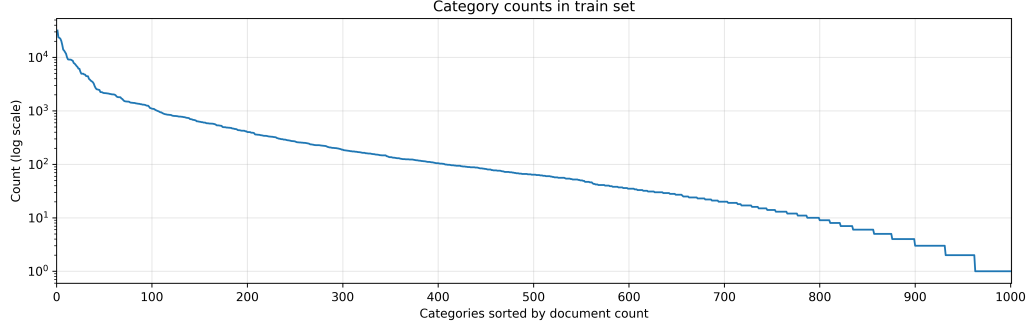
3.4 Categories and Labels

As discussed in Section 1.1.1, the IPTC Media Topics taxonomy comprises more than 1,200 distinct labels. In the dataset used for this research, the training set contains 1,001 unique labels, of which 823 are also present in the test set. Furthermore, 170 categories appear exclusively in the training set, and 5 categories are found only in the test set. These exclusive categories exist because of the chosen split; all of them have fewer than 10 samples in the whole dataset.

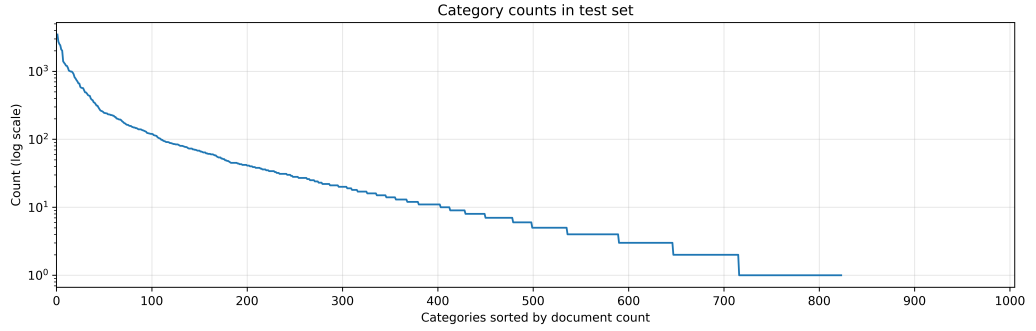
Figure 3.3a shows a considerable class imbalance in our training set. While this imbalance is partially due to the naturally higher frequency of certain news topics (e.g., politics or economy), it is primarily a consequence of the hierarchical structure of the IPTC taxonomy. Under this structure, an article assigned a specific low-level leaf category (e.g., synchronized free routine) automatically inherits all its parent categories, so higher-level categories (e.g., sport) have much higher frequencies.

If we look closely at distribution of labels in the training set, we will see:

- 19 categories appear in more than 10,000 articles;
- 132 categories appear in more than 1,000 articles;



(a) Category count in the training set.



(b) Category count in the test set.

Figure 3.3 Distribution of category counts across the training and test sets.

- 425 categories appear in more than 100 articles;
- 802 categories appear in more than 10 articles;
- 1,001 categories in total.

The label distribution in the test set is similar to that of the training data, as shown in Figure 3.3b. Some categories do not have any test samples. This is expected because our test set comprises 10% of the total data, and there are approximately 200 categories with fewer than 10 samples in the training set.

The baseline classifier does not restrict the output label space based on frequency in the training data. However, the extreme sparsity observed in certain labels suggests that a minimum occurrence threshold for filtering may improve model stability. Furthermore, the evaluation process should account for these variations in label density. Categories with limited representation are likely to have higher variance in performance metrics; observed improvements in these sparse categories may therefore lack statistical significance compared to those in more frequent classes.

3.5 Dataset Characteristics and Limitations

Now we will present the strengths and limitations of our dataset. These properties influence the choice of the train-test split, which is discussed later in Section 3.6.

3.5.1 Strengths

The primary strength of the corpus lies in its scale and the quality of its annotations. With over 110,000 human-verified articles, the dataset provides a solid basis for our research. Key advantages include:

- **Task-Specific Annotation:** The dataset was specifically annotated for our classification task, ensuring that the labels correspond 1:1 to the intended use case. IPTC categories are also continuously updated to reflect the current IPTC standard.
- **Multilingual Diversity:** The inclusion of multiple languages and various media sources allows for a comprehensive analysis of how entity-based features perform across languages.
- **Rich Entity Metadata:** The dataset includes pre-linked entities with associated types and relevance scores. These entities are connected to a knowledge base that is regularly updated and synchronized with Wikidata, providing access to external information for each linked entity.
- **High Source Variety:** The diversity of the news corpora allows us to evaluate the model’s robustness across different journalistic styles and regional topics.

3.5.2 Limitations and Challenges

Despite its strengths, the dataset presents certain challenges that influence the experimental design:

- **Missing Data:** Publication dates are not consistently available for all articles. This prevents a strictly chronological train–test split, which is often preferred for news data to avoid temporal leakage.
- **Data Reduction:** Prioritizing data quality (gold data) over quantity (silver data), combined with the downsampling of overrepresented sources, results in a smaller total article count than was initially available.

3.6 Dataset Division

To rigorously evaluate our classifier, the dataset must be partitioned into training, development (validation), and test sets. Although a predefined randomized split exists from previous iterations of the model, we changed the splitting strategy to better suit our experiments. Three partitioning strategies were considered to balance experimental integrity with practical evaluation requirements. For hyperparameter tuning, we use *5-fold cross-validation*. Therefore, instead of explicitly dividing the dataset into training, development, and test sets, we first split the data into training and test sets and then perform cross-validation on the training set, with a dedicated development set derived from it.

3.6.1 Evaluated Partitioning Strategies

We considered three train–test partitioning strategies that trade off temporal integrity, corpus representation, and the practical constraints caused by missing publication dates. Each strategy is described below, followed by the rationale for adopting corpus-level chronological splitting.

Strategy 1: Global Chronological Split

A strictly chronological split divides the entire dataset based on publication timestamps, ensuring the model is trained exclusively on articles published earlier than the test articles. Although this prevents temporal leakage and reflects real-world deployment, it was rejected for the following reasons:

- **Loss of Test Set Diversity:** This is the main reason. The temporal distribution varies considerably across sources; a global chronological split would restrict the test set to a narrow time window. As a result, 10 of the 18 corpora would be completely excluded from the test set. Each corpus represents a specific customer or distinct domain, and excluding them from the test set would prevent per-customer and per-language evaluations that we need to properly assess our model.
- **Missing Metadata:** More than 20,000 articles lack timestamp metadata. Strict global chronology would require either discarding this solid volume of data or placing it arbitrarily in the training set, which violates the chronological constraint. In addition, a whole corpus (*DNA (NL)*) would be completely lost due to the absence of dates.

Strategy 2: Random Stratified Split (Origin)

The existing baseline model uses a random stratified split, distributing articles into training, development, and test sets in an 8:1:1 ratio. This approach guarantees that all corpora are represented proportionally in the evaluation phases. However, it ignores temporal dynamics entirely, allowing future articles to inform predictions on past articles within the same domain, which is contrary to reality.

Strategy 3: Corpus-Level Chronological Split (Chosen Approach)

To combine the representativeness of a random split with the temporal integrity of a chronological split, we adopted a hybrid approach: data is split chronologically *within* each individual corpus.

In this setup, each corpus is ordered by time, and the newest articles (e.g., the most recent 10%) are reserved for the test set. The remaining data form the training set and are then further divided to create a development set. This strategy offers several advantages:

- **Complete Representation:** Every corpus and language is represented in the evaluation phase, aligning with the core objective of analyzing model behavior across diverse customers and linguistic contexts.

- **Minimized Temporal Leakage:** The test partition of each corpus contains its newest articles. Although global chronological ordering is not enforced, the risk of temporal leakage between corpora is smaller than within a single corpus, because the corpora cover different languages, regions, and journalistic topics.
- **Robust Handling of Missing Data:** Under this strategy it is reasonable to place articles lacking timestamps into the training set, since we do not enforce strict global chronology. In addition, the corpus *DNA (NL)*, which is completely devoid of timestamps, can be partitioned randomly.

As described in Section 3.1.2, the category predicted by the IPTC classifier serves as a signal for entity disambiguation. However, changing the train–test split could introduce a dependency between the production IPTC classifier and our evaluated models. Specifically, the production classifier might have been trained on articles that belong to the test set under our proposed corpus-level chronological split.

To evaluate the impact of this potential dependency, we re-evaluated all models using the entity sources from Experiment 2 (Section 4.7.2). We compared the proposed chronological split (Strategy 3) against the original random stratified split (Strategy 2). Under Strategy 2, both the production IPTC classifier and our models share the same train–test partitioning.

The results of this comparison are presented in Table A.9. The performance metrics are nearly identical across both partitioning strategies: all models achieve very similar Micro and Macro F_1 scores, and the relative improvements over the baseline remain consistent. We therefore conclude that the dependency introduced by the production IPTC classifier has a negligible effect on the evaluation. Consequently, we adopt the corpus-level chronological split for subsequent experiments, as it mitigates temporal leakage while maintaining independent corpus evaluation.

3.6.2 Train Split Statistics

This section analyzes the composition of the training split across the selected corpora. The statistics for the test split are given in Appendix A.1.

Table 3.4 summarizes the training split, which comprises 99,836 articles across 17 distinct corpora. The corpora vary considerably in document length and category distribution. For instance, *CNN (EN)* and *DW (EN)* contain the longest articles (averaging 837 and 858 tokens, respectively), while *DNA (NL)* consists of notably shorter texts (averaging 60 tokens). The average number of IPTC labels per document also varies across sources: the *RTS Ing (FR)* corpus has the highest annotation density, with 9.2 labels per document on average, whereas the *Es (ES)* corpus has the lowest, with an average of only 3.7 labels.

Furthermore, while the combined training set contains 406 distinct categories with over 100 documents each, this distribution varies greatly across individual corpora. It ranges from a minimum of 6 categories in *Es (NL)* to a maximum of 150 categories in *NoordHB (NL)*. The number of well-supported categories does not strictly correlate with overall corpus size. For example, the various *Es* datasets have considerably fewer well-supported categories than other datasets of similar size. Specifically, *Es (DE)* has only 9 categories with over 100 examples across

3,267 articles. In contrast, the slightly smaller *DPA (DE)* corpus (2,963 articles) contains 34 such categories. This difference is likely because the *Es* datasets focus exclusively on sports and therefore omit other top-level categories.

To account for the unbalanced corpus sizes, the table reports both macro and micro averages for the relevant metrics.

Corpus	Articles	% of Total	Article Length		Labels/Doc	Cat. > 100
			$\bar{x}$	σ		
Mafra (CS)	1,890	1.9	509	346	5.3	23
APA (DE)	12,938	13.0	252	180	5.7	119
DPA (DE)	2,963	3.0	303	250	5.0	34
Es (DE)	3,267	3.3	403	333	3.8	9
BBC (EN)	10,035	10.1	363	434	6.5	116
CNN (EN)	5,376	5.4	837	526	7.0	77
DW (EN)	8,988	9.0	858	737	7.5	117
Es (EN)	4,167	4.2	635	560	3.9	15
Mediahuis (EN)	704	0.7	564	406	7.1	11
Es (ES)	1,518	1.5	443	275	3.7	8
RTS Ing (FR)	1,072	1.1	263	81	9.2	19
RTS (FR)	13,931	14.0	420	262	6.4	118
Es (NL)	948	0.9	293	260	4.1	6
DNA (NL)	1,587	1.6	60	22	6.8	22
Mediahuis (NL)	620	0.6	310	291	5.8	7
NoordHB (NL)	25,258	25.3	413	306	5.6	150
NU (NL)	4,574	4.6	309	210	6.9	70
Avg. (Macro)	5,872	5.9	426	322	5.9	54
Avg. (Micro)	—	—	446	346	6.0	—
Total	99,836	100.0	—	—	—	406

Table 3.4 Corpus summary for the training split: total article count, percentage share within the split, mean article length in tokens (with standard deviation), mean number of IPTC labels per document, and the number of distinct categories containing more than 100 documents.

Table 3.5 presents an alternative aggregation of the same training data, by language rather than by individual news source. While Table 3.4 highlights the variance among specific publishing corpora, this language-level view reveals dependencies within a language.

Excluding the Czech (cs) and Spanish (es) datasets, which constitute marginal fractions of the training split (1.9% and 1.5%, respectively), the data distribution is relatively balanced across the remaining four languages. Dutch (nl), English (en), German (de), and French (fr) account for most of the corpus, each representing between 15.0% and 33.0% of the total volume.

Similar to the corpus-level analysis, the average number of IPTC labels per document and the article lengths differ across languages. French (fr) and English (en) have the highest annotation densities, averaging 6.6 and 6.5 labels per document, respectively, whereas Spanish (es) has the lowest, with an average of only 3.7 labels. English articles are the longest, with a mean of 646 tokens, and consequently contain the highest number of well-supported categories (241 out of the 406 distinct categories in the training set). The underrepresented Spanish subset contains only 8 categories with support exceeding 100 documents.

Lang	Arts	Arts %	Art. $\bar{x}$	Len σ	Lbl/doc	Cat.>100
cs	1,890	1.9	509	346	5.3	23
de	19,168	19.2	286	231	5.3	139
en	29,270	29.3	646	616	6.5	241
es	1,518	1.5	443	275	3.7	8
fr	15,003	15.0	409	256	6.6	131
nl	32,987	33.0	376	297	5.8	195
avg. (macro)	16,639	16.7	445	337	5.5	123
avg. (micro)	—	—	426	346	6.0	—
Total	99,836	100.0	—	—	—	406

Table 3.5 Per-language summary: Arts (article count), Art.% (share within the split), Art. Len (mean total article length and standard deviation), Lbl/doc (mean IPTC labels per document), and Cat.>100 (categories with more than 100 documents in the language). Statistics computed from the train split only.

Tables 3.6 and 3.7 illustrate the extremes of the IPTC category distribution within the training split, revealing a highly skewed dataset.

Table 3.6 details the 20 most frequent categories. These predominantly consist of broad, top-level nodes within the IPTC media topics hierarchy. The corpus is dominated by general news subjects: *politics and government* is the most common category, assigned to 31.8% of all training documents. This is closely followed by *sport* (23.8%) and *economy, business and finance* (23.2%). When secondary nodes appear among the most frequent topics, they still represent broad domains, such as *sport > competition discipline* or *politics and government > government*. A notable outlier in this top-heavy distribution is *sport > competition discipline > soccer*. Despite being a specific, third-level leaf node, it ranks 17th overall and appears in 8.7% of the training documents, indicating a strong topical concentration on soccer within the corpus’s sports coverage.

Conversely, Table 3.7 presents a sample of the least common categories, each appearing in only a single document (effectively 0.001% coverage). In contrast to the top-level dominance of the frequent topics, these rare labels represent specific leaf nodes located at the bottom levels of the IPTC hierarchy. These rare categories often extend three or four levels deep, describing granular subjects such as niche sports disciplines (e.g., *sport > competition discipline > track and field > cross-country run*) or specific educational and religious sub-branches. This steep drop-off in representation demonstrates a pronounced long-tail distribution across the labels.

Entity Statistics in Our Train Set

Beyond corpus- and language-level summaries, this subsection characterizes the entity signal available in the training split. It reports per-article entity counts, type composition, and the relationship between entity volume and relevance across sources. Figure 3.4 illustrates the distribution of recognized named entities per article within the training dataset. The distribution is heavily right-skewed, with a mean of 28.0 entities per document and a standard deviation of 21.4. While the total number of entities in a single article ranges from 0 to 882, most documents

Rank	Category (IPTC ID)	Docs	%
1	politics and government (11000000)	31,776	31.8
2	sport (15000000)	23,746	23.8
3	economy, business and finance (04000000)	23,164	23.2
4	sport > competition discipline (20000822)	22,347	22.4
5	politics and government > government (20000593)	20,291	20.3
6	economy, business and finance > products and services (20000209)	17,154	17.2
7	crime, law and justice (02000000)	13,964	14.0
8	sport > sport event (20001108)	13,360	13.4
9	society (14000000)	12,478	12.5
10	arts, culture, entertainment and media (01000000)	11,661	11.7
11	politics and government > government > political party (20001384)	10,007	10.0
12	environment (06000000)	9,177	9.2
13	human interest (08000000)	9,091	9.1
14	crime, law and justice > crime (20000082)	9,088	9.1
15	health (07000000)	9,066	9.1
16	politics and government > government policy (20000621)	8,829	8.8
17	sport > competition discipline > soccer (20001065)	8,661	8.7
18	conflict, war and peace (16000000)	7,862	7.9
19	arts, culture, entertainment and media > arts and entertainment (20000002)	7,632	7.6
20	health > disease and condition (20000446)	7,282	7.3

Table 3.6 Top 20 IPTC categories by document coverage across all corpora. Statistics computed from the train split only.

Category (IPTC ID)	Docs	%
sport > competition discipline > track and field > cross-country run (20000828)	1	0.0
sport > competition discipline > softball (20001066)	1	0.0
sport > competition discipline > kiting (20001306)	1	0.0
religion > religious text > Qur'an (20000707)	1	0.0
sport > competition discipline > track and field > shot put (20000843)	1	0.0
religion > belief systems > Confucianism (20000673)	1	0.0
education > school > private school (20001214)	1	0.0
education > school > religious school (20001212)	1	0.0
sport > competition discipline > golf > mini golf (20000941)	1	0.0
sport > competition discipline > dog racing (20000919)	1	0.0

Table 3.7 Bottom 10 IPTC categories by document coverage across all corpora. Statistics computed from the train split only.

cluster densely around the lower end of this spectrum.

The entity linking process achieves high coverage and successful resolution to the knowledge base. On average, 24.5 entities per article are successfully linked to a Wikidata identifier, leaving a mean of only 3.6 unrecognized or unlinked entities per document. The extraction pipeline identified at least one entity in nearly every document: only 21 articles in the entire training split contain zero linked entities.

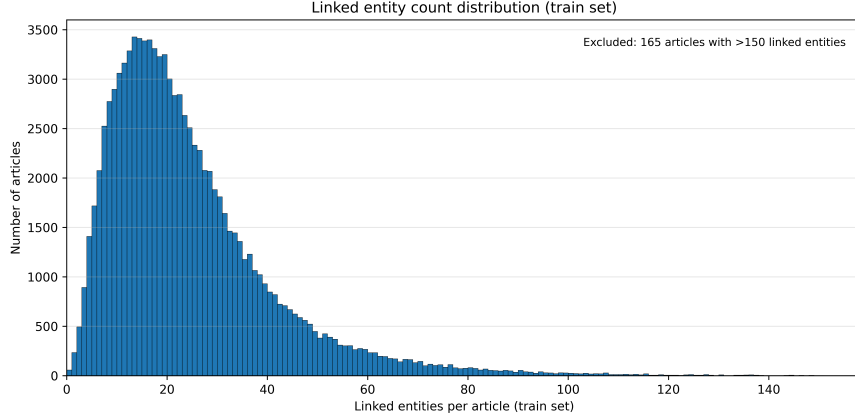


Figure 3.4 Distribution of entity counts in the training dataset.

Table 3.8 details the type distribution of the successfully linked entities. Nearly half of the recognized items are classified as “general” entities (46.0%), although this category has the lowest average relevance score (13.9). Conversely, more specific named entities (locations at 27.7%, persons at 11.3%, organizations at 10.0%, and events at 2.4%) achieve higher average relevance scores, ranging from 26.7 to 29.6. This suggests that these concrete entity types provide stronger discriminative signals for categorizing news articles.

Entity Type	Linked Entities	%	Avg. Relevance
general	1,121,934	46.0	13.9
location	676,428	27.7	27.0
person	275,680	11.3	27.9
organization	244,453	10.0	26.7
product	64,831	2.7	21.7
event	57,886	2.4	29.6

Table 3.8 Distribution of linked-entity types and their average relevance scores across all corpora. Statistics are computed from the training split only.

Table 3.9 reports the average per-article entity statistics for each corpus. The data reveals a notable inverse relationship between entity volume and entity relevance across several corpora. For instance, *DW (EN)* has the highest average number of linked entities (42.5 per document) but the lowest mean relevance score (13.8). Conversely, the *DNA (NL)* corpus contains the fewest linked entities on average (6.7 per document) yet has the highest mean relevance score (55.7). This suggests that shorter articles, or those with stricter editorial guidelines, tend to

contain fewer but highly topical entities, whereas longer articles accumulate a larger volume of peripheral entities.

Corpus	Entities	Linked		Relevance	
		$\bar{x}$	$\bar{x}$	$\bar{x}$	σ
Mafra (CS)	32.8	28.2	17.3	18.3	27.4
APA (DE)	20.5	16.9	12.9	29.5	31.1
DPA (DE)	26.5	24.6	24.6	22.2	28.6
Es (DE)	25.4	22.9	18.5	24.2	28.7
BBC (EN)	21.3	19.1	18.0	23.3	31.1
CNN (EN)	44.8	41.8	22.0	14.1	23.2
DW (EN)	46.2	42.5	29.2	13.8	23.0
Es (EN)	28.9	25.6	19.8	22.6	28.4
Mediahuis (EN)	31.9	26.4	24.6	19.8	27.5
Es (ES)	23.1	16.0	11.5	30.9	28.4
RTS Ing (FR)	24.2	21.6	8.8	24.7	31.7
RTS (FR)	32.4	29.2	16.5	18.9	28.0
Es (NL)	19.3	16.9	13.6	26.1	32.3
DNA (NL)	7.4	6.7	3.0	55.7	37.1
Mediahuis (NL)	21.2	17.5	14.3	25.9	31.7
NoordHB (NL)	25.3	20.4	11.7	23.4	30.5
NU (NL)	22.0	19.5	12.0	24.8	31.2
Avg. (Macro)	26.7	23.3	16.4	24.6	29.4
Avg. (Micro)	28.0	24.5	19.2	20.9	28.8

Table 3.9 Corpus-level entity statistics per article: total recognized entities, subset of linked entities, and linked-entity relevance (mean and standard deviation). Statistics are computed from the training split only.

Table 3.10 presents the entity extraction and linking statistics aggregated by language. This macro-level view confirms that the inverse relationship between the volume of extracted entities and their average relevance, previously observed at the corpus level, persists across languages.

Articles in English (en) contain the highest average number of recognized entities (34.6) and successfully linked entities (31.6) per document, yet they have the lowest mean relevance score (17.0). In contrast, the Spanish (es) subset has the fewest linked entities per article (16.0) but the highest average relevance score (30.9). German (de) and Dutch (nl) exhibit intermediate values, both averaging fewer than 20 linked entities per document while maintaining relatively high relevance scores (27.0 and 24.3, respectively). Consistent with the corpus-level data, the standard deviation for the relevance metric remains high across all languages, indicating a heavily skewed distribution of relevance values regardless of the source language.

Language	Entities	Linked		Relevance	
		$\bar{x}$	$\bar{x}$	$\bar{x}$	σ
cs	32.8	28.2	17.3	18.3	27.4
de	22.3	19.1	16.5	27.0	30.3
en	34.6	31.6	25.4	17.0	26.0
es	23.1	16.0	11.5	30.9	28.4
fr	31.8	28.7	16.1	19.2	28.2
nl	23.8	19.5	12.0	24.3	31.1
Avg. (Macro)	28.0	23.8	16.5	22.8	28.6
Avg. (Micro)	28.0	24.5	19.2	20.9	28.8

Table 3.10 Language-level entity statistics per article: total recognized entities, subset of linked entities, and linked-entity relevance (mean and standard deviation). Statistics are computed from the training split only.

3.7 Data Format

The initial dataset was provided in JSONL (JSON Lines) format, where each line contains a natural language analysis of one article. For easier processing, these raw corpora were transformed into a tabular CSV format using automated internal preprocessing scripts.

3.7.1 Feature Representation in CSV

The resulting CSV files comprise the following core attributes, which serve as the foundation for the classification task:

- **id**: A unique string identifier assigned to each article (e.g., `nl_nu_iptc_976`).
- **title**, **lead**, **text**: The raw textual components of the article, divided into the headline, introductory paragraph, and main body text.
- **cats**: The ground truth labels, encoded as pipe-separated strings of IPTC Media Topic codes (e.g., `02000000|04000000|11000000`). For context, `02000000` represents *Crime, Law and Justice*, `04000000` denotes *Economy, Business and Finance*, and `11000000` corresponds to *Politics*.
- **metadata**: A JSON dictionary containing corpus provenance and pipeline heuristics. This includes the `corpusName`, the specific data split (`corpusPart`, e.g., `dev_all`), and confidence scores (`catScores`) that indicate the certainty of the upstream pipeline for each assigned category.
- **entities**: A JSON array containing the entities extracted from our article analysis.

3.7.2 Entity Enrichment and Knowledge Base Mapping

To support the primary goal of this research, the baseline dataset was augmented by extracting named entities and publication dates from the source JSONL

files. These entities were appended to the CSV dataset and preserved as JSON arrays. Retaining this native structured format allows the classifier to use entity-level metadata, such as relevance scores, for subsequent feature engineering and weighting. Listing 3.1 shows the internal structure of a single extracted entity.

Listing 3.1 Standardized JSON representation of an extracted entity, including internal identifier (gkbId) and relevance metrics.

```
{
  "stdForm": "hokejista",
  "type": "general",
  "id": "e5",
  "gkbId": "G11774891",
  "feats": {
    "relevance": "10"
  },
  "mentions": [
    {
      "text": "hokejistovi",
      "mwl": "hokejista",
      "id": "m12",
      "tokenIds": ["w36"]
    },
    {
      "text": "lední hokejista",
      "mwl": "hokejista",
      "id": "m32",
      "tokenIds": ["w126"]
    }
  ]
}
```

Because the proposed architecture relies on Wikidata for structural knowledge, the internal entities must be explicitly linked to Wikidata Identifiers (WDIDs). The NLP pipeline identifies entities using internal Geneea Knowledge Base identifiers (gkbId). To enable WDID linkage and ensure experimental reproducibility without requiring live access to the proprietary GKB, a static mapping file, was compiled.

The final input pipeline for the proposed classifier consists of entity-enriched CSV files paired with this static TSV mapping file.

4 Methodology

This chapter details the methodological approach and system architecture used to train and evaluate the classifiers presented in this thesis. First, we outline the end-to-end pipeline, including data ingestion, feature extraction, and the baseline classifier structure. Next, we discuss the evaluation metrics tailored to our highly imbalanced dataset, followed by the tuning and model selection protocols. Finally, we introduce a series of experiments designed to address class imbalance and evaluate various entity-aware classification enhancements.

4.1 System Architecture Overview

Our classification pipeline is structured as a sequence of individually traceable stages. These stages transform a multilingual news corpus, annotated with IPTC Media Topics, into a trained multi-label classifier and an accompanying evaluation report. The system design is driven by two main principles.

First, every stage consumes well-typed inputs and produces a serializable output. This allows any downstream stage to be re-executed in isolation when only its immediate dependencies change, saving computational time during experimentation.

Second, all stages are integrated as ClearML pipeline components. ClearML automatically caches each step; tracks all inputs, outputs, logs, and metrics; and records the exact code versions and computational environments used. Furthermore, it also provides a dynamic user interface that lets us modify parameters and rerun pipelines. Consequently, this architecture guarantees the complete reproducibility of every experimental run. Figure 4.1 presents a high-level overview of the resulting end-to-end process.

The pipeline is driven by a single configuration object that fully specifies the corpus, embedding sources, model and training hyperparameters, cross-validation strategy, and evaluation thresholds. Given this configuration, six stages are executed in order:

1. **Data loading and normalization** reads the train and test CSVs, harmonizes the IPTC label set (filtering removed categories, applying optional down-sampling), and attaches the pre-computed Wikidata-linked entities to each document, discarding entities whose relevance falls below a configurable threshold if one is set.
2. **Article embedding** obtains a fixed-length representation $h_{\text{art}} \in \mathbb{R}^{d_a}$ for every document through a multilingual transformer encoder. Vectors are persisted to a cache keyed by document identifier so that subsequent runs only compute the missing entries.
3. **Entity lookup and pooling** maps the Wikidata identifiers attached in stage 1 to vectors taken from a pre-trained entity embedding store. A configurable pooling operator (described in Section 4.7.4) collapses the per-document bag of entity vectors into a representation $h_{\text{ent}} \in \mathbb{R}^{d_e}$ for

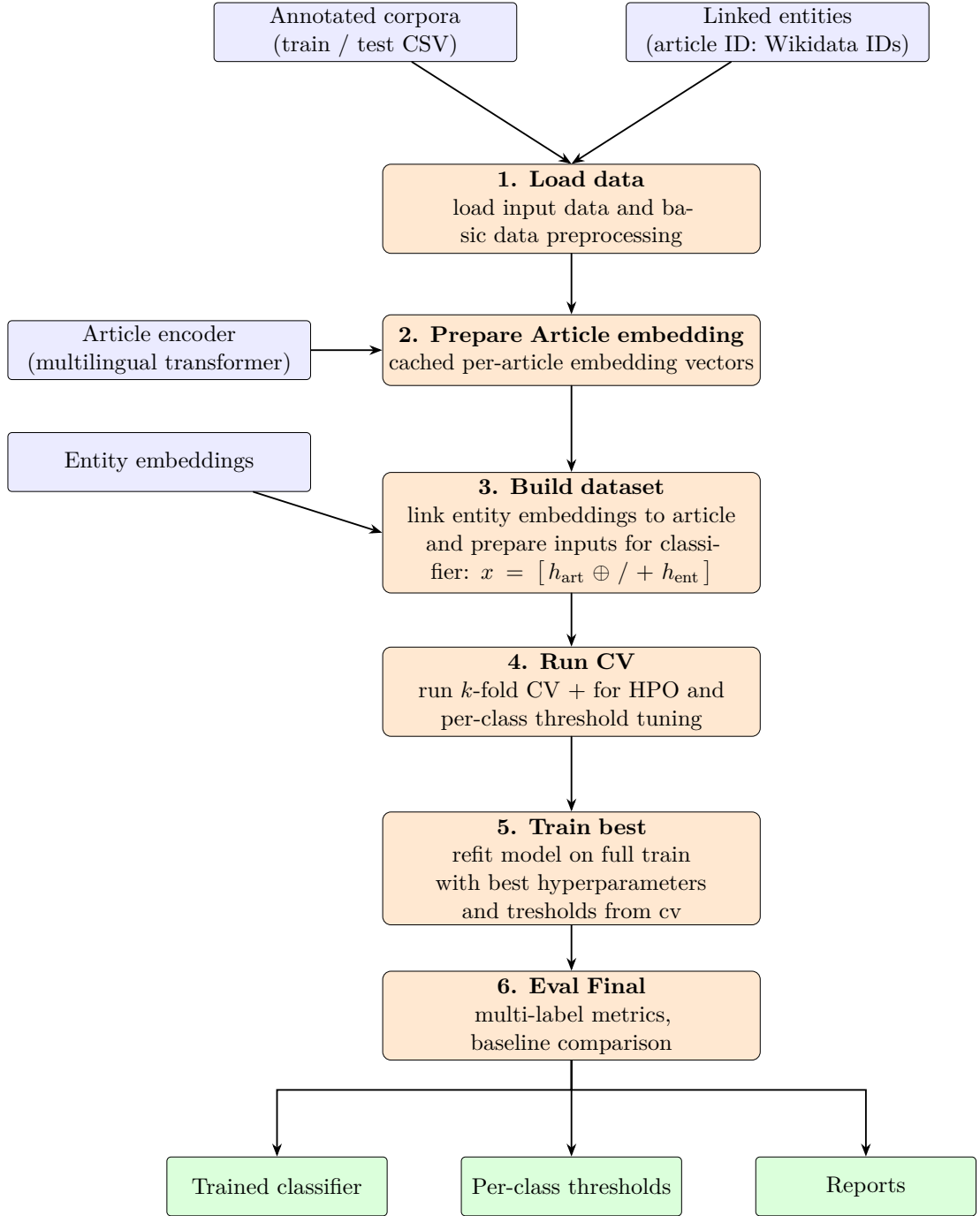


Figure 4.1 End-to-end architecture of the entity-enhanced IPTC Media Topics classification pipeline. Blue boxes denote external inputs and pre-trained resources; orange boxes are stages of the orchestrated pipeline; green boxes symbolize the outputs of the pipeline

the model, or keeps the full ragged set of entities to be processed later by attention in the model.

This step also includes **feature fusion**, which concatenates the article and entity representations to produce the input for the model: $x = [h_{\text{art}} \oplus h_{\text{ent}}] \in \mathbb{R}^{d_a+d_e}$. Either the article or the entity vector can be disabled; in that case, the output is only the remaining vector.

4. **Cross-validation and hyperparameter optimization** fits the multi-label classifier over k stratified folds of the training split, with hyperparameters from a search over the configured space. The best configuration is selected using a single objective metric (e.g. micro-F1 across all labels), and per-class decision thresholds are tuned on out-of-fold predictions.
5. **Final training** retrains the selected model on the full training split with the chosen hyperparameters, reusing the early-stopping epoch budget identified during cross-validation and per-class tuned thresholds.
6. **Test evaluation** applies the trained model and tuned thresholds to the held-out test set, computes per-class and aggregate multi-label metrics, optionally compares them against a stored baseline run, and persists the artifacts (model weights, thresholds, score matrices, and report tables).

The architecture is also designed to support an ensemble extension in which two independently configured members are trained in parallel and combined per class based on their cross-validation performance; this variant reuses stages 1–5 unchanged for each member and inserts a class-routing step before stage 6. For clarity, Figure 4.1 shows the single-model variant; the ensemble extension is described in Section 4.7.

4.1.1 Article Embeddings

The article representation, $h_{\text{art}} \in \mathbb{R}^{384}$, is a 384-dimensional multilingual vector. This language-agnostic embedding allows a single classifier to predict IPTC labels across all six languages without requiring separate models. To ensure a fair comparison with the baseline, the article representation is produced using the same encoder configuration as the original in-house classifier.

Encoder and Text Preparation. The text is encoded using the multilingual sentence-transformer `paraphrase-multilingual-MiniLM-L12-v2` citreimers-2019-sentence-bert,wang2020minilm. The input text is constructed by concatenating the `title`, `lead`, and `text` fields in their original order, ensuring the most critical information appears first.

Sentence Chunking. To fit the encoder’s 128-token context window, the document is first segmented into sentences and then greedily packed into chunks of up to 300 characters. A sentence is appended to the current chunk until the 300-character limit is reached, at which point a new chunk is started. This approach maintains linguistic coherence. Each chunk c_i is embedded independently to produce a corresponding chunk vector e_i .

Weighted Chunk Aggregation. The final document vector h_{art} is a convex combination of chunk vectors e_i . Because the title and the introduction empirically carry a disproportionate share of the topical signal in news writing, the aggregation guarantees a minimum weight of 0.3 for the initial chunk (e_1), so that it is not overshadowed in longer articles.

Let N be the total number of chunks. The aggregation is defined as:

$$h_{\text{art}} = \begin{cases} \frac{1}{N} \sum_{i=1}^N e_i, & \text{if } \frac{1}{N} \geq 0.3 \\ 0.3e_1 + 0.7\frac{1}{N-1} \sum_{i=2}^N e_i, & \text{if } \frac{1}{N} < 0.3 \end{cases}$$

In short, for articles that fit into three or fewer chunks ($N \leq 3$), this reduces to a simple mean. For longer articles ($N \geq 4$), the first chunk receives a fixed weight of 0.3, and the remaining chunks uniformly share the residual weight of 0.7.

The 300-character limit and the weight of the first chunk were empirically chosen for the original model. We retain these values, as optimizing article-chunking parameters is outside the scope of this research.

4.2 Entity Embeddings

Each article in the corpus is processed by an entity linker, which detects entities mentioned in the text and links them to a knowledge base. This section describes how this set of linked entities is turned into a single fixed-size *entity embedding*. The whole construction is summarized in Figure 4.2.

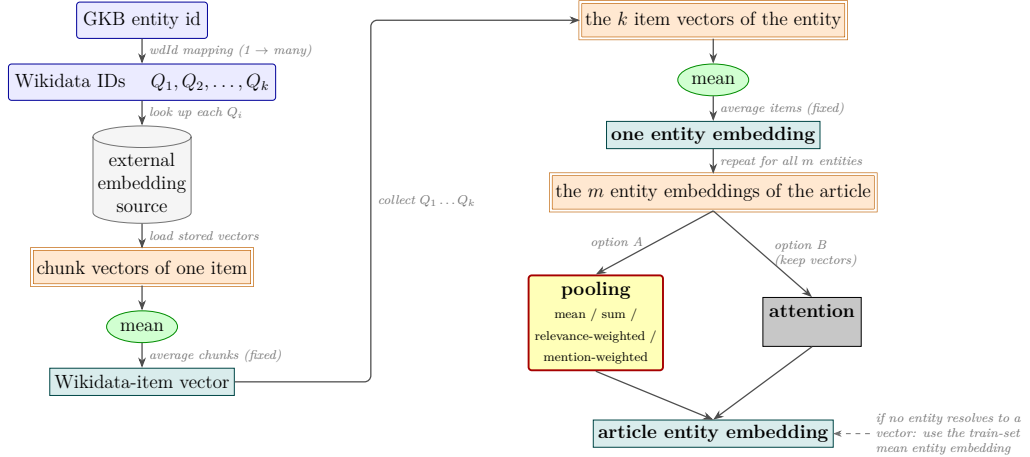


Figure 4.2 Construction of the article-level entity embedding. A linked entity starts as a GKB id that is mapped to several Wikidata IDs; their external vectors (averaged over chunks, then over items) yield one embedding per entity. The set of entity embeddings of an article is then reduced to a single vector either by a configurable pooling strategy or by an attention module. Single-bordered boxes are single vectors, double-bordered boxes are sets of many vectors; the two averaging steps are fixed, while the final reduction is the configurable part of the pipeline.

From a GKB Identifier to Wikidata Items. The entity linker identifies every entity by a *GKB id* (an identifier in Geneea’s internal knowledge base). A GKB id is not directly tied to a pretrained embedding, so it is first translated into Wikidata. A static mapping table resolves each GKB id into a list of *Wikidata IDs* ($Q_1, \dots, Q_k$). This step is one-to-many: a single linked entity may point to several Wikidata items, which are the actual objects for which embeddings exist. This linking is provided by the knowledge base.

Loading External Item Embeddings and Chunk Averaging. For every Wikidata ID, a precomputed embedding is loaded from an external source. Because the textual description of an item can be longer than the context window of the model that produced the embeddings, a single item may have been split into several *chunks*, each stored as its own vector. When this happens, we do not assume any difference in relevance between the chunk vectors of one item and average them into a single vector. This is the first averaging step. The result is one vector per Wikidata item. This approach was adopted to support embeddings from the Wikidata Embedding Project.

From Wikidata Items to One Entity Embedding. A linked entity can map to several Wikidata items, so the per-item vectors of one entity are averaged into a single *entity embedding*. This is the second fixed averaging step. After it, every linked entity of the article is represented by exactly one vector.

From Entities to One Article-Level Entity Embedding. An article usually contains several linked entities, hence several entity embeddings. This is the stage where the pipeline offers a real choice, and it is the main configurable point of the whole construction. Two routes are possible:

- **Pooling** collapses a set of entity embeddings into a single vector using a fixed reduction operation (such as mean or max pooling).
- **Attention** preserves the entity embeddings as a set of separate vectors and processes them through an attention module. This module learns to combine the vectors into a single representation by computing a weighted average based on attention scores. Because the attention mechanism requires training, it is integrated directly into the classifier architecture. In contrast, pooling is a static operation applied before passing the aggregated vector to the model.

Both routes output a single article-level entity embedding.

Fallback for Articles Without Resolvable Entities. Some articles either have no linked entities or link only to entities for which no external embedding exists. For these articles the entity embedding cannot be built from the article’s own entities, so the mean entity embedding computed over the whole training corpus is used instead. This guarantees that every article receives a well-defined, fixed-size entity embedding.

4.3 Evaluation Metrics

Having defined how article and entity information is encoded, we now specify how classifier quality is measured.

4.3.1 Data Distribution and Metric Constraints

The dataset has a heavily unbalanced long-tailed distribution across IPTC categories. Recalling the category distribution:

- **0 samples:** 183 categories,
- **1–14 samples:** 479 categories,
- **15+ samples:** 344 categories.

This distribution directly affects the suitability of standard classification metrics. Because the label space is large and most articles have only a few active categories, the confusion matrix for any individual class contains an overwhelming proportion of True Negatives (TNs). From a practical perspective, TNs are uninformative. In production, a user receives a filtered list of suggested categories. The primary concerns are whether the suggested categories are correct (*Precision*) and whether the model successfully finds all applicable categories (*Recall*). Categories correctly left unassigned do not add value to the end-user experience.

4.3.2 Limitations of Alternative Metrics

Given the data properties described above, several standard metrics were unsuitable as primary objectives:

- **Accuracy:** This metric is heavily dominated by True Negatives. A trivial baseline model that predicts no labels for any article would achieve an accuracy score greater than 99%, making accuracy uninformative for comparing model improvements.
- **Matthews Correlation Coefficient (MCC):** While MCC provides a balanced global view by incorporating all four quadrants of the confusion matrix (TP, TN, FP, FN) and is robust against class imbalance, it remains heavily influenced by the large volume of TNs, which are of little interest here. MCC is also less intuitive to interpret than the *F*-score when evaluating a category recommendation system.
- **Macro F_1 -Score:** Macro averaging treats every category equally, regardless of its frequency. If chosen as the primary objective, a model could be favored even if it makes frequent errors on highly populated classes, as long as it performs well on rare classes.

4.3.3 Primary Objective: Micro F_1 -Score

We select the **Micro F_1 -score** as the primary evaluation metric. The Micro F_1 -score aggregates the global counts of True Positives, False Positives, and False Negatives across all classes before calculating a single score:

$$\text{Micro } F_1 = \frac{2 \cdot \text{TP}_{\text{total}}}{2 \cdot \text{TP}_{\text{total}} + \text{FP}_{\text{total}} + \text{FN}_{\text{total}}} \quad (4.1)$$

This calculation implicitly weights each instance and label equally, meaning the score is influenced more by frequent classes. This property is desirable for our scenario because we assume the class distribution in our test set closely mirrors the real-world distribution of our users.

Although the IPTC taxonomy is hierarchical, we treat all categories as having equal value because we lack concrete data to weight specific topics higher than others, therefore, every individual prediction carries identical weight. A high error rate in a high-frequency class would lead to a large influx of False Positives or False Negatives, damaging user trust in the classifier. On the other hand, a high False Negative or False Positive rate in rare classes is less damaging overall because those categories seldom appear in daily operations.

Since determining an analytical or business-driven weight for each class without arbitrary assumptions is not feasible, letting natural class frequency drive the weighting via the Micro F_1 -score is the most objective approach.

4.3.4 Secondary Metrics Suite

While the Micro F_1 -score serves as our primary optimization target, it is insufficient on its own to capture all aspects of model behavior or partial improvements in specific areas. We therefore report a secondary evaluation suite grouped into four levels:

a) Micro-Level Metrics

- **Micro Precision and Micro Recall:** These are reported alongside the Micro F_1 -score to expose the specific trade-offs the model makes between precision and recall.

b) Binned Macro-Level Metrics

These metrics prevent performance on frequent labels from dominating the overall evaluation and better reflect performance on rare labels, giving a clearer picture of how the model performs across multiple domains. To prevent rare classes with insufficient data from distorting the macro metrics, we divide the categories into two distinct buckets:

- **Macro Tail (< 15 samples):** The same evaluation metrics are computed on the 478 categories with fewer than 15 test samples. The long tail introduces two distinct challenges. First, classes with low test support typically correspond to classes with limited training data; consequently, the classifier may not adequately learn the underlying data distribution, often resulting in low or zero metrics. Second, the small number of test

samples per class increases metric variance (noise) at the individual category level. The primary purpose of the Macro Tail metric is to determine whether adding entities systematically improves the classification of very low-resource categories.

- **Macro Head (≥ 15 samples):** Macro-averaged F_1 -score, Precision, and Recall are calculated exclusively across the 344 classes that contain 15 or more test samples. This bin provides a performance view of categories with sufficient statistical support, without being overly pessimistic due to zero values in small classes.

These head and tail metrics exclude the 183 categories with no test samples.

For a more detailed view of the model, we further partition the first bucket into three: classes with 15–100, 100–1000, and 1000 or more samples.

c) Per-Corpus Performance

Because our data originates from diverse international publishers (e.g., *APA* for German, *BBC* for English, *Noordhollands Dagblad* for Dutch), we expect localized domain shifts. We report Micro metrics per corpus to evaluate model stability across individual datasets and client environments. We use a micro average for the same reasons as for our objective metric.

d) Per-Class Diagnostics

For granular error analysis, we track detailed diagnostics for individual classes, including F_1 , Precision, Recall, Support, and the assigned decision threshold. In particular, we monitor the absolute number of False Positives per class. This allows us to identify problematic low-support categories that might exhibit near-zero precision due to a disproportionately high volume of False Positives.

4.4 Tuning and Model Selection

The optimization process balances three sets of configurations: model prototypes (architecture and embedding choices), classifier hyperparameters, and per-class decision thresholds.

To prevent overfitting and data leakage, we use multilabel-stratified five-fold cross-validation (CV) on the training set. A fixed random seed ensures the exact same folds are used across all setups, guaranteeing fair comparisons. The test split is strictly reserved for final evaluation and for empirically supporting or refuting our claims; it does not influence the model architecture. Although test metrics are reported for consistency across all experiments, all architectural decisions are based exclusively on the results of the cross-validated development set.

Hyperparameters

We handle the three configuration levels as follows:

- **Classifier hyperparameters** of the MLP (e.g., hidden layer size, dropout rates, and hidden layer size of the attention module) are optimized

using an exhaustive grid search over the full Cartesian product of candidate values. The default search grid is:

$$\text{hidden_dim} \in \{1024, 2048, 4096, 8192\},$$

$$\text{dropout}_1 \in \{0.0, 0.1\},$$

$$\text{dropout}_2 \in \{0.0, 0.3, 0.5\}.$$

The learning rate, optimizer selection, and batch size are fixed at the baseline values inherited from the deployed origin classifier. These operational parameters were tuned in an earlier, independent optimization sweep; re-evaluating them for every prototype would enlarge the computational grid with no observable performance benefit. Because the classification head is structurally lightweight and inexpensive to train, the candidate grid remains focused, yielding deterministic and reproducible results for every experiment. This grid search requires approximately two to five hours of training time. The grid boundaries were determined empirically, guided by the configuration of the origin classifier.

- **Model-side choices** (e.g., entity embedding source, pooling strategy, whether entity elements are included, attention parameters) define distinct *model prototypes*. The hyperparameter grid search runs independently for each prototype, allowing us to compare their optimal configurations. Functionally, the model prototype configuration can be viewed as an overarching macro-parameter of the optimization space, given that the same classifier hyperparameter grid search is executed uniformly across all prototypes.
- **Per-class decision thresholds** are excluded from the main grid search due to the high number of target classes. They are calibrated inline during cross-validation, as detailed later.

Search and Selection. For each hyperparameter configuration λ , the model is trained from scratch on the CV folds using early stopping. The winning configuration, λ^* , is the one that achieves the highest mean micro-F₁ score across all five folds.

Thresholds

Even after training, a uniform decision threshold of 0.5 is poorly suited to the long-tailed label distribution because minority classes often produce lower sigmoid scores. We therefore calibrate a separate decision threshold for each IPTC category on development-fold predictions. The classifier outputs independent sigmoid probabilities for each IPTC category. Due to severe class imbalance, the loss function under-penalizes errors on infrequent labels, resulting in persistently low raw probabilities for minority classes. [69] To address this, we adjust the decision threshold per class using a grid of $\{0.25, 0.30, \dots, 0.75\}$ to maximize the micro F1-score, which is the primary evaluation metric for our prototype. We restricted candidate thresholds to the range $[0.25, 0.75]$ for two reasons. First, a lower bound of 0.25 prevents the model from over-predicting imbalanced classes and introducing excessive false positives. Second, an upper bound of 0.75 is

sufficient because the main goal of tuning is to prevent rare classes from being entirely suppressed. This upper limit is supported by the results of Experiment 1 (Section 4.5), where no optimal class threshold exceeded 0.65. Testing lower bounds of 0.10 and 0.15 in the baseline model decreased the overall micro F1-score due to an increase in false positives, so we set the lower bound to 0.25. This provides a safety margin and avoids over-tuning the threshold boundary to a specific model. The validity of this choice is supported by the results of Experiment 1 (Section 5.1.1), which show that the model with tuned thresholds outperformed the model using the default threshold.

Cross-Fitted Development Evaluation. Tuning thresholds and evaluating them on the exact same data mathematically overstates performance. To prevent this bias, we use a cross-fitting approach on the development folds, as summarized in Figure 4.3.

Each development fold is divided into two equal, multilabel-stratified halves, A and B . We tune the thresholds on subset A and evaluate them on subset B , and vice versa. Averaging the metrics of both subsets yields an unbiased threshold-aware metric for the fold, and allows us to spot incorrect threshold hyperparameter settings in the dev-set results.

Threshold Aggregation. After recording the unbiased fold metric, we re-tune the thresholds on the entire development fold. This step is strictly for test set predictions and does not affect the metrics reported in cross-validation. We average these per-fold thresholds across all five folds to create the final configuration (θ^*) for the fully retrained model, which is then used to make predictions on the test set. Categories tuned in fewer than three folds revert to a default threshold of 0.5 to prevent anomalously low thresholds derived from low support.

Applying this methodology ensures fair prototype comparison by removing the bias caused by threshold tuning. Because the evaluation thresholds are tuned on only half of the development data, they may be less stable. Consequently, the cross-validation metrics likely provide a conservative estimate of the actual performance gains from threshold tuning.

4.4.1 Number of Epochs and Early Stopping

During hyperparameter tuning, determining the optimal training duration for the final classifier is essential. To ensure a fair comparison across all models, we use an early stopping mechanism. Although the maximum training duration was set to 100 epochs, all classifiers converged and stopped before reaching this limit.

Early stopping is applied during hyperparameter optimization on the train set by monitoring the validation loss with a tolerance of 1×10^{-10} . Although standard frameworks such as scikit-learn use a much higher default tolerance (e.g., 1×10^{-4} for MLPClassifier [70]), we chose a smaller tolerance because of the class imbalance in our dataset. We anticipated that minority classes would require more training epochs, and a lower tolerance allows the model to capture late-stage improvements in these classes. This value is close to zero and can capture improvements in small classes, but remains above the precision limit of floating-point operations in Python [71].

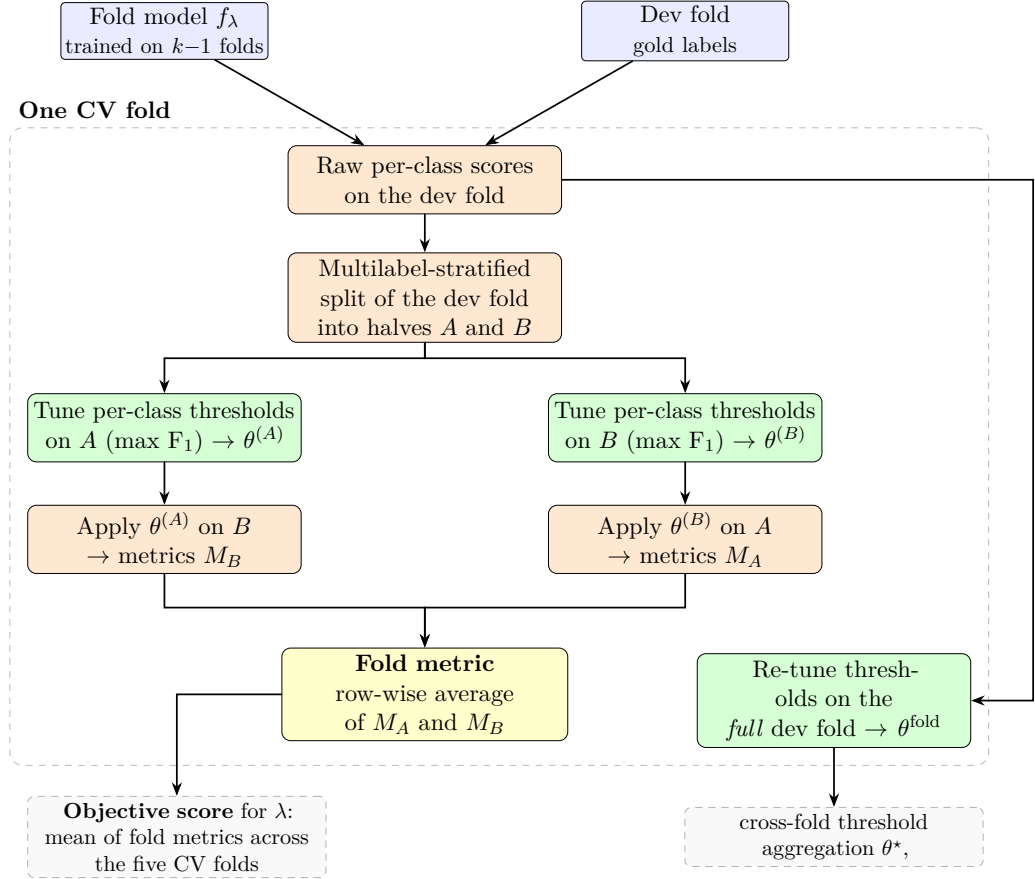


Figure 4.3 Objective-metric and threshold-tuning computation on one dev fold. The development fold is split into halves A and B . Thresholds tuned on subset A are evaluated on B , and vice versa. Averaging the two resulting metric tables prevents data leakage. The complete development fold is only reused afterward to estimate the per-fold threshold map, which feeds into the cross-fold aggregate θ^* for the final model.

The patience parameter was set to 5 epochs, chosen empirically.

This approach introduces a minor data leak, as the same validation fold (which we explicitly frame as a validation/development fold) is used for both early stopping decisions and final performance evaluation. In an ideal setup, a separate dataset would be reserved exclusively for early stopping. However, partitioning an independent set solely for early stopping would further reduce the size of the training pool, potentially degrading the model’s ability to learn minority class characteristics or lowering overall performance. We therefore accept this minor leak as a necessary trade-off. It ensures that all models are compared at their optimal training stage rather than at an arbitrarily fixed number of epochs.

Once early stopping is triggered, the model weights are restored to those of the epoch that achieved the lowest validation loss. The final cross-validation metrics for that fold are then recorded using these optimal weights.

When training the final model with the selected optimal hyperparameters, applying early stopping directly is not feasible without a dedicated validation set. Using the held-out test set for early stopping would introduce a data leak. To avoid this while still benefiting from early stopping, we log the exact number of epochs required to reach the minimum validation loss for each fold during cross-validation. The final model is then trained on the complete training dataset for a fixed number of epochs, calculated as the average number of epochs across all folds for that specific hyperparameter configuration. Because this average is typically a floating-point value, it is rounded to the nearest integer.

Alternatively, we could use the micro F1-score, our primary objective, as the criterion for early stopping. However, this approach increases the risk of overfitting because it ignores the overall loss. Furthermore, our training and testing datasets do not include data from all of our customers. To reduce the risk of overfitting and making poor predictions on unseen customer data, we base our early stopping mechanism on the validation loss.

4.5 Experiment 1: Addressing Class Imbalance

The label distribution introduced in Section 4.3.1 is heavily long-tailed. In long-tailed multilabel datasets, frequent labels contribute disproportionately to the binary cross-entropy optimization process, causing the learned classifier to become biased toward head classes. Consequently, tail labels often receive lower confidence scores and are under-predicted when a uniform decision threshold (e.g., 0.5) is used.

Before adding entities to the models, we evaluated four lightweight interventions on the origin classifier architecture. These interventions address the imbalance from different perspectives. This baseline assessment ensures that introducing entities does not simply solve a problem that could be resolved through a more straightforward approach. Because our primary objective is the investigation of entity addition, we did not pursue more time-consuming experiments.

Tested Interventions. All variants share the optimizer, batch size, and HPO grid of the baseline (Section 4.4); they differ only in a single design choice:

- **Pre-baseline.** ReLU MLP with a fixed decision threshold of 0.5. Serves as the reference point for all models in this experiment.
- **GELU activation.** ReLU is replaced with GELU in the hidden layer. The motivation is that the smoother, non-zero negative branch of GELU avoids the dead-ReLU effect, which can disproportionately suppress neurons that fire only for tail classes. [17]
- **Leaky ReLU.** A complementary check using Leaky ReLU instead of GELU. It preserves the piecewise-linear behavior of ReLU but keeps a small gradient on the negative side, again aimed at retaining gradient flow for rarely activated tail-label features.
- **Prior-initialized output bias.** The output bias of the final linear layer is initialized to the logit of each class’s prior π_c in the training set, $b_c = \log(\pi_c/(1-\pi_c))$, so that an untrained model already reproduces the empirical base rate of every label. This removes the early-training regime in which the network has to spend capacity learning to suppress tail labels. [69]
- **Per-class threshold tuning.** The post-hoc calibration described in Section 4.4 is enabled: instead of a global threshold of 0.5, each class receives an individual decision threshold τ_c tuned on the cross-fitted development folds. This intervention does not change the learned representation; it only re-balances the precision/recall trade-off per class.

We also considered implementing Focal Loss [72] and weighted Binary Cross-Entropy (BCE) loss. Although these loss functions are designed to mitigate issues associated with long-tail distributions, they would likely have a negative impact on our primary objective metric. Because micro-averaged metrics treat every sample with equal importance, applying non-uniform class weights conflicts with our evaluation goals. Standard BCE loss aligns naturally with micro-averaging because it treats all instances equally. Standard BCE also avoids the additional complexity of tuning the hyperparameters associated with weighted or focal losses. Finally, according to the original classifier’s author, these options were included for experimental purposes but did not outperform standard BCE in their benchmarks.

Outcome and Adopted Choice. Of the four interventions, only per-class threshold tuning produced a consistent improvement in the objective micro F_1 and the macro F_1 of the relevant classes; the activation swaps and the prior-initialized bias matched the baseline within cross-validation noise. Threshold tuning is therefore retained as a default component of all subsequent prototypes in this thesis, while the architecture-level variants are discarded. Detailed fold-level numbers and per-bin breakdown are presented in the results chapter.

4.6 Baseline Classification Model

This section describes the baseline model, which serves throughout the thesis as the reference point for all entity-enhanced variants. Figure 4.4 illustrates the end-to-end inference pipeline; the three dashed groups correspond to the three stages described below.

Embedding Stage. Article embeddings are computed as described in Section 4.1.1, resulting in a 384-dimensional multilingual article representation. This approach replicates the behavior of the original classifier. Although our training and testing data splits differ from the original study (Section 3.6), this configuration keeps our baseline as comparable to the original system as possible.

Furthermore, Figure 4.4 shows the feature fusion stage, which aggregates entity and article embeddings. Because the baseline model relies only on textual embeddings, this step acts as an identity function. It is included in the diagram to maintain architectural consistency with the entity-aware variants introduced later in this chapter.

MLP Classifier. The classifier is a single-hidden-layer feed-forward network:

$$\mathbf{z} = W_2 \left(\text{Dropout}_{d_2} \left(\text{ReLU} \left(W_1 \text{Dropout}_{d_1}(\mathbf{x}) + \mathbf{b}_1 \right) \right) \right) + \mathbf{b}_2,$$

with $W_1 \in \mathbb{R}^{H \times 384}$, $W_2 \in \mathbb{R}^{1005 \times H}$ and an output dimension of 1005 (one logit per IPTC class). The hidden width H and the two dropout rates d_1, d_2 are not fixed. The output of this stage is a logit for each class.

Training and Model Selection. As noted in Section 3.6, the training and testing data are slightly different from the original setup. To make the comparison with other models fair, we perform the same HPO and threshold selection as described in Section 4.4, resulting in the following parameters: hidden dim = 4096, $d_1 = 0.0$, $d_2 = 0.5$, learning rate = 0.00037.

Decision Stage. Logits $\mathbf{z}$ are turned into independent class probabilities by an element-wise sigmoid, $\hat{\mathbf{y}} = \sigma(\mathbf{z})$, with the thresholds obtained during the CV step. Because IPTC categories form a tree, the final prediction set is additionally closed under the ancestor relation before scoring: predicting a leaf category implies all of its ancestors, producing the final output of the model.

In the remainder of the thesis this baseline is held fixed and is extended only through model-side parameters. The entity branch shown in Figure 4.4 is enabled and various pooling and fusion strategies are evaluated against this model.

4.6.1 Baseline Verification and Sanity Check

To validate our data preparation pipeline, we conducted a preliminary sanity check by comparing its performance with the original baseline framework. Our model achieved a Micro F_1 -score of 0.706 on the test set, whereas the original implementation reported a Micro F_1 -score of 0.688. These results are not strictly comparable because the models were evaluated on different test sets, and our model does not include silver data. However, both test sets share an approximately identical class distribution, even though the specific articles differ. Despite this limitation, achieving a comparable score confirms that our data preparation pipeline is fundamentally sound, has not introduced any critical errors, and provides a reliable foundation for the subsequent experiments.

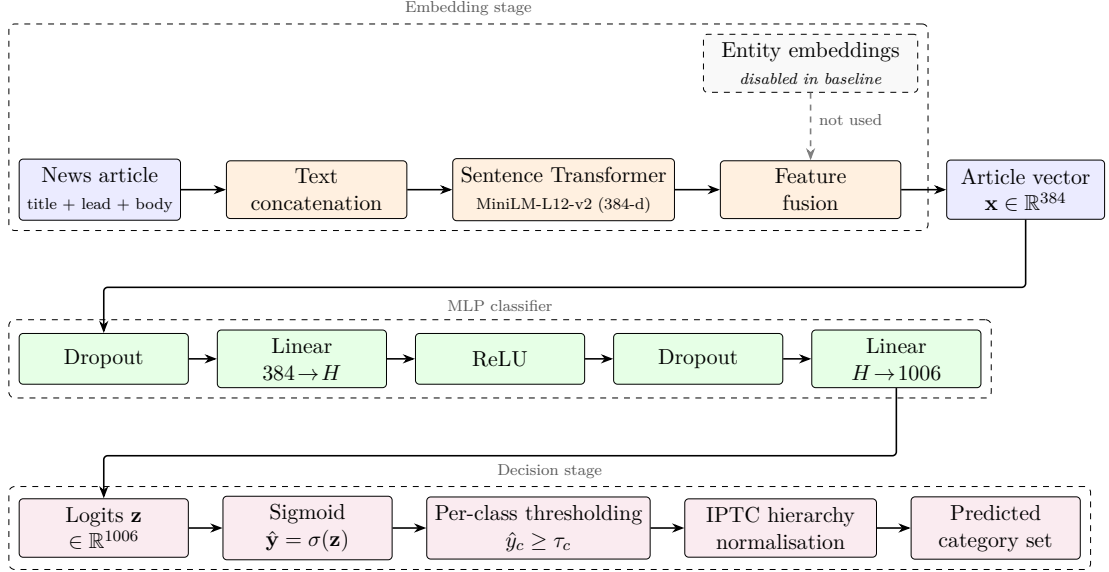


Figure 4.4 Baseline article-only IPTC classifier. Raw article text is encoded into a 384-dimensional multilingual embedding, passed to a feature fusion step, routed through a single-hidden-layer MLP, and converted to a multi-label prediction via sigmoid scores, per-class thresholds, and IPTC hierarchy normalisation. Entity embeddings are present in the architecture but disabled in this baseline.

4.7 Enhanced Entity-Aware Classifiers

All models in this section are derived from the baseline classifier described in Section 4.6 and reuse its three-stage classification pipeline (embedding $\rightarrow$ single-hidden-layer MLP $\rightarrow$ sigmoid thresholding / per-class thresholding / IPTC hierarchy normalization), shown in Figure 4.4. With one exception, every variant is trained and selected with the *same* hyperparameter search and per-class threshold-tuning protocol described in Section 4.4, so that any difference in behavior can be attributed to the architectural change rather than to the optimization budget. The exception is when we add entity attention: the attention hyperparameters must also be optimized, and running the full grid search over both attention and classifier hyperparameters would be infeasible. We therefore use the best classifier hyperparameters tuned without attention and search only over the attention parameters.

4.7.1 Embedding Selection and Multilingual Strategy

Given our computational resources, we use existing pre-trained models rather than training our own. These models are applied in one of two ways: either by generating an embedding from a single textual representation of an entity, or by aggregating multiple text embeddings into a single vector.

To construct these textual representations, we use data from Wikidata and Wikipedia. While both platforms provide multilingual data, text coverage varies considerably across languages. Many entities lack descriptions or linked Wikipedia articles in certain languages. However, approximately 90% of the entities in our dataset have an English textual representation. [73] Because extracting and processing text across all available languages is time-consuming, we focus primarily

on English. If these initial experiments yield promising results, expanding to other languages can be evaluated in future work.

4.7.2 Experiment 2: Entity Embedding Sources

The purpose of this experiment is to compare multiple sources of entity embeddings. The first entity-aware models simply switch the entity branch on. Each linked entity is represented by a precomputed embedding; the mean of the entity embeddings is concatenated with the article embedding in the feature-fusion step (Figure 4.5). The MLP input therefore grows from 384 to $384 + d_e$ dimensions, where d_e is the width of the entity embedding; nothing else changes in the pipeline. When no entity embedding is available, a default mean embedding computed on the train set is used.

We compare several entity sources that differ only in how the per-entity vector is obtained: **Wikipedia2Vec** graph-and-text embeddings, embeddings of the short **Wikidata description**, embeddings of the **Wikipedia intro** (the first paragraphs of the article), embeddings of the **whole Wikipedia article**, and the **Wikidata Embedding Project** entity representation. The individual sources are described in their respective sections; here they act as interchangeable inputs to the same fusion step.

Each source is evaluated in two regimes. In the *article+entity* regime the pooled entity vector is concatenated with the article embedding as above. In the *entity-only* regime the article branch is removed and the MLP is fed the pooled entity vector alone, testing whether the linked entities carry enough signal to predict the IPTC categories by themselves, without the article text.

We also use the same model deployed for article embeddings to generate entity embeddings from the text representation of the **Wikidata Embedding Project**. This enables a direct comparison to determine whether any observed improvements are caused by the difference in representation or by the embedding model itself.

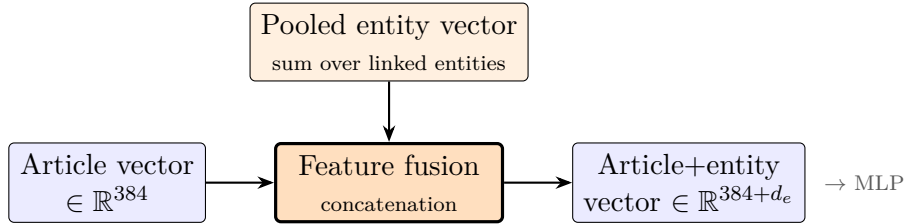


Figure 4.5 Entity embedding sources. The previously disabled entity branch is enabled: a pooled entity vector is concatenated with the article vector at the feature-fusion step (highlighted). In the entity-only regime the article input is removed and only the pooled entity vector is forwarded.

Article and Entity Vector Pooling

The question of how to fuse the article and entity embeddings also arises here. In this experiment, some article embeddings and some entity embeddings are produced by different LLMs, yielding different vector spaces of different dimensions. Their concatenation therefore mixes two unrelated coordinate systems. For this reason, we chose concatenation as the default fusion strategy for this experiment.

Summing entity and article vectors is also a valid strategy, and is often used, e.g., in “A Multilingual Bag-of-Entities Model for Zero-shot Cross-lingual Text Classification” by Yamada and Shindo [74]. We tested this option for models that use the same embedding model for both entities and articles.

Wikidata Embedding Project

The Wikidata Embedding Project is described in Section 2.3.1. We obtained the entity embeddings using the publicly available source code from the project’s GitHub repository [47]. This repository contains a pipeline that extracts text chunks, generates embeddings using the *jina-embeddings-v3* model [75], stores the vectors in Astra DB, and publishes both the cleaned Wikidata JSON and the cached vectors to Hugging Face datasets. By default, the pipeline generates embeddings for each entity and language separately, supporting English and German.

The project requires each entity to have a label, a description, and a Wikipedia page in the chosen language. Otherwise, a text representation is not generated due to insufficient data. Consequently, only 83% of the entities in the training set and 86% in the test set were successfully linked. The dimensionality of the resulting entity embeddings is 1024.

Although the default model is multilingual, it computes embeddings based on a textual representation of an entity built from the Wikidata knowledge graph in a single language. Since our dataset contains four additional languages, we extended the pipeline to support all of them. To generate these entity embeddings, we used only part of the original pipeline, specifically the text extraction and embedding computation stages.

We also integrated our internal embedding model, which is typically used to generate article embeddings. This lets us compare our approach directly against the baseline repository. This baseline introduces new text-based embeddings of knowledge graph entries using a new embedding model. In addition, by applying our internal model to the text representations extracted by their pipeline, we can isolate and evaluate the quality of the text representations themselves.

We also added support for a local vector database. This modification eliminated the financial costs associated with using an external cloud vector store and considerably accelerated the embedding process, as the generation pipeline is no longer constrained by the Astra DB API rate limits.

Wikipedia2vec Embeddings

The Wikipedia2vec framework is described in Section 2.3.3. This project provides precomputed entity embeddings, available on its official website [76]. For our experiments, we used the 500-dimensional precomputed embeddings for English, trained on the 2018 Wikipedia dump with a window size of 10.

Since our dataset spans a period of time, some entities may have changed or evolved after 2018. We initially accepted this temporal misalignment as a potential limitation, as retraining the embeddings from scratch would be computationally intensive. However, after reviewing the preliminary results, this approach did not yield notable performance improvements over our other methods, so we chose not to recompute these embeddings on the current Wikipedia dump.

The constraints for obtaining embeddings for a Wikidata item are much stricter with this method. Simply having a Wikipedia article is not enough; the entity must also appear frequently as a hyperlink within other articles. Consequently, even after averaging embeddings from multiple sources, we only achieve an entity coverage of 61.6% on the training set and 67.5% on the test set.

4.7.3 Custom Sources of Entity Embeddings

We explored multiple approaches to representing entities based on data from Wikipedia and Wikidata. As a baseline, we used our internal article embedding model to generate these entity representations, ensuring consistency across our experiments.

Wikidata Descriptions

First, we generate embeddings using only the short Wikidata description of an entity. This represents the simplest configuration and lets us evaluate whether a brief text snippet is sufficient for effective classification. Wikidata descriptions are typically concise (often a single sentence) and are occasionally missing. Because these descriptions are a core component of the text representations used in the baseline Wikidata Embedding Project, this experiment isolates whether incorporating additional Wikidata predicates introduces useful signal or merely adds noise.

To obtain entity embeddings using this approach, we used an internal Wikidata mirror built from the March 1, 2026 dump [77]. To generate an embedding for a specific Wikidata ID, the corresponding item must have a non-empty description in the chosen language. Although many items do not satisfy this condition, by averaging embeddings from multiple sources we obtained embeddings for 95.8% of the entities in the training set and 96.8% in the test set.

Full Wikipedia Articles

Second, we constructed entity embeddings from full Wikipedia articles. Although not all Wikidata entities link to a Wikipedia page, a large proportion do. For any entity connected to an English Wikipedia article, we used the entire text of that article to compute its embedding. This experiment evaluates whether full-text data provides useful context for the classifier and helps us decide whether full-text integration is worthwhile or whether we should pivot toward graph-based approaches such as GNNs. To map and extract these articles, we processed an English Wikipedia dump using the *WikiExtractor* tool [78].

To generate this representation, an item must have a Wikipedia article in the chosen language. Although not every Wikidata ID (WDID) has a corresponding Wikipedia page, we achieve 99% coverage across both the training and test sets by averaging embeddings from multiple sources. We used the `enwiki-latest-pages-articles-multistream.xml` file from the April 1, 2026 Wikipedia dump [79].

Wikipedia Introductions

Third, we experimented with embeddings derived exclusively from the introductory sections of Wikipedia articles. Wikipedia articles typically begin with an introduction that provides a dense summary of the topic. This approach tests whether the introduction alone provides enough signal, under the hypothesis that the rest of the article may contain sparse or noisy information.

We prepared this representation using the same initial dataset and extraction tools as for the full Wikipedia articles, so the preparation process and entity coverage are identical. Since no standard tool was available for isolating only the introductory sections from the raw text dump, we developed a custom heuristic based on the text layout produced by *WikiExtractor*. The specific thresholds used in this heuristic were chosen empirically, based on manual inspection of a sample validation set. The extraction pipeline follows a sequential process:

Step 1: Initial Extraction

Capture the first 3 lines of the article text automatically.

↓

Step 2: Conditional Evaluation

Continue including lines *until* encountering a line shorter than 36 characters.

↓

Step 3: Boundary Limit

Stop the extraction and cap the total output at a maximum of 7 lines.

While a line generated by *WikiExtractor* usually corresponds to a full paragraph, and introductions can span multiple paragraphs, this heuristic intentionally favors over-inclusion to ensure the information-dense summary is captured.

4.7.4 Experiment 3: Entity Pooling Strategies and Attention

The baseline models described in Section 4.6 aggregate linked entities using a simple mean, which treats all entities equally. In this experiment, we keep the rest of the pipeline unchanged and modify only the pooling function within the feature fusion step. We evaluate the following strategies:

- **Sum:** Sums entity vectors. This preserves information about the total number of entities, but leaves the magnitude of the resulting entity embedding unbounded.
- **Relevance-weighted sum:** Scales each entity embedding by its relevance score before summation.
- **Mention-weighted sum:** Weights each entity embedding by its mention frequency within the document.
- **Mean (default option):** A mean of the entity embeddings. This removes the dependence on the total number of linked entities, bounds the size of the entity embedding, and drops the information about entity count.

- **Mention-weighted mean:** A mention-frequency-weighted average, normalized by the total weight to eliminate dependence on the entity count.
- **Relevance-weighted mean:** A relevance-weighted average, normalized by the total weight to eliminate dependence on the entity count.

Entity Attention

Unlike the static heuristic pooling strategies described above, the attention-based variants learn how to weight entities during training. Instead of aggregating the entity vectors beforehand, the variable-length set of entity vectors is passed directly to the MLP model. A small attention sub-network then scores each entity conditioned on the article vector, producing a learned weighted representation that is concatenated with the article vector before entering the main (unchanged) part of the MLP classifier.

The attention scorer is a two-layer network with a sigmoid gating mechanism. Given an article vector $\mathbf{a}$ and a set of N entity vectors $\{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N\}$, the attention weight α_i for the i -th entity is defined as:

$$\alpha_i = \sigma(\mathbf{W}_2 \cdot \text{Dropout}(\text{ReLU}(\mathbf{W}_1[\mathbf{a}; \mathbf{e}_i] + \mathbf{b}_1)) + b_2) \quad (4.2)$$

where $[\mathbf{a}; \mathbf{e}_i]$ represents the concatenation of the vectors of the article and entity, $\mathbf{W}_1$, $\mathbf{W}_2$, $\mathbf{b}_1$, b_2 are the learnable weights and biases of the sub-network and σ is the sigmoid activation function. These weights are then used to compute the pooled entity vector $\mathbf{e}_{\text{pool}}$ via a normalized weighted mean:

$$\mathbf{e}_{\text{pool}} = \frac{\sum_{i=1}^N \alpha_i \mathbf{e}_i}{\sum_{i=1}^N \alpha_i} \quad (4.3)$$

This approach allows the model to down-weight entities that are irrelevant to a specific article while keeping the embedding bounded.

We also evaluated several alternative attention mechanisms. To avoid time-consuming HPO, we tested a few randomly selected hyperparameter configurations for each of the following approaches:

- **Attention with Entity Gate:** Uses the same per-entity pooling strategy as the baseline, but introduces an additional sigmoid gating mechanism to scale the pooled entity vector prior to concatenation.
- **Two-Stage Softmax Fusion:** Applies the same pooling strategy as above, but instead of using a sigmoid to scale only the entity vector, it uses a softmax to scale both the article and entity vectors.
- **Single-Head MHA:** Uses standard attention pooling with the article embedding as the query and the entity embeddings as the keys and values within a single attention head.
- **8-Head MHA:** Extends the single-head architecture by using eight parallel attention heads. This allows the model to capture richer interactions between the article and entities, at the cost of increased parameter complexity.

Surprisingly, the most basic attention mechanism achieved the best performance. We hypothesize that this is due to its lower parameter count. The more complex, classic transformer-based attention (MHA) likely lacked sufficient training samples to properly distinguish high-quality entity embeddings. Introducing a gate to explicitly weight the entity or article vectors also did not improve results, likely because the subsequent hidden layers already learn such weighting relationships. Adding explicit gating mechanisms only introduced redundant parameters that required training and additional hyperparameters to tune.

Source of Entities

We selected entity embeddings from the Wikidata Embedding Project, as they achieved the highest performance in Experiment 2 (Section 4.7.2). To determine the most effective pooling operation, we evaluated several methods and found that the attention mechanism yielded the best results. We then evaluated this attention mechanism across four distinct embedding sources for comparison:

1. Wikidata Embedding Project embeddings generated via the Jina v3 model,
2. Wikidata Embedding Project embeddings generated via our default article embedding model,
3. Wikipedia2vec embeddings, and
4. full Wikipedia article embeddings.

To streamline our evaluation, we excluded embeddings generated solely from Wikipedia article introductions, as representations using the full article consistently provided better context and performance. For the same reason, we excluded Wikidata descriptions, because their text representations are subsets of the broader Wikidata Embedding Project. This filtering allowed us to avoid redundant testing and focus computational resources on the most promising methods.

4.7.5 Experiment 4: Adding Different Languages

When incorporating multiple languages into our classification pipeline, we considered two strategies:

1. **Multilingual Pooling:** This involves aggregating embeddings across all available languages for a single entity. The main disadvantage of this approach is the risk of introducing noise, as the quality and depth of information vary across languages. We hypothesized that this strategy would be inferior. Initial empirical testing with a single hyperparameter combination supported this assumption, leading us to adopt the second strategy.
2. **Default with Fallback:** This strategy uses a single default language (e.g., English) as the primary source, falling back to alternative languages only when the primary text is missing. Because better-supported languages typically have higher-quality Wikidata and Wikipedia articles [80], this approach allows us to generate representations for more entities, adding richer information to our entity embeddings.

We proceed with the fallback strategy and tested the following language priority variants:

- **[en, de, fr, nl, es, cs]:** Ordered from the most to the least represented language on Wikipedia, to maximize support.
- **[en, de]:** The two most represented languages, chosen to avoid potential noise from smaller, less curated wikis.
- **[en, nl]** and **[nl, de]:** English is the most represented language on Wikipedia overall, while Dutch (nl) is the most represented language in our specific dataset. We compare these pairs to test whether the priority order of the languages makes a noticeable difference in performance.

4.7.6 Experiment 5: Summation vs. Concatenation

In the study by Yamada et al. [74], the authors sum the entity and word embeddings instead of concatenating them, as this approach yielded better classification results. We can apply the same technique to our article and entity embeddings. However, this operation is only valid when the embedding models for both the entities and the articles are identical, so that the combined embeddings belong to the same vector space [81].

Consequently, this method cannot be applied when using embeddings from different sources, such as combining Wikipedia2Vec with the Jina-v3 model. To determine the most effective approach, we evaluate two versions of the classifier: one using vector summation and the other using vector concatenation.

The models evaluated in this experiment are defined as follows:

- **Wiki Article:** Uses entity embeddings generated from full Wikipedia articles.
- **Wiki Intro:** Uses entity embeddings generated exclusively from the introduction section of Wikipedia articles.
- **Wikidata Description:** Uses entity embeddings derived from Wikidata descriptions.
- **WP Entities PMM:** Uses entity embeddings from the Wiki Project representation of entities, embedded with the same model used for the article embeddings.

4.7.7 Code

The source code is distributed across three public Git repositories. The main experimental pipeline, including a data sample for testing, is available at <https://github.com/ProkopDivin/entity-enhance-classification>. Its README describes the installation, configuration, and execution of the experiments, and links to the two supporting repositories used to prepare entity representations.

The implementation and reuse of code are summarized below:

- **Pipeline architecture:** The main repository implements a six-stage pipeline which is described in Section 4.1.
- **Components implemented for this thesis:** The data layer loads the corpus, parses linked entities, maps entity identifiers to Wikidata identifiers, normalizes IPTC labels, and prepares the data for training. The implementation also includes entity-vector loading, pooling, and feature construction through combination with article embeddings; multilabel five-fold cross-validation; per-class threshold tuning; typed configuration; pipeline orchestration; and final-results processing. The latter includes result comparison and statistical testing.
- **Reused components:** The pipeline was inspired by an earlier Python implementation. Its `legacy_reuse.py` module ports the previous training and evaluation routines to preserve their behaviour and compatibility with the earlier pipeline. The neural-network architectures, corpus data structures, IPTC utilities, and selected evaluation utilities are provided by the internal `geneea` package and were not developed as part of this thesis.
- **Wikidata entity representations:** The WikiProject embeddings are generated by the WikidataTextEmbedding project. This is a fork of an existing project that was adapted to produce the Wikidata-based representations required in this study. Its processing of a part of the Wikidata dump requires at least 120 GB of free disk space.
- **Wikipedia entity representations:** Wikipedia article and introduction text is extracted by the separate `extractWikiArticles` project. The extracted text is subsequently embedded by the main pipeline. Processing the relevant part of the Wikipedia dump requires at least 250 GB of free disk space to accommodate decompression. Detailed instructions for both entity-representation projects are provided in their respective README files.

Experimental Hardware and Software

All experiments were conducted on a workstation provided by Geneea. The system ran Ubuntu 22.04.5 LTS (Linux kernel 5.15.0-173) and was equipped with an Intel Core i5-13500 processor (14 cores and 20 threads), 62 GB of RAM, and an NVIDIA RTX 4000 SFF Ada Generation GPU with 20 GB of VRAM. The experimental pipeline was implemented in Python 3.10.12 using PyTorch 2.10.0 with CUDA 12.8.

5 Results

This chapter reports the empirical outcomes of the experiments defined in Chapter 4. It first establishes a text-only baseline on the development set, then evaluates entity-aware variants to select a final configuration, and finally measure that model performance on the held-out test set.

5.1 Establishing the Baseline Model

To properly evaluate the performance of our proposed models, we first establish a baseline.

5.1.1 Experiment 1: Addressing Class Imbalance

During the exploratory data analysis, we observed that the dataset exhibits a considerable class imbalance. We therefore first investigated whether this issue could be mitigated through standard, lightweight interventions before introducing entity-based modifications.

Name	Micro F1 (%)	Micro F1 σ (%)	Macro F1 (%)	Macro F1 σ (%)
Article Only	70.7	0.17	54.0	0.45
Article Only + Prior	70.3	0.18	52.8	0.55
Article Only + GELU	70.3	0.10	53.4	0.37
Article Only + Leaky ReLU	70.7	0.17	54.0	0.46
(Base) Article Only + Tuned	71.2	0.17	56.3	0.27

Table 5.1 Class imbalance experiments (Dev). Comparison of activation-function variants on the development set. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold marks the highest value in each column.

As shown in Table 5.1, initializing the output bias with class priors did not improve performance. This outcome likely occurs because minority classes have very low prior probabilities, resulting in a low initial bias. Consequently, it is difficult for their predicted logits to exceed the activation threshold. Furthermore, the limited number of positive samples does not generate sufficient gradient updates to overcome this initialization during training. Conversely, when we do not use prior initialization, the default bias starts higher. But because negative samples are highly abundant, the network easily learns to suppress the baseline output bias for these classes when necessary.

The GELU activation function achieved a Micro F_1 score identical to the model with prior activation, but a slightly higher Macro F_1 score. However, this difference lies within 1.5 standard deviations, which could indicate that the GELU activation treats small classes marginally better than large ones. Leaky ReLU performed identically to the standard ReLU baseline, further indicating that altering the activation function does not improve either micro or macro performance.

In contrast, per-class threshold tuning clearly improved classification performance, yielding an increase of 0.5% in the Micro F_1 metric and 2.3% in the

Macro F_1 metric. While the small improvement in the Micro score reflects a more optimized global trade-off between precision and recall, the larger increase in the Macro metric shows that threshold tuning effectively compensates for the lack of positive training samples in minority classes.

This effect is clearly visible in the selected decision thresholds. The average threshold across the 344 relevant categories included in the Macro F_1 calculation is 0.325 ($\sigma = 0.0582$), substantially lower than the default of 0.5. This downward shift is a general characteristic of the model rather than a property exclusive to rare topics: only three categories required a decision threshold greater than or equal to 0.5.

These results confirm that threshold tuning is an effective strategy for mitigating class imbalance and that the lower bound for threshold tuning is set correctly. We therefore adopt this configuration as our baseline model. Based on this evaluation, the underlying architecture for model training and evaluation remains identical to that of the original classifier.

5.2 Choosing the Best Model

With the baseline fixed, we follow with evaluation of entity-aware extensions on the development set. Each experiment isolates one design choice, such as embedding source, pooling strategy, multilingual fallback, or fusion operator. Then, the final configuration is selected.

5.2.1 Experiment 2: Entity Embedding Sources

We first compare alternative sources of entity embeddings, evaluating each representation both in isolation and when fused with article embedding. The pooling operation for entity embeddings remains mean, as it is the default choice.

Name	Micro F1 (%)	Micro F1 σ (%)	Macro F1 (%)	Macro F1 σ (%)
WP Entities (only)	66.7	0.15	49.4	0.34
WP Entities PMM (only)	65.5	0.15	47.1	0.24
Wikipedia2Vec (only)	66.4	0.20	49.5	0.29
Wikidata Description (only)	65.3	0.18	46.9	0.52
Wiki Intro (only)	65.7	0.22	47.4	0.44
Wiki Article (only)	66.0	0.35	47.7	0.66
WP Entities	72.5	0.14	58.4	0.11
WP Entities PMM	72.2	0.10	58.3	0.40
Wikipedia2Vec	72.6	0.12	58.8	0.39
Wikidata Description	72.3	0.13	58.2	0.22
Wiki Intro	72.3	0.14	58.3	0.18
Wiki Article	72.2	0.10	58.5	0.26
Base	71.2	0.17	56.3	0.27

Table 5.2 Entity embedding source experiments (Dev). Comparison of entity embedding sources on the development set. Rows ending in *(only)* use entity embeddings without article text; the remaining rows combine entity embeddings with article embedding. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold marks the highest value in each column.

In this experiment, we evaluate various sources of entity embeddings and analyze their performance, both in isolation and when combined with article

embeddings. Table 5.2 presents the results on the development set, comparing these representations against our baseline classifier.

First, we examine the performance of models using only entity embeddings (denoted *only*). While these models classify articles reasonably well, they perform worse than the baseline. This gap can be attributed to several factors: the NER system may miss entities or make linking errors, and some extracted entities may be unrelated to the core topic of the article. However, the best entity-only configuration (*WP Entities (only)*) achieves a Micro F_1 score of 66.7%. The fact that this is only about 5% lower than the article-text baseline suggests that entity representations alone already carry a strong predictive signal.

All entity-aware approaches outperformed the baseline model, demonstrating that entities provide valuable additional information for classification. The performance across these models is generally consistent, with the two best-performing models being Wikipedia2Vec and WP Entities. These achieved Micro F_1 scores of 72.59% and 72.47%. The marginal 0.12% difference between them is smaller than their standard deviations, indicating it is not statistically significant. However, there is a noticeable performance gap between these two main models and the remaining entity-based approaches, which plateaued at a Micro F_1 score of approximately 72.25%.

The performance difference between WP Entities and WP Entities PMM can likely be attributed to the Jina v3 model, which uses a larger context window and more advanced embedding techniques.

Then, we compared different components of the entity representations to justify our modeling choices. We found that incorporating Wikidata properties alongside textual metadata (*WP Entities PMM*, 72.2%) does not yield notably better performance than embedding the description alone (*Wikidata Description*, 72.3%). This suggests that structural properties are not used effectively by the current MLP classifier and may even introduce noise. Finally, comparing the full Wikipedia article (*Wiki Article*) against the Wikipedia introduction (*Wiki Intro*) indicates that the summary in the introductory section alone provides a sufficient signal for the model.

All evaluated models improved the Macro F_1 score. The models achieved an improvement of approximately 1% in Micro F_1 and roughly 2% in Macro F_1 . The larger proportional gain in the macro-averaged metric indicates that incorporating entities makes the models more effective at predicting low- and mid-frequency classes.

Table 5.3 presents the seven corpora that demonstrated the most significant improvement over the baseline model. Detailed results for all evaluated corpora are available in the Appendix (Table A.7).

A surprising observation is that all entity-aware approaches improved classification performance on the Eurosport corpora. This is notable because the baseline model’s performance on these specific datasets is approximately 5 to 6 percentage points lower than its global average. This improvement is likely driven by the high discriminative quality of the entities in sports journalism. For example, identifying a specific athlete in an article often directly indicates the sport being played, which provides a strong, unambiguous signal to determine the correct IPTC category.

Furthermore, the Eurosport datasets were the only corpora where using entity

Name	Es(ES)	Es(DE)	Es(NL)	Es(EN)	DPA(DE)	NoordHB(NL)	DNA(NL)
WP Entities (only)	+3.3	+2.5	+2.6	+1.6	-3.8	-5.1	-4.5
WP Entities PMM (only)	+2.4	+1.5	+1.4	+1.1	-5.5	-6.5	-6.6
Wikipedia2Vec (only)	+2.6	+1.7	+1.3	+1.3	-4.7	-5.4	-4.9
Wikidata Description (only)	+2.8	+1.5	+1.0	+1.0	-5.7	-6.6	-6.7
Wiki Intro (only)	+2.6	+1.3	+0.9	+0.8	-5.5	-6.2	-5.7
Wiki Article (only)	+2.5	+1.8	+1.6	+1.0	-5.0	-6.0	-5.3
WP Entities	+3.9	+2.2	+2.5	+1.3	+1.8	+1.7	+1.9
WP Entities PMM	+3.6	+2.1	+2.8	+1.2	+2.2	+1.5	+1.2
Wikipedia2Vec	+4.3	+2.7	+2.9	+1.9	+2.9	+2.1	+2.1
Wikidata Description	+3.9	+2.1	+2.6	+1.5	+1.9	+1.6	+1.6
Wiki Intro	+3.5	+2.0	+2.5	+1.2	+1.9	+1.6	+1.4
Wiki Article	+3.2	+2.1	+3.0	+1.1	+2.1	+1.5	+1.5

Table 5.3 Per-corpus $\Delta F1$ vs. baseline (Dev). Per-experiment $\Delta F1$ (%) vs. the baseline (*Article Only (Tuned)*) for six selected corpora. Green cells indicate improvement, red cells indicate degradation; colour intensity is proportional to the magnitude (saturates at ± 4 percentage points).

embeddings *in isolation* outperformed the textual baseline. In contrast, combining article and entity embeddings improved performance in almost all test cases. Two minor exceptions occurred with the Mediahuis (NL) corpus, which showed a performance degradation of 0.09% when using the WP Entities PMM model and a degradation of 0.31% with the Wiki Article model. These performance drops suggest that excessively broad entity representations can sometimes introduce noise into the model. This hypothesis is supported by the performance of more focused models: when using narrower representations, specifically Wikidata descriptions (a subset of WP Entities PMM) or Wikipedia introductions (a subset of Wiki Articles), the model instead achieved positive gains of 0.76% and 0.20%. This indicates that a stricter entity filtering or weighting mechanism could be a worthwhile optimization.

5.2.2 Experiment 3: Entity Pooling Strategies and Attention

The primary objective of this experiment is to identify an optimal pooling strategy and to investigate the influence of the attention mechanism. For our evaluation, we selected the most successful entity embedding model, WP Entities.

Name	Micro F1 (%)	Micro F1 σ (%)	Macro F1 (%)	Macro F1 σ (%)
WP Entities	71.8	0.17	57.1	0.52
WP Entities (rel. weighted sum)	67.5	0.18	49.6	0.51
WP Entities (mention weighted sum)	71.3	0.19	56.3	0.39
WP Entities (mean)	72.5	0.12	58.4	0.11
WP Entities (rel. weighted mean)	72.5	0.11	58.6	0.26
WP Entities (mention weighted mean)	72.5	0.07	58.6	0.23
WP Entities (attention)	72.9	0.06	58.9	0.30
Base	71.2	0.17	56.3	0.27

Table 5.4 WP Entities pooling strategies (Dev). Comparison of heuristic entity pooling operators on WP Entities embeddings on the development set. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold marks the highest value in each column.

As shown in Table 5.4, models using a summation strategy perform noticeably worse than those using mean-based pooling. This degradation is most pronounced

in the *WP Entities (rel. weighted sum)* configuration. This performance drop is likely because simple summation lacks an upper bound on vector magnitude. This unboundedness also explains why the relevance-weighted sum performed the worst: relevance scores generally range from 0 to 100, creating significantly higher variance than weighting by mention counts, which are typically small integers.

In contrast, the three mean-based models demonstrate highly similar performance. A closer examination reveals that the mention-weighted mean achieved a Micro F1 score of 72.51%, while both the unweighted mean and the relevance-weighted mean achieved 72.47%. This suggests that bounding the embedding vectors is more critical for model stability than the specific weighting metric applied. Furthermore, the competitive performance of the unweighted mean is unexpected, as we initially hypothesized that relevance scores would provide a stronger classification signal. However, we cannot conclusively identify the mention-weighted mean as the superior model. The performance differences among these three variants fall within the standard deviation, indicating these minor fluctuations may result from random variance.

Overall, all mean-pooling variants outperformed the sum-pooling models, confirming that averaging is a superior entity aggregation strategy for this architecture.

The most effective pooling strategy is attention, yielding the highest Micro F1 score of 72.85%. This result motivated us to evaluate the attention mechanism across different entity embedding sources. As presented in Table 5.5, while the *WP Entities* model remains the best performing in Micro F1, *Wikidata Description* (72.75%) and *Wiki Intro* (72.77%) achieve competitive results.

Name	Micro F1 (%)	Micro F1 σ (%)	Macro F1 (%)	Macro F1 σ (%)
WP Entities (attention)	72.9	0.06	58.9	0.30
WP Entities PMM (attention)	72.5	0.17	58.3	0.19
Wikipedia2Vec (attention)	72.5	0.04	59.0	0.18
Wikidata Description (attention)	72.8	0.19	58.8	0.57
Wiki Intro (attention)	72.8	0.04	59.2	0.26
Wiki Article (attention)	72.5	0.13	58.7	0.40

Table 5.5 Entity embedding source experiments with attention (Dev). Comparison of entity embedding sources with attention as pooling strategy on the development set. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold marks the highest value in each column.

5.2.3 Experiment 4: Adding Different Languages

As previously noted in the methodology, approximately 90% of the entities in our dataset possess an English textual representation. Because processing text across all available languages is highly time-consuming, we initially decided to focus primarily on English. However, to determine whether incorporating additional languages could improve classification performance, we experimented with several language fallback strategies.

Table 5.6 presents the cross-validation results of the Wikipedia Entities representations using these different fallback configurations on the development set. The *All langs* strategy uses a complete fallback chain (en→de→fr→nl→es→cs),

meaning that if an English representation is missing, the system sequentially searches for German, French, and so on.

Name	Micro F1 (%)	Micro F1 σ (%)	Macro F1 (%)	Macro F1 σ (%)
WP Entities All langs	72.53	0.12	58.49	0.18
WP Entities en→de	72.48	0.15	58.44	0.24
WP Entities en→nl	72.46	0.15	58.37	0.08
WP Entities nl→en	72.45	0.12	58.38	0.19
WP Entities (English only)	72.47	0.14	58.39	0.11

Table 5.6 WP Entities language fallback experiments (Dev). Comparison of WP Entities representations with different language fallback strategies on the development set. Each row uses entity embeddings from the Wikipedia project combined with article text. *All langs* means the fallback chain en→de→fr→nl→es→cs: if no English representation exists for an entity, German is tried next, then French, and so on. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold marks the highest value in each column.

The *All langs* configuration achieved the highest overall scores, reaching a Micro F1 of 72.53% and a Macro F1 of 58.49%. However, the performance differences between this approach and the baseline *English only* strategy (Micro F1 of 72.47%) are negligible. The measured improvements fall well within the standard deviation of the cross-validation runs.

Furthermore, a detailed inspection of the results at the per-corpus level revealed no significant improvements for any specific corpora. Because these gains are marginal, there is no strong empirical motivation to pursue further testing with multilingual entity embeddings. We conclude that while integrating multiple languages via a fallback strategy has the potential to add minor value.

5.2.4 Experiment 5: Concatenation vs. Summation

After selecting the entity representations and pooling strategy, we compare two ways of combining article and pooled-entity vectors: concatenation and element-wise summation. Table 5.7 reports the development-set results for representative embedding sources.

Name	Micro F1 (%)	Micro F1 σ (%)	Macro F1 (%)	Macro F1 σ (%)
Wiki Article	72.2	0.10	58.5	0.26
Wiki Article (sum)	71.9	0.14	58.0	0.25
Wiki Intro	72.3	0.14	58.3	0.18
Wiki Intro (sum)	71.9	0.09	57.5	0.32
Wikidata Description	72.3	0.13	58.2	0.22
Wikidata Description (sum)	71.8	0.10	57.4	0.34
WP Entities PMM	72.2	0.10	58.3	0.40
WP Entities PMM (sum)	71.9	0.10	57.7	0.26
Base	71.2	0.17	56.3	0.27

Table 5.7 Comparison of concatenation vs. summation for fusing article and entity embeddings on the development set. Rows without a suffix use concatenation; rows marked *(sum)* add the article and pooled-entity vectors element-wise. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold text indicates the highest score in each column.

As shown in Table 5.7, while all evaluated models outperform the baseline, the summation strategy consistently performs worse than concatenation. This contradicts the findings of Yamada et al. [74], which originally inspired this experiment.

This discrepancy is likely due to the different contexts of our tasks. Yamada et al. summed the embedding of a specific word directly with its linked entity. In contrast, our approach sums a single embedding representing the entire article with a pooled embedding representing all of its extracted entities. Because the full article embedding already contains the most critical information for the classification task, adding the pooled entity vector directly to it may introduce noise. Concatenating the vectors, however, allows the classifier to treat the article text and the entity information as distinct signals. This makes it easier for the model to weigh their relevance separately, despite the resulting increase in parameters.

5.2.5 Our Choice

Based on the results from the development set, we selected the WP Entities (attention) model with fallback strategy for other languages as our final configuration. Although the improvements observed in Experiment 4 (5.2.3) were not statistically significant, they indicated a minor performance gain. Furthermore, the standard deviation of the F1-score decreased, suggesting that the model is more stable.

5.3 Test Set Results

We evaluated the best-performing configurations for each entity embedding source on the held-out test set. In all cases, the top-performing models used the learned attention pooling mechanism, followed closely by models that weighted entity embeddings by the number of their mentions within the article (Mention-Weighted Mean). The performance comparison against the baseline is summarized in Table 5.8.

As shown in Table 5.8, all models incorporating entity features from external sources consistently outperformed the text-only baseline classifier. The overall best-performing model, *WP Entities All langs (att.)*, uses learned attention pooling combined with a multilingual fallback strategy across all available datasets. This configuration achieved the highest performance on both the development and test sets.

Our best model outperformed the baseline by 1.7% in the Micro F_1 score and by 2.4% in the Macro F_1 score on classes with a support of ≥ 15 . The *WP Entities (M.W.M)* variant achieved an identical Micro F_1 score of 72.3%. This shows that a simple mention-based heuristic weighting can be competitive with more complex attention mechanisms for overall instance-level predictions. However, this model lags behind the top model in the Macro F_1 score by a margin of 0.5%.

Overall, the performance across most entity-aware models remains highly similar, with the *Wikipedia2Vec (att.)* model slightly behind the rest. The results suggest that expanding the context window to include full Wikipedia articles, rather than just their introductions (*Wiki Intro*), does not yield performance

Name	Micro F1 (%)	Macro F1 (%)
WP Entities (M.W.M)	72.3	59.1
WP Entities PMM (M.W.M)	71.8	58.7
Wikipedia2Vec (M.W.M)	71.9	59.0
Wikidata Description (M.W.M)	71.7	58.5
Wiki Intro (M.W.M)	71.7	59.2
Wiki Article (M.W.M)	71.7	58.5
WP Entities (att.)	72.2	59.1
WP Entities PMM (att.)	72.1	59.0
Wikipedia2Vec (att.)	72.0	58.9
Wikidata Description (att.)	72.2	59.2
Wiki Intro (att.)	72.2	59.4
Wiki Article (att.)	72.2	59.2
WP Entities All langs (att.)	72.3	59.6
Article Only (Tuned)	70.6	57.2

Table 5.8 Final model comparison on the test set. Comparison of selected models on the held-out test set. The baseline uses article text only; subsequent rows combine article text with entity embeddings using entity embedding made by weighting entities by their number of mentions (M.W.M) or learned attention pooling. Metrics are read from the final test evaluation. Bold marks the highest value in each column among rows with available results.

gains and may introduce unnecessary semantic noise. While the test set results remained identical between the two approaches, we observed a slight performance degradation on the development set when using full articles.

Furthermore, our development phase indicated that the choice of the underlying embedding model introduces only minor performance variations. From this, we deduce that the primary value added by external entities lies within the core item descriptions themselves, regardless of whether these descriptions originate from a Wikipedia introduction or a Wikidata entry. A more detailed diagnostic analysis of our best-performing model, *WP Entities All langs (att.)*, is provided in the following section.

5.3.1 Best Model vs. Baseline

Beyond aggregate F_1 scores, we compare the selected model, *WP Entities All langs (att.)*, with the tuned text-only baseline along several diagnostic dimensions. The following analysis examines corpus and class-level effects, statistically significant changes, and performance across class-support levels.

Corpora Results

We first examine whether entity enrichment benefits all news corpora equally. Figures 5.1 and 5.2 show the per-corpus Micro F_1 difference between the best entity-aware model and the baseline, split into the largest and smallest gains.

Figures 5.1 and 5.2 illustrate the per-corpus performance improvements of the entity-enhanced model compared to the baseline. Above-average improvements

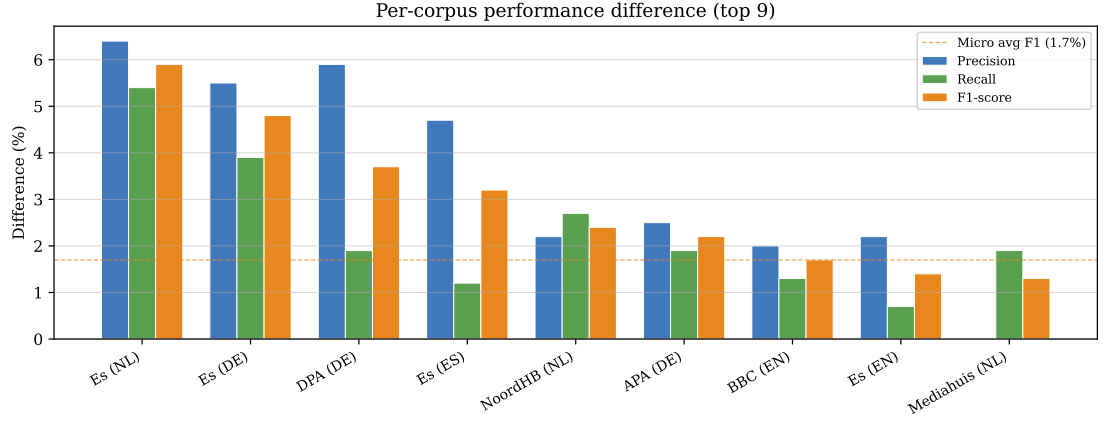


Figure 5.1 Per-corpus performance difference between the entity-enhanced and base classifier (top 9 corpora by micro F_1 -score improvement). Dashed horizontal lines indicate the micro-averaged difference across all corpora.

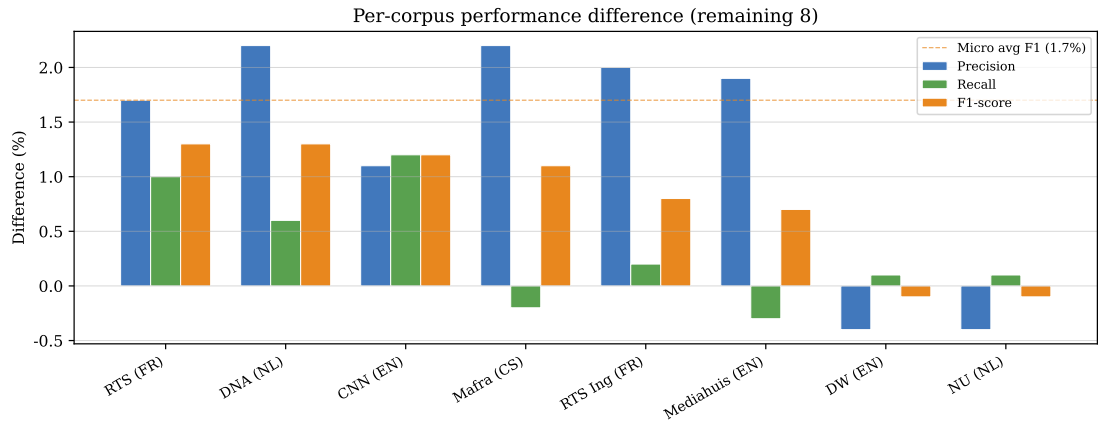


Figure 5.2 Per-corpus performance difference between the entity-enhanced and base classifier (remaining 8 corpora by micro F_1 -score improvement). Dashed horizontal lines indicate the micro-averaged difference across all corpora.

are observed in the Eurosport datasets (Dutch, German, and Spanish), as well as in DPA (German), Noordhollands Dagblad (Dutch), and APA (German).

The results show a large improvement in sports-related corpora, where more than 99% of articles have sports-related topics. Datasets such as DPA and Noordhollands Dagblad, which contain approximately 50% fewer sports topics than the average article, still exhibit sizeable gains. The DPA corpus contains a higher proportion of health-related topics and fewer articles on disasters, crime, and society. In contrast, the Noordhollands Dagblad corpus exhibits the opposite distribution. The APA dataset lacks coverage in environmental and health topics, but benefits from a high volume of sports topics. This suggests that sports topics are not the only drivers of performance gains. Although no other clear topical patterns emerged at this level of aggregation, a detailed class-level analysis is provided in Section 5.3.1.

Conversely, the DW (English) and NU (Dutch) corpora are the only datasets where performance degraded, though the 0.1% decrease in F_1 -score is nearly negligible. These corpora contain fewer sports topics, and this degradation may correlate with their higher proportion of crime and conflict articles, which also appeared more often in the lower-performing corpora.

Interestingly, the English Eurosport dataset showed a much smaller improvement of just 1.4%, compared to the 3–6% gains seen in other Eurosport corpora. This is unlikely due to poor entity or embedding quality. Instead, it is likely constrained by a ceiling effect, as the baseline F_1 -score for English Eurosport is already high at 92%. This leaves less room for improvement compared to the 85–87% baseline scores of the other Eurosport datasets. Despite the smaller margin of improvement, English Eurosport remains the highest-performing corpus overall.

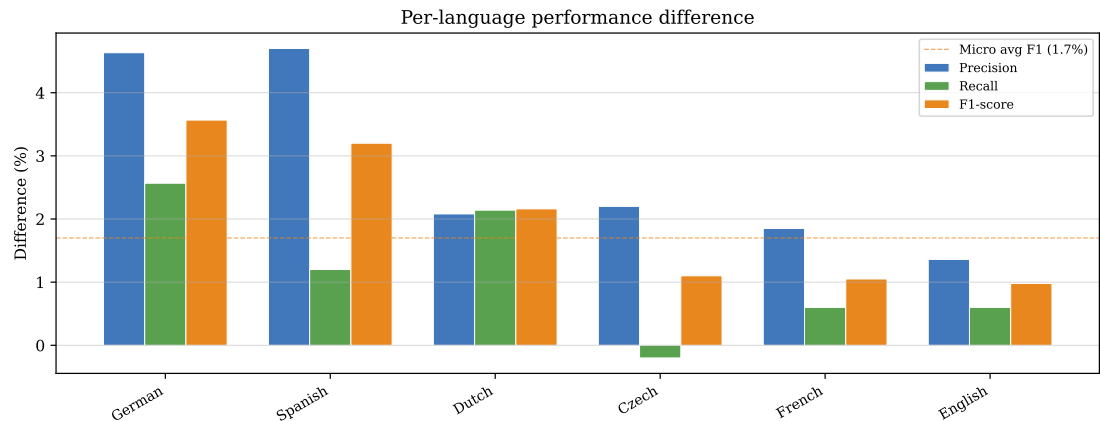


Figure 5.3 Per-language macro-averaged performance difference between the entity-enhanced and base classifier. Dashed horizontal lines indicate the micro-averaged difference.

Figure 5.3 shows that English recorded the lowest overall improvement. This is likely due to the poor performance on the DW and Mediahuis corpora, suggesting that entities provided minimal benefit for classification in these specific datasets. Spanish exhibited an above-average improvement, driven entirely by its single corpus, Eurosport. Another language represented by a single corpus is Czech, which ranks fifth from the bottom among the evaluated corpora. This lower ranking

is likely due to a lack of sports-related articles and an increased proportion of texts focused on human interest and the arts.

German achieved the largest overall improvement. Among the German datasets, only one had a baseline score below the average (69%), while the other two had relatively strong baseline scores of 78% and 87%.

Interestingly, one might expect corpora with above-average baseline scores to show smaller improvements due to a performance ceiling. However, five of these high-performing corpora actually achieved above-average gains. Specifically, the average improvement for these six corpora was 3.3%, compared to 1.1% for the remaining datasets, with the macro-averaged cross-corpus improvement sitting at 1.9% (distinct from the 1.7% micro-averaged F_1 -score across all articles). This indicates that entity enrichment does not benefit all topics equally; rather, it provides substantial boosts to specific domains, such as sports, regardless of the starting baseline.

Class-Level Results

To complement the corpus-level view, we aggregate test-set performance by IPTC top-level topic. Figure 5.4 summarizes macro-averaged F_1 improvements per topic and indicates the number of qualifying subclasses.

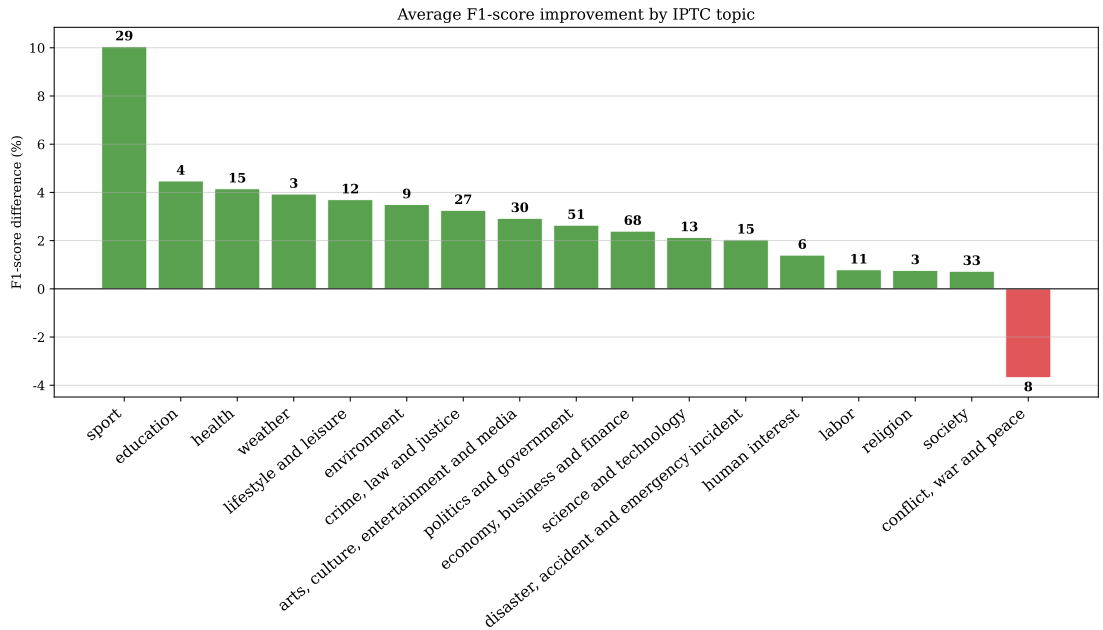


Figure 5.4 Macro-averaged F_1 -score improvement per IPTC top-level topic. Numbers above bars indicate the count of qualifying subclasses (support ≥ 15).

Only one top-level topic, *conflict, war and peace*, exhibited a negative average score. This group contains eight subcategories, including *act of terror* (20000053), *war* (20000062), *demonstration* (20000066), *peace process* (20000073), and *armed conflict* (20000056). This performance drop is likely due to the vocabulary used in conflict reporting. Text embeddings easily capture descriptive keywords (e.g., “invasion,” “shelling,” “troops,” “missiles”) that strongly indicate a conflict but are not extracted as entities. In contrast, the extracted entities are often prominent figures or organizations (e.g., Vladimir Putin, Volodymyr Zelenskyy, NATO,

United Nations, European Central Bank). Because these entities also appear frequently in political, diplomatic, and economic contexts, they introduce noise. The attention mechanism struggles to filter this noise because these entities are common and statistically relevant across multiple domains.

This result might also be overly pessimistic. The topic contains only eight classes, and this comparison uses a macro average over corpora regardless of class size. Furthermore, most of the performance drop is driven by a single subcategory, *peace process (20000073)* with a support of 17, which degraded by 25.8%.

Conversely, the 29 subcategories under the *sport* topic showed the highest average improvement, increasing by nearly 10%. The greatest gains occurred in specific sports, such as *speed skating (20001067)*, *track and field (20000827)*, *futsal (20001393)*, *ski jumping (20001061)*, and *mountain biking (20000903)*. Based on a qualitative review of the articles, we hypothesize that entity embeddings help the model differentiate between specific sports. For instance, articles frequently mention athletes by name without explicitly stating their sport.

This hypothesis is supported by the performance of the parent class, *competition discipline (20000822)*, which gained only 9 additional True Positives (TP). In contrast, its child classes with a support of over 15 samples collectively gained 176 TP. For certain classes, this represents an increase of up to 37% in TP.

Other topics with smaller average gains include *human interest (08000000)*, *labor (09000000)*, *religion (12000000)*, and *society (14000000)*, which aligns with our expectations based on the results for individual corpora. The remaining topics showed an improvement ranging from 2.0% to 4.1%. Because these performance gains are similar, it is difficult to detect other distinct patterns across individual corpora, aside from changes in the *sport* topics.

When we examine the results of the best-performing models for each source of entity embeddings (Wikidata descriptions, Wikipedia2Vec, etc.), we observe that the *conflict, war and peace* topic consistently exhibits a performance degradation. Other topics that appear at least twice among the degraded categories are *society, disaster, accident and emergency incident, science and technology, and environment*.

The topic with the largest improvement is consistently *sport*. Other categories that appear most frequently among the top five improved topics are *arts, culture, entertainment and media, lifestyle and leisure, and crime, law and justice*.

Statistically Significant Classes

Aggregate topic-level trends can mask reliable per-class effects for individual classes. We therefore apply McNemar’s test with false-discovery-rate correction(FDR) to identify IPTC categories for which the entity-enhanced model significantly outperforms the baseline.

Due to the small support of many classes, the results for individual categories can be noisy. We therefore performed McNemar’s test to identify classes with statistically significant improvements or degradations. To ensure the statistical validity of the test, we set the minimum number of paired disagreements to 25. This threshold reduced the number of testable classes to 108 candidates for significant improvement (classes with an increased F1-score) and 30 candidates for significant degradation (classes with a decreased F1-score).

IPTC Category	Support	F1 Base (%)	F1 Current (%)	$\Delta F1$ (%)	p-value (FDR)
economy+ >>> electricity (20000260)	32	46.5	60.3	13.8	0.0034
arts+ >> television (20000051)	70	47.9	60.7	12.8	0.0001
economy+ >>> stocks and securities (20000396)	146	79.4	88.7	9.3	0.0000
sport >> cycling (20000892)	90	81.4	90.1	8.7	0.0252
sport >> skiing (20001056)	190	85.2	93.9	8.7	0.0000
economy+ >> business financing (20000183)	183	73.6	81.9	8.3	0.0000
law+ >> homicide (20000099)	121	59.3	67.5	8.2	0.0001
sci-tech >> medical research (20000737)	64	50.3	58.3	8.0	0.0000
politics and government >> local government (20000612)	146	55.2	62.1	6.9	0.0001
arts+ > culture (20000038)	375	58.0	63.7	5.7	0.0096
economy+ > business information (20000170)	345	65.8	71.1	5.3	0.0004
politics and government >> political party (20001384)	1006	73.6	78.5	4.9	0.0000
environment > sustainability (20001374)	991	76.4	81.1	4.7	0.0001
politics and government >> coalition building (20001382)	163	54.5	59.2	4.7	0.0214
economy+ >> utilities (20001244)	77	43.9	48.3	4.4	0.0068
sport >> tennis (20001085)	258	94.6	98.6	4.0	0.0017
sci-tech > scientific research (20000735)	153	55.1	59.0	3.9	0.0320
disaster+ >>> road accident and incident (20000147)	149	81.7	85.3	3.6	0.0465
environment (06000000)	1198	79.2	82.7	3.5	0.0004
economy+ > market and exchange (20000385)	223	76.3	79.7	3.4	0.0002
weather > weather phenomena (20001129)	42	29.2	32.4	3.2	0.0147
sport >> soccer (20001065)	1022	93.6	96.2	2.6	0.0000
sport > sport event (20001108)	1211	72.9	75.0	2.1	0.0000
politics and government > government (20000593)	2089	76.7	78.7	2.0	0.0017
politics and government > government policy (20000621)	958	62.1	63.8	1.7	0.0203
arts+ - arts, culture, entertainment and media (01000000)	1279	79.4	80.9	1.5	0.0100
economy+ - economy, business and finance (04000000)	2728	79.6	80.9	1.3	0.0019
law+ > crime (20000082)	906	75.1	76.3	1.2	0.0494
politics and government (11000000)	3472	83.0	84.1	1.1	0.0005
sport > competition discipline (20000822)	2410	96.3	97.0	0.7	0.0008

Table 5.9 All statistically significantly improved categories for *WP Entities All langs (att.)* vs. the base model (McNemar test, FDR-corrected $p < 0.05$). Categories are sorted by $\Delta F1$ in descending order.

Given the high number of evaluated classes, we applied the False Discovery Rate (FDR) correction described in Section 1.6.2. Using this procedure, we identified 30 classes with significant improvements, shown in Table 5.9. Conversely, only one class showed a significant degradation: *politics and government > international relations (20000638)*, which dropped by 0.2% (FDR-corrected $p = 0.0147$). Other classes experienced performance drops, but these were not statistically significant. Examples include *society > fundamental rights (20000587)*, *society » death and dying (20000810)*, and *politics and government » head of state (20000610)*.

Among the significantly improved classes, sports and economy topics are strongly represented. Table 5.9 lists these categories along with their support sizes, F1-score changes, and corrected p-values.

Increasing Performance with Class Support

Next, we examine whether entity features help uniformly across the frequency spectrum. Table 5.10 and Figure 5.5 compare baseline and best-model performance as a function of class support.

To evaluate how class support affects classifier performance, we sorted the classes by their support size and plotted the moving average of the F1-score (window size = 70) for both the baseline and our best classifier. As shown in Figure 5.5, the raw per-class results are noisy, therefore we chose a window size of 70 to smooth the data.

The figure shows that performance improves as class support increases for both classifiers. The performance gap between the two models decreases for classes with larger support, likely due to a ceiling effect. However, our proposed model

Class Size Category (Support)	Base F1 (%)	Current F1 (%)	Δ F1 (%)
Tail classes (1–15)	22.9	24.6	+1.7
Small classes (16–100)	49.4	53.0	+3.6
Medium classes (101–1000)	66.3	68.1	+1.8
Large classes (> 1000)	80.6	82.1	+1.5

Table 5.10 Macro F1-score comparison between the baseline and the best model across different class support categories.

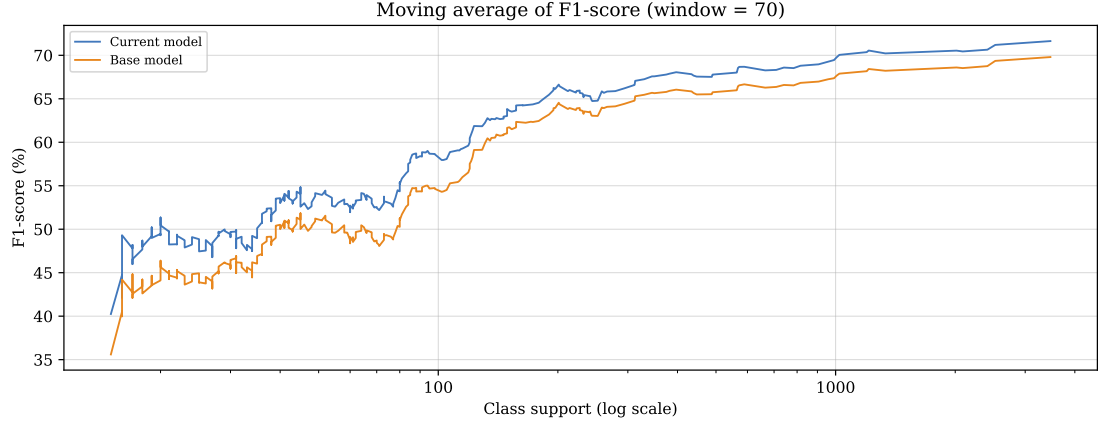


Figure 5.5 Moving average (window = 70) of absolute F1-score for both classifiers plotted against class support on a logarithmic x-axis.

remains consistently above the baseline across the entire support spectrum. This suggests that the entities add value for both small and large classes.

We also divided the classes into four categories based on their support sizes to compare the macro F1-scores of both models. The results, summarized in Table 5.10, show that our best model outperforms the baseline across all support levels, with the largest improvement seen in the small classes.

Threshold Analysis

We also analyzed the classification thresholds to ensure long-tail classes were handled correctly. Prior to this analysis, the per-class thresholds were optimized to maximize the F1-score on a validation set. We ordered the classes by their support and plotted these thresholds in Figure 5.6, along with a rolling mean with window 20.

The regression line shows that the optimal threshold generally increases with class support. This indicates that classes with fewer samples require lower thresholds to maintain performance. None of the classes reached the maximum allowed threshold of 0.70. Conversely, 18 classes hit the minimum threshold boundary of 0.25.

While these classes hitting the minimum boundary suggest that it could potentially be lowered further, empirical tests showed that decreasing it worsened our micro F1-score due to an increase in false positives. Because the vast majority of classes remained comfortably above this limit, and lowering it hurt the primary metric, we conclude that the minimum threshold boundary of 0.25 was chosen correctly.

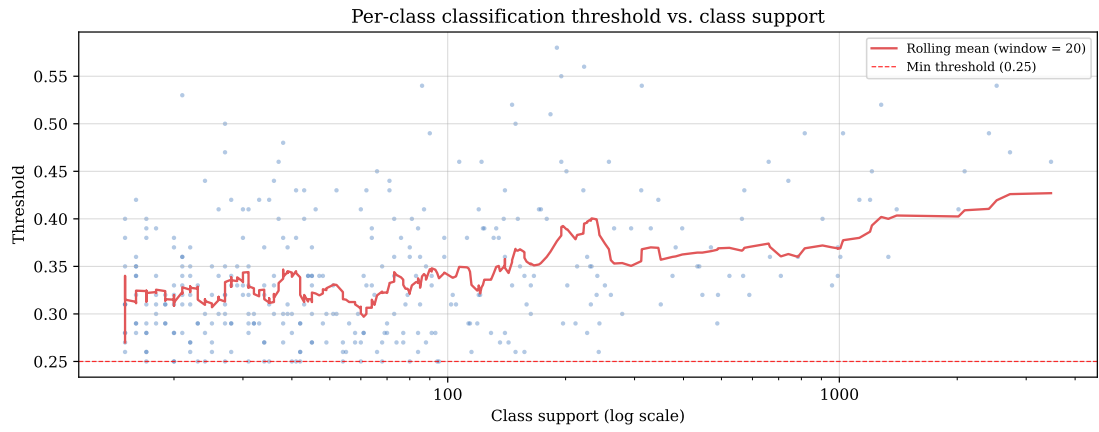


Figure 5.6 Per-class classification threshold plotted against class support on a logarithmic x-axis, with a rolling mean (window = 20, classes sorted by ascending support) overlaid to show the trend. The red dashed line marks the minimum threshold of 0.25.

Conclusion

5.4 Summary of Contributions

The primary objective of this thesis was to investigate whether incorporating entity information can improve the performance of a newspaper article classifier. Building upon an initial text-only baseline, we systematically evaluated multiple entity embedding sources, pooling strategies, and fusion methods to integrate structured entity knowledge into the classification pipeline. Our empirical evaluation on a diverse set of journalistic corpora confirms that entity-aware models consistently outperform text-only approaches. The best-performing model achieved a 1.7% improvement in the Micro F_1 score and a 2.4% improvement in the Macro F_1 score on the held-out test set, demonstrating that named entities provide a highly valuable supplementary signal for topic classification.

5.5 Key Findings

5.5.1 Architectural and Modeling Insights

As a first step we established our baseline. Our experiments showed that while standard interventions like modifying activation functions or adjusting initialization priors had negligible effects on addressing class imbalance, explicit threshold tuning clearly improved the Macro F_1 score. By lowering the classification threshold for minority classes, the model compensated for the lack of positive training samples. It also improved the Micro F_1 score, our objective metric.

When integrating entity features, our evaluation highlighted several important architectural considerations. First, we evaluated representations derived from Wikipedia and Wikidata, including two external projects (the Wikidata Embedding Project, for which we added support for 5 new languages, and Wikipedia2Vec) and three of our own sources. All of these sources performed similarly. Although they differ in context size, their comparable performance suggests that increasing the context does not necessarily improve the results. Second, regarding aggregation strategies, mean-based pooling outperformed summation. Mean pooling bounded the vector magnitude and prevented the introduction of numerical noise. Finally, using a learned attention mechanism proved to be the best pooling strategy, allowing the model to dynamically weigh the relevance of different entities within a given article.

Furthermore, our experiments on vector fusion revealed that concatenating the article and entity embeddings is superior to element-wise summation. Concatenation enables the classifier to process the textual and entity features as distinct signals, avoiding the semantic interference observed when summing dense representations. Finally, our investigation into multilingual entity representations showed that implementing a language fallback chain provided only marginal, statistically insignificant improvements over an English-only approach, suggesting that translating or mapping cross-lingual entities is not strictly necessary for this specific pipeline.

5.5.2 Domain and Class-Level Impact

The addition of entity embeddings did not uniformly improve classification across all topics; rather, it provided substantial, targeted boosts to specific domains. Analyzing the per-corpus and per-class results revealed large improvements in sports journalism. Entities such as specific athlete names act as strong, unambiguous discriminators for identifying exact sports disciplines, leading to a nearly 10% average Macro F_1 improvement across sport-related subcategories.

Statistically significant improvements, validated via McNemar’s test, were also observed in economics, arts, and local government topics. Furthermore, our analysis of class support showed that the entity-enhanced model outperformed the baseline across all class sizes, with the most pronounced gains occurring in small-to-medium classes. This is likely because these classes have more room for improvement, given that per-class performance depends on support size, as we showed.

5.6 Limitations

Despite the overall positive results, our approach has limitations. In topics such as *conflict*, *war and peace*, the inclusion of entity embeddings resulted in a minor performance degradation. In these contexts, the extracted entities are frequently prominent politicians, international organizations, or geographical entities that appear across a wide variety of news topics. Because these entities lack discriminative specificity, they act as semantic noise, which even the learned attention mechanism struggled to fully filter out.

Additionally, we observed a ceiling effect in certain corpora, such as the English Eurosport dataset. In datasets where the text-only baseline already achieved highly accurate classification ($> 90\%$ F_1), the introduction of entity embeddings yielded diminishing returns, leaving even smaller room for further measurable improvement.

5.7 Future Work

We see several areas of possible improvement. First, future architectural improvements could introduce a category-specific weighting or gating mechanism. Because the importance of entity embeddings varies by subcategory (highly informative for sports or politics, but potentially noisy for conflicts), allowing the architecture to independently weigh article and entity embeddings for each specific category or topic could be beneficial.

Second, while our experiments with structural Wikidata properties yielded no significant improvements, we remain skeptical that explicitly modeling relational structures via Graph Neural Networks (GNNs) would be effective, as our current evaluations suggest that increasing context is unhelpful. A more viable direction is improving data quality through representation cleaning. Future research should explore identifying entities with insufficient descriptions and repairing or enriching them using Large Language Models (LLMs). In addition, investigating novel ways to fuse Wikidata descriptions with Wikipedia introductions, or incorporating alternative knowledge sources, could yield richer representations. Upgrading

the underlying embedding architecture to newer models, such as subsequent generations of the Jina embedding family (e.g., Jina-v3 or Jina-v5), could also provide higher-quality, more nuanced entity representations.

Third, experimenting with reduced entity embedding dimensionality presents an interesting optimization opportunity. Using smaller embeddings from the same model architecture could prevent the over-generalization of entity representations. Because the model currently relies on rather broad information captured in large embedding spaces, reducing the dimensionality could force the embeddings to retain only the more general, discriminative features, thereby improving computational efficiency and reducing semantic noise.

Finally, the thresholding process can be further optimized with a specific focus on smaller, long-tail classes. Future work could explore assigning multiple decision thresholds for each class or adopting a conditional thresholding strategy. By dynamically adjusting the threshold of a minority class based on the model’s co-predicted categories, it may be possible to increase recall for small classes while minimizing the corresponding increase in false positives.

Bibliography

1. ZHANG, Min-Ling; ZHOU, Zhi-Hua. A review on multi-label learning algorithms. *IEEE Transactions on Knowledge and Data Engineering*. 2013, vol. 26, no. 8, pp. 1819–1837.
2. GIUDICE, Alessio et al. Hierarchical Text Classification: A Review Of Current Research. *arXiv preprint*. 2024.
3. MEEGLE. *Text Classification Techniques*. 2025. Available also from: https://www.meegle.com/en_us/topics/natural-language-processing/text-classification-techniques. Accessed: 2026-06-24.
4. KOWSARI, Kamran; JAFARI MEIMANDI, Kiana; HEIDARYSAFA, Mojtaba; MENDU, Sanjana; BARNES, Laura; BROWN, Donald. Text Classification Algorithms: A Survey. *Information*. 2019, vol. 10, no. 4, p. 150. Available from DOI: 10.3390/info10040150.
5. INTERNATIONAL PRESS TELECOMMUNICATIONS COUNCIL (IPTC). *Media Topics* [<https://iptc.org/standards/media-topics/>]. 2025. [Online; accessed 17-November-2025].
6. YADAV, Vikas; BETHARD, Steven. A Survey on Recent Advances in Named Entity Recognition from Deep Learning Models. *Proceedings of COLING*. 2019.
7. TJONG KIM SANG, Erik F.; DE MEULDER, Fien. Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition. In: *Proceedings of CoNLL*. 2003.
8. CUCERZAN, Silviu. Large-Scale Named Entity Disambiguation Based on Wikipedia Data. In: EISNER, Jason (ed.). *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*. Prague, Czech Republic: Association for Computational Linguistics, 2007, pp. 708–716. Available also from: <https://aclanthology.org/D07-1074/>.
9. WIKIPEDIA. *Wikipedia, The Free Encyclopedia* [<https://en.wikipedia.org/wiki/Wikipedia>]. 2026. [Online; accessed 23-Jan-2026].
10. WIKIPEDIA CONTRIBUTORS. *Wikipedia:Manual of Style/Linking — Wikipedia, The Free Encyclopedia*. 2024. Available also from: https://en.wikipedia.org/wiki/Wikipedia:Manual_of_Style/Linking. [Online; accessed 23-January-2026].
11. VRANDEČIĆ, Denny; KRÖTZSCH, Markus. Wikidata: a free collaborative knowledgebase. *Communications of the ACM*. 2014, vol. 57, no. 10, pp. 78–85. Available from DOI: 10.1145/2629489.
12. WIKIDATA CONTRIBUTORS. *Wikidata: Introduction* [<https://www.wikidata.org/wiki/Wikidata:Introduction>]. 2024. Accessed: 2026-01-23.

13. JEBLAD. *Linked Data – San Francisco* [Wikimedia Commons, Creative Commons Attribution–ShareAlike 3.0 (CC BY-SA 3.0)]. 2012. Available also from: https://commons.wikimedia.org/wiki/File:Linked_Data_-_San_Francisco.svg. Example diagram of linked data applied to a Wikidata item, adapted for illustrative purposes.
14. WIKIDATA CONTRIBUTORS. *Help: Sitelinks* [<https://www.wikidata.org/wiki/Help:Sitelinks>]. 2024. Accessed: 2026-01-23.
15. REIMERS, Nils; GUREVYCH, Iryna. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019. Available also from: <http://arxiv.org/abs/1908.10084>.
16. YAMADA, Ikuya; ASAI, Akari; SAKUMA, Jin; SHINDO, Hiroyuki; TAKEDA, Hideaki; TAKEFUJI, Yoshiyasu; MATSUMOTO, Yuji. Wikipedia2Vec: An Efficient Toolkit for Learning and Visualizing the Embeddings of Words and Entities from Wikipedia. In: LIU, Qun; SCHLANGEN, David (eds.). *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, 2020, pp. 23–30. Available from DOI: 10.18653/v1/2020.emnlp-demos.4.
17. LEUNG, Kenneth. *The Dying ReLU Problem, Clearly Explained*. 2021-03. Available also from: <https://towardsdatascience.com/the-dying-relu-problem-clearly-explained-42d0c54e0d24/>. Towards Data Science (Medium), accessed 2026-06-06.
18. HENDRYCKS, Dan; GIMPEL, Kevin. Gaussian error linear units (gelus). *arXiv preprint arXiv*. 2016. Available from eprint: 1606.08415.
19. VASWANI, Ashish; SHAZEER, Noam; PARMAR, Niki; USZKOREIT, Jakob; JONES, Llion; GOMEZ, Aidan N; KAISER, Łukasz; POLOSUKHIN, Illia. Attention is all you need. In: *Advances in Neural Information Processing Systems*. 2017, pp. 5998–6008.
20. BAHDANAU, Dzmitry; CHO, Kyunghyun; BENGIO, Yoshua. Neural Machine Translation by Jointly Learning to Align and Translate. In: *3rd International Conference on Learning Representations, ICLR 2015*. 2015.
21. LUONG, Minh-Thang; PHAM, Hieu; MANNING, Christopher D. Effective Approaches to Attention-based Neural Machine Translation. In: *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. 2015, pp. 1412–1421.
22. DIETTERICH, Thomas G. Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. *Neural Computation*. 1998, vol. 10, no. 7, pp. 1895–1923. Available from DOI: 10.1162/089976698300017197.
23. BENJAMINI, Yoav; HOCHBERG, Yosef. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B (Methodological)*. 1995, vol. 57, no. 1, pp. 289–300. Available from DOI: 10.1111/j.2517-6161.1995.tb02031.x.

24. LI, Jun; WITTEN, Daniela M.; JOHNSTONE, Iain M.; TIBSHIRANI, Robert. Normalization, Testing, and False Discovery Rate Estimation for RNA-Sequencing Data. *Biostatistics*. 2012, vol. 13, no. 3, pp. 523–538. Available from DOI: 10.1093/biostatistics/kxr031.
25. ZHANG, Yanzhao; LI, Mingxin; LONG, Dingkun; ZHANG, Xin; LIN, Huan; YANG, Baosong; XIE, Pengjun; YANG, An; LIU, Dayiheng; LIN, Junyang; HUANG, Fei; ZHOU, Jingren. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint*. 2025. Available from arXiv: 2506.05176 [cs.CL].
26. LEE, Jinhyuk; CHEN, Feiyang; DUA, Sahil; CER, Daniel; SHANBHOUE, Madhuri; NAIM, Iftekhar; HERNÁNDEZ ÁBREGO, Gustavo; LI, Zhe; CHEN, Kaifeng; VERA, Henrique Schechter, et al. Gemini Embedding: Generalizable Embeddings from Gemini. *arXiv preprint arXiv*. 2025. Available from arXiv: 2503.07891 [cs.CL].
27. LEE, Chankyu; ROY, Rajarshi; XU, Mengyao; RAIMAN, Jonathan; SHOEYBI, Mohammad; CATANZARO, Bryan; PING, Wei. NV-Embed: Improved Techniques for Training LLMs as Generalist Embedding Models. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2025.
28. CHEN, Jianlv; XIAO, Shitao; ZHANG, Peitian; LUO, Kun; LIAN, Defu; LIU, Zheng. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv preprint arXiv*. 2024. Available from arXiv: 2402.03216 [cs.CL].
29. TENNEY, Ian; DAS, Dipanjan; PAVLICK, Ellie. BERT Rediscovered the Classical NLP Pipeline. In: *Proceedings of ACL*. 2019.
30. WANG, Xiaozhi; GAO, Tianyu; ZHU, Zihan; LIU, Zhengyan; LI, Juanzi; TANG, Jian; ZHOU, Jie. KEPLER: A Unified Model for Knowledge Embedding and Pre-trained Language Representation. *Transactions of the Association for Computational Linguistics*. 2021, vol. 9, pp. 176–194.
31. YU, Donghan; ZHU, Chenguang; YANG, Yiming; ZENG, Michael. JAKET: Joint Pre-training of Knowledge Graph and Language Understanding. *arXiv preprint arXiv*. 2020. Available from arXiv: 2010.00796 [cs.CL].
32. SUN, Yu; WANG, Shuohuan; LI, Yukun; FENG, Shikun; TIAN, Hao; WU, Hua; WANG, Haifeng. ERNIE: Enhanced Representation through Knowledge Integration. *arXiv preprint arXiv*. 2019. Available from eprint: 1904.09223.
33. QIN, Guanghui; WENG, Zeqiu; CHEN, Yun; HUANG, Minlie; CAI, Deng; JIANG, Daxin. ERICA: Improving Entity and Relation Understanding for Pre-trained Language Models via Contrastive Learning. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL-IJCNLP)*. 2021, pp. 3350–3363.
34. WANG, Ruize et al. K-Adapter: Infusing Knowledge into Pre-Trained Models with Adapters. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. 2020, pp. 1405–1418.
35. POERNER, Nina; WALTINGER, Ulli; SCHÜTZE, Hinrich. E-BERT: Efficient-Yet-Effective Entity Embeddings for BERT. In: *Findings of the Association for Computational Linguistics: EMNLP-IJCNLP 2019*. 2019, pp. 803–818.

36. LIU, Weijie; ZHOU, Peng; ZHAO, Zhe; WANG, Zhiruo; JU, Qi; DENG, Haotang; WANG, Ping. K-BERT: Enabling Language Representation with Knowledge Graph. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2020, vol. 34, pp. 2901–2908. No. 03. Available from DOI: 10.1609/aaai.v34i03.5681.
37. YAMADA, Ikuya; ASAI, Akari; SHINDO, Hiroyuki; TAKEDA, Hideaki; MATSUMOTO, Yuji. LUKE: Deep Contextualized Entity Representations with Entity-aware Self-attention. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 6442–6454. Available from DOI: 10.18653/v1/2020.emnlp-main.523.
38. ZHANG, Zhengyan; HAN, Xu; LIU, Zhiyuan; JIANG, Xin; SUN, Maosong; LIU, Qun. ERNIE: Enhanced Language Representation with Informative Entities. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 2019, pp. 1441–1451. Available from DOI: 10.18653/v1/p19-1139.
39. PETERS, Matthew E.; NEUMANN, Mark; LOGAN IV, Robert; SCHWARTZ, Roy; JOSHI, Vidur; SINGH, Sameer; SMITH, Noah A. Knowledge Enhanced Contextual Word Representations. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 43–54.
40. WANG, Alex; SINGH, Amanpreet; MICHAEL, Julian; HILL, Felix; LEVY, Omer; BOWMAN, Samuel R. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2019. Available also from: <https://openreview.net/forum?id=rJ4km2R5t7>.
41. KUTBI, Mohammed. Named Entity Recognition Utilized to Enhance Text Classification While Preserving Privacy. *IEEE Access*. 2023, vol. 11, pp. 117576–117581. Available from DOI: 10.1109/ACCESS.2023.3325895.
42. BIE, Qingyuan. *Words are not enough! Short text classification using words as well as entities*. Karlsruhe, Germany, 2019. Available also from: <https://www.fiz-karlsruhe.de/sites/default/files/FIZ/Dokumente/Forschung/ISE/Masterarbeiten/Thesis-Qingyuan-Bie.pdf>. Master’s thesis. Karlsruhe Institute of Technology (KIT).
43. YAMADA, Ikuya; SHINDO, Hiroyuki. Neural Attentive Bag-of-Entities Model for Text Classification. In: BANSAL, Mohit; VILLAVICENCIO, Aline (eds.). *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 563–573. Available from DOI: 10.18653/v1/K19-1052.
44. WIKIMEDIA DEUTSCHLAND. *Wikidata:Embedding Project* [https://www.wikidata.org/wiki/Wikidata:Embedding_Project]. 2025. Accessed: 2026-06-01.
45. WIKIMEDIA DEUTSCHLAND. *Wikidata: Vector Database*. Wikidata, 2026. Available also from: https://www.wikidata.org/wiki/Wikidata:Vector_Database. Accessed: 2026-06-01.

46. STURUA, Saba; MOHR, Isabelle; AKRAM, Mohammad Kalim; GÜNTHER, Michael; WANG, Bo; KRIMMEL, Markus; WANG, Feng; MASTRAPAS, Georgios; KOUKOUNAS, Andreas; WANG, Nan; XIAO, Han. jina-embeddings-v3: Multilingual Embeddings With Task LoRA. *arXiv preprint arXiv:2409.10173*. 2024. Available also from: <https://arxiv.org/abs/2409.10173>.
47. SAADÉ, Philippe; CONTRIBUTORS. *WikidataTextEmbedding: Convert Wikidata Items to Vector Embeddings* [<https://github.com/exowanderer/WikidataTextEmbedding>]. 2026. GitHub repository. Accessed: 2026-06-01.
48. IDIO. *wiki2vec: Utility for learning word and entity embeddings from Wikipedia* [<https://github.com/idio/wiki2vec>]. 2015. GitHub repository, archived February 2021.
49. YAMADA, Ikuya; SHINDO, Hiroyuki; TAKEDA, Hideaki; TAKEFUJI, Yoshiyasu. Joint Learning of the Embedding of Words and Entities for Named Entity Disambiguation. In: *Proceedings of the 25th International Conference on Computational Linguistics (COLING 2016)*. 2016, pp. 250–259.
50. STUDIO OUSIA. *Wikipedia2Vec Documentation*. 2026. Available also from: <https://wikipedia2vec.github.io/wikipedia2vec/>. Accessed: 2026-06-01.
51. DEVLIN, Jacob; CHANG, Ming-Wei; LEE, Kenton; TOUTANOVA, Kristina. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. Available from arXiv: 1810.04805 [cs.CL].
52. LIU, Yinhan; OTT, Myle; GOYAL, Naman; DU, Jingfei; JOSHI, Mandar; CHEN, Danqi; LEVY, Omer; LEWIS, Mike; ZETTLEMOYER, Luke; STOYANOV, Veselin. RoBERTa: A Robustly Optimized BERT Pretraining Approach. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019, pp. 1–13. Available from DOI: 10.18653/v1/2020.acl-main.663.
53. NISHIKAWA, Sosuke; YAMADA, Ikuya; TSURUOKA, Yoshimasa; ECHIZEN, Isao. A Multilingual Bag-of-Entities Model for Zero-Shot Cross-Lingual Text Classification. In: *Proceedings of the 26th Conference on Computational Natural Language Learning (CoNLL)*. Association for Computational Linguistics, 2022, pp. 1–12. Available from DOI: 10.18653/v1/2022.conll-1.1.
54. OPENAI. *Embeddings Guide* [<https://developers.openai.com/api/docs/guides/embeddings>]. 2024. Accessed: 2026-03-19.
55. *iDNES.cz* [online]. MAFRA, 2026. [visited on 2026-07-09]. Available from: <https://www.idnes.cz>.
56. *Lidovky.cz* [online]. MAFRA, 2026. [visited on 2026-07-09]. Available from: <https://www.lidovky.cz>.
57. *Expres.cz* [online]. MAFRA, 2026. [visited on 2026-07-09]. Available from: <https://www.expres.cz>.
58. *Austria Presse Agentur* [online]. APA, 2026. [visited on 2026-07-09]. Available from: <https://www.apa.at>.
59. *Deutsche Presse-Agentur* [online]. dpa, 2026. [visited on 2026-07-09]. Available from: <https://www.dpa.com>.

60. *Eurosport* [online]. Warner Bros. Discovery, 2026. [visited on 2026-07-09]. Available from: <https://www.eurosport.com>.
61. *BBC News* [online]. British Broadcasting Corporation, 2026. [visited on 2026-07-09]. Available from: <https://www.bbc.com/news>.
62. *CNN* [online]. Cable News Network, 2026. [visited on 2026-07-09]. Available from: <https://edition.cnn.com>.
63. *Deutsche Welle* [online]. Deutsche Welle, 2026. [visited on 2026-07-09]. Available from: <https://www.dw.com>.
64. *Mediahuis* [online]. Mediahuis, 2026. [visited on 2026-07-09]. Available from: <https://www.mediahuis.com>.
65. *Radio Télévision Suisse* [online]. RTS, 2026. [visited on 2026-07-09]. Available from: <https://www.rts.ch>.
66. *EventDNA* [online]. EventDNA, 2026. [visited on 2026-07-09]. Available from: <https://eventdna.nl>.
67. *Noordhollands Dagblad* [online]. Mediahuis Nederland, 2026. [visited on 2026-07-09]. Available from: <https://www.noordhollandsdagblad.nl>.
68. *NU.nl* [online]. DPG Media, 2026. [visited on 2026-07-09]. Available from: <https://www.nu.nl>.
69. ZHANG, Ming-Liang; ZHANG, Xu-Yao; WANG, Chuang; LIU, Cheng-Lin. Towards prior gap and representation gap for long-tailed recognition. *Pattern Recognition*. 2023, vol. 133, p. 109012. ISSN 0031-3203. Available from DOI: <https://doi.org/10.1016/j.patcog.2022.109012>.
70. SCIKIT-LEARN DEVELOPERS. *MLPClassifier* [https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html]. 2026. scikit-learn API documentation, accessed 2026-06-03.
71. PYTHON SOFTWARE FOUNDATION. *Floating-Point Arithmetic: Issues and Limitations* [<https://docs.python.org/3/tutorial/floatingpoint.html>]. 2025. Python 3 Documentation, accessed 2026-06-03.
72. LIN, Tsung-Yi; GOYAL, Priya; GIRSHICK, Ross; HE, Kaiming; DOLLÁR, Piotr. Focal loss for dense object detection. In: *Proceedings of the IEEE international conference on computer vision (ICCV)*. 2017, pp. 2980–2988.
73. WIKIPEDIA CONTRIBUTORS. *Wikipedia: Size of Wikipedia* [PERMANENT_REVISION_URL]. 2026. Wikipedia, The Free Encyclopedia. Accessed: 2026-06-02.
74. YAMADA, Ikuya; SHINDO, Hiroyuki. A Multilingual Bag-of-Entities Model for Zero-shot Cross-lingual Text Classification. *arXiv preprint arXiv*. 2022. Available from DOI: 10.48550/arXiv.2110.07792.
75. STURUA, Saba; MOHR, Isabelle; AKRAM, Mohammad Kalim; GÜNTHER, Michael; WANG, Bo; KRIMMEL, Markus; WANG, Feng; MASTRAPAS, Georgios; KOUKOUNAS, Andreas; WANG, Nan; XIAO, Han. *jina-embeddings-v3: Multilingual Embeddings With Task LoRA*. 2024. Available from arXiv: 2409.10173 [cs.CL].
76. STUDIO OUSIA. *Pretrained Embeddings - Wikipedia2Vec* [<https://wikipedia2vec.github.io/wikipedia2vec/pretrained/>]. n.d. Accessed: 2026-06-02.

77. WIKIMEDIA FOUNDATION. *Wikidata Database Dump*. 2026. Version 20260301. Available also from: <https://dumps.wikimedia.org/wikidatawiki/20260301/>. Accessed 2026-06-05.
78. ATTARDI, Giuseppe. *WikiExtractor* [<https://github.com/attardi/wikie extractor>]. GitHub, 2015.
79. WIKIPEDIA CONTRIBUTORS. *English Wikipedia Dump (2026-04-01)* [<https://dumps.wikimedia.org/enwiki/20260401/>]. Wikimedia Foundation, 2026. Accessed: 2026-06-05. Specifically, the pages-articles dump.
80. WIKIPEDIA CONTRIBUTORS. *Reliability of Wikipedia* [https://en.wikipedia.org/wiki/Reliability_of_Wikipedia]. 2026. Wikipedia, The Free Encyclopedia. Accessed: 2026-07-03.
81. YANG, Beining; CAO, Yang; REN, Yang. Integrating Vector Databases across Embedding Models. *Proc. ACM Manag. Data*. 2025, vol. 3, no. 6. Available from DOI: 10.1145/3769803.

List of Figures

1.1	Interconnection of Wikidata items via properties within statements [13].	14
3.1	Example of a derived entity chain within the geographic hierarchy. Identifying a city explicitly triggers the inclusion of its higher-level administrative regions.	28
3.2	Overview of the entity creation pipeline.	29
3.3	Distribution of category counts across the training and test sets. .	36
3.4	Distribution of entity counts in the training dataset.	43
4.1	End-to-end architecture of the entity-enhanced IPTC pipeline . .	48
4.2	Construction of the entity embedding	50
4.3	Objective-metric and threshold-tuning computation on one dev fold	57
4.4	Baseline article-only IPTC classifier. Raw article text is encoded into a 384-dimensional multilingual embedding, passed to a feature fusion step, routed through a single-hidden-layer MLP, and converted to a multi-label prediction via sigmoid scores, per-class thresholds, and IPTC hierarchy normalisation. Entity embeddings are present in the architecture but disabled in this baseline. . . .	61
4.5	Entity embedding sources. The previously disabled entity branch is enabled: a pooled entity vector is concatenated with the article vector at the feature-fusion step (highlighted). In the entity-only regime the article input is removed and only the pooled entity vector is forwarded.	62
5.1	Per-corpus performance difference between the entity-enhanced and base classifier (top 9 corpora by micro F_1 -score improvement). Dashed horizontal lines indicate the micro-averaged difference across all corpora.	78
5.2	Per-corpus performance difference between the entity-enhanced and base classifier (remaining 8 corpora by micro F_1 -score improvement). Dashed horizontal lines indicate the micro-averaged difference across all corpora.	78
5.3	Per-language macro-averaged performance difference between the entity-enhanced and base classifier. Dashed horizontal lines indicate the micro-averaged difference.	79
5.4	Macro-averaged F_1 -score improvement per IPTC top-level topic. Numbers above bars indicate the count of qualifying subclasses (support ≥ 15).	80
5.5	Moving average (window = 70) of absolute F_1 -score for both classifiers plotted against class support on a logarithmic x-axis. .	83
5.6	Per-class classification threshold plotted against class support on a logarithmic x-axis, with a rolling mean (window = 20, classes sorted by ascending support) overlaid to show the trend. The red dashed line marks the minimum threshold of 0.25.	84

A.1	Distribution of entity counts in the training dataset.	107
A.2	Overview of the entity creation pipeline. Each stage transforms the intermediate representation, culminating in fully linked and scored entities.	109
A.3	Three parallel recognition methods feed into a unified span set. Each method contributes type hypotheses and source metadata. .	110

List of Tables

1.1	Examples of Wikidata statements forming a relational chain. . . .	14
3.1	Overview of the news corpora used in this work.	33
3.2	Quantitative overview of the corpora, detailing the total number of articles and the average token counts for the title, lead, and body text segments.	34
3.3	Named entity statistics per corpus, detailing the average number of detected entities, the subset successfully linked to Wikidata, and their associated relevance scores.	35
3.4	Corpus summary for the training split: total article count, percentage share within the split, mean article length in tokens (with standard deviation), mean number of IPTC labels per document, and the number of distinct categories containing more than 100 documents.	40
3.5	Per-language summary: Arts (article count), Art.% (share within the split), Art. Len (mean total article length and standard deviation), Lbl/doc (mean IPTC labels per document), and Cat.>100 (categories with more than 100 documents in the language). Statistics computed from the train split only.	41
3.6	Top 20 IPTC categories by document coverage across all corpora. Statistics computed from the train split only.	42
3.7	Bottom 10 IPTC categories by document coverage across all corpora. Statistics computed from the train split only.	42
3.8	Distribution of linked-entity types and their average relevance scores across all corpora. Statistics are computed from the training split only.	43
3.9	Corpus-level entity statistics per article: total recognized entities, subset of linked entities, and linked-entity relevance (mean and standard deviation). Statistics are computed from the training split only.	44
3.10	Language-level entity statistics per article: total recognized entities, subset of linked entities, and linked-entity relevance (mean and standard deviation). Statistics are computed from the training split only.	45
5.1	Class imbalance experiments (Dev). Comparison of activation-function variants on the development set. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold marks the highest value in each column.	70

5.2	Entity embedding source experiments (Dev). Comparison of entity embedding sources on the development set. Rows ending in (<i>only</i>) use entity embeddings without article text; the remaining rows combine entity embeddings with article embedding. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold marks the highest value in each column.	71
5.3	Per-corpus $\Delta F1$ vs. baseline (Dev). Per-experiment $\Delta F1$ (%) vs. the baseline (<i>Article Only (Tuned)</i>) for six selected corpora. Green cells indicate improvement, red cells indicate degradation; colour intensity is proportional to the magnitude (saturates at ± 4 percentage points).	73
5.4	WP Entities pooling strategies (Dev). Comparison of heuristic entity pooling operators on WP Entities embeddings on the development set. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold marks the highest value in each column.	73
5.5	Entity embedding source experiments with attention (Dev). Comparison of entity embedding sources with attention as pooling strategy on the development set. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold marks the highest value in each column.	74
5.6	WP Entities language fallback experiments (Dev). Comparison of WP Entities representations with different language fallback strategies on the development set. Each row uses entity embeddings from the Wikipedia project combined with article text. <i>All langs</i> means the fallback chain $en \rightarrow de \rightarrow fr \rightarrow nl \rightarrow es \rightarrow cs$: if no English representation exists for an entity, German is tried next, then French, and so on. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold marks the highest value in each column.	75
5.7	Comparison of concatenation vs. summation for fusing article and entity embeddings on the development set. Rows without a suffix use concatenation; rows marked (<i>sum</i>) add the article and pooled-entity vectors element-wise. Each row reports the best hyperparameter trial (ranked by micro F1) from a 5-fold cross-validation run. Bold text indicates the highest score in each column.	75
5.8	Final model comparison on the test set. Comparison of selected models on the held-out test set. The baseline uses article text only; subsequent rows combine article text with entity embeddings using entity embedding made by weighting entities by their number of mentions (M.W.M) or learned attention pooling. Metrics are read from the final test evaluation. Bold marks the highest value in each column among rows with available results.	77
5.9	All statistically significantly improved categories for <i>WP Entities All langs (att.)</i> vs. the base model (McNemar test, FDR-corrected $p < 0.05$). Categories are sorted by $\Delta F1$ in descending order. . . .	82

5.10	Macro F1-score comparison between the baseline and the best model across different class support categories.	83
A.1	Quantitative overview of the corpora, showing the number of articles and average word counts for title, lead, and body text segments. Statistics computed from the test split only.	105
A.2	Per-corpus summary: Arts (article count), Art.% (share within the split), Art. Len (mean total article length and standard deviation), Lbl/doc (mean IPTC labels per document), and Cat.>15 (categories with more than 15 documents in the corpus). Statistics computed from the test split only.	106
A.3	Per-language summary: Arts (article count), Art.% (share within the split), Art. Len (mean total article length and standard deviation), Lbl/doc (mean IPTC labels per document), and Cat.>15 (categories with more than 15 documents in the language). Statistics computed from the test split only.	106
A.4	Top 20 IPTC categories by document coverage for all corpora. Statistics computed from the test split only.	107
A.5	Per-corpus entity statistics per article: total entities, linked entities, and linked-entity relevance (mean and standard deviation). Statistics computed from the test split only.	108
A.6	Per-language entity statistics per article: total entities, linked entities, and linked-entity relevance (mean and standard deviation). Statistics computed from the test split only.	108
A.7	Per-corpus micro F1 improvements over baseline (Dev). Per-corpus micro F1 improvements over the baseline.	114
A.8	Per-corpus micro F1 degradations vs baseline (Dev). Per-corpus micro F1 degradations vs. the baseline.	117
A.9	Data split comparison on the test set. Comparison of model performance using the proposed (New) corpus-level chronological split versus the baseline (Old) data split. Each row reports the evaluation metrics on the test set. Bold marks the highest value in each column.	120

List of Abbreviations

- **API** – Application Programming Interface
- **BCE** – Binary Cross-Entropy
- **BERT** – Bidirectional Encoder Representations from Transformers
- **BGE** – Beijing Academy of Artificial Intelligence Generalized Embedding
- **BoW** – Bag of Words
- **CBOW** – Continuous Bag-of-Words
- **CNN** – Convolutional Neural Network / Cable News Network
- **CRF** – Conditional Random Fields
- **CSV** – Comma-Separated Values
- **CV** – Cross-Validation
- **DB** – Database
- **FDR** – False Discovery Rate
- **FN** – False Negative
- **FP** – False Positive
- **GELU** – Gaussian Error Linear Unit
- **GKB** – Geneea Knowledge Base
- **GNN** – Graph Neural Network
- **HPO** – Hyperparameter Optimization
- **IPTC** – International Press Telecommunications Council
- **JSON** – JavaScript Object Notation
- **JSONL** – JSON Lines
- **KE** – Knowledge Embedding
- **KG** – Knowledge Graph
- **LLM** – Large Language Model
- **MCC** – Matthews Correlation Coefficient
- **MHA** – Multi-Head Attention
- **MLM** – Masked Language Modeling
- **MLP** – Multilayer Perceptron
- **MWL** – Multi-Word Lemma

- **NDA** – Non-Disclosure Agreement
- **NER** – Named Entity Recognition
- **NLP** – Natural Language Processing
- **PLM** – Pre-trained Language Model
- **RAG** – Retrieval-Augmented Generation
- **ReLU** – Rectified Linear Unit
- **RoBERTa** – Robustly Optimized BERT Approach
- **S-BERT** – Sentence-BERT
- **SVM** – Support Vector Machine
- **TF-IDF** – Term Frequency-Inverse Document Frequency
- **TN** – True Negative
- **TP** – True Positive
- **UID** – Unique Identifier
- **WDID** – Wikidata Identifier
- **XML** – Extensible Markup Language

A Attachments

This appendix provides supplementary data, extended experimental results, and detailed methodological steps that support the primary findings of this thesis. The supplementary materials are organized into the following sections:

- **Test Split Analysis:** Provides comprehensive quantitative statistics, word counts, and entity distributions specifically for the held-out test datasets.
- **Linked Entity Creation:** Details the complete multi-stage pipeline used to recognize, normalize, disambiguate, and score named entities from the raw corpora.
- **Experiment 2 Extensions:** Contains full per-corpus performance tables, documenting both micro F1-score improvements and degradations relative to the baseline classifier.
- **Data Split Comparison:** Presents a comparative validation of the proposed corpus-level chronological split against the legacy random data split.

A.1 Test Split Analysis

Corpus	Articles	Article length			Total	Total σ
		Title	Lead	Body		
cs_mafra_iptc	210	12	50	471	534	360
de_apa	1,438	8	9	250	267	217
de_dpa_iptc	330	8	5	313	326	266
de_eurosport_iptc	363	12	53	277	342	293
en_bbc_iptc	844	9	0	481	490	347
en_cnn_iptc	598	14	34	852	900	610
en_dw_iptc	999	9	30	1062	1102	950
en_eurosport_iptc	463	16	59	604	679	631
en_mediahuis_iptc	79	12	29	704	745	710
es_eurosport_iptc	169	17	61	370	448	226
fr_rts_instagram_iptc	120	24	0	242	266	79
fr_rts_iptc	1,527	11	28	391	430	285
nl_eurosport_iptc	106	14	52	276	343	363
nl_eventdna	177	0	0	60	60	22
nl_mediahuis_iptc	69	11	0	293	304	241
nl_noordhollandsdagblad	2,807	14	40	367	421	302
nl_nu_iptc	410	13	40	325	379	243
avg. (macro)	629	12	29	432	473	361
avg. (micro)	—	12	29	456	497	376
Total	10,709	—	—	—	—	—

Table A.1 Quantitative overview of the corpora, showing the number of articles and average word counts for title, lead, and body text segments. Statistics computed from the test split only.

A.1.1 Entity Test Statistics

Corpus	Arts	Arts %	Art. $\bar{x}$	Len σ	Lbl/doc	Cat.>15
cs_mafra_iptc	210	2.0	534	360	5.2	20
de_apa	1,438	13.4	267	217	5.9	94
de_dpa_iptc	330	3.1	326	266	5.3	34
de_eurosport_iptc	363	3.4	342	293	3.6	13
en_bbc_iptc	844	7.9	490	347	7.2	81
en_cnn_iptc	598	5.6	900	610	6.8	54
en_dw_iptc	999	9.3	1102	950	7.9	81
en_eurosport_iptc	463	4.3	679	631	3.9	12
en_mediahuis_iptc	79	0.7	745	710	6.7	6
es_eurosport_iptc	169	1.6	448	226	3.5	6
fr_rts_instagram_iptc	120	1.1	266	79	7.9	10
fr_rts_iptc	1,527	14.3	430	285	6.5	101
nl_eurosport_iptc	106	1.0	343	363	3.7	6
nl_eventdna	177	1.7	60	22	6.7	13
nl_mediahuis_iptc	69	0.6	304	241	5.8	5
nl_noordhollandsdagblad	2,807	26.2	421	302	5.5	131
nl_nu_iptc	410	3.8	379	243	6.6	41
avg. (macro)	629	5.9	473	361	5.8	42
avg. (micro)	—	—	497	376	6.0	—
Total	10,709	100.0	—	—	—	335

Table A.2 Per-corpus summary: Arts (article count), Art.% (share within the split), Art. Len (mean total article length and standard deviation), Lbl/doc (mean IPTC labels per document), and Cat.>15 (categories with more than 15 documents in the corpus). Statistics computed from the test split only.

Lang	Arts	Arts %	Art. $\bar{x}$	Len σ	Lbl/doc	Cat.>15
cs	210	2.0	534	360	5.2	20
de	2,131	19.9	289	242	5.4	116
en	2,983	27.9	813	740	6.8	179
es	169	1.6	448	226	3.5	6
fr	1,647	15.4	418	279	6.6	109
nl	3,569	33.3	394	300	5.7	164
avg. (macro)	1,784	16.7	483	358	5.5	99
avg. (micro)	—	—	497	408	6.0	—
Total	10,709	100.0	—	—	—	335

Table A.3 Per-language summary: Arts (article count), Art.% (share within the split), Art. Len (mean total article length and standard deviation), Lbl/doc (mean IPTC labels per document), and Cat.>15 (categories with more than 15 documents in the language). Statistics computed from the test split only.

Rank	Category	Docs	%
1	politics (11000000)	3,472	32.4
2	economy, business and finance (04000000)	2,728	25.5
3	sport (15000000)	2,522	23.6
4	sport > competition discipline (20000822)	2,410	22.5
5	politics > government (20000593)	2,089	19.5
6	economy, business and finance > products and services (20000209)	2,015	18.8
7	crime, law and justice (02000000)	1,401	13.1
8	society (14000000)	1,334	12.5
9	arts, culture, entertainment and media (01000000)	1,279	11.9
10	sport > sport event (20001108)	1,211	11.3
11	environment (06000000)	1,198	11.2
12	conflict, war and peace (16000000)	1,125	10.5
13	sport > competition discipline > soccer (20001065)	1,022	9.5
14	politics and government » political party (20001384)	1,006	9.4
15	environment > sustainability (20001374)	991	9.3
16	human interest (08000000)	991	9.3
17	politics > government policy (20000621)	958	8.9
18	crime, law and justice > crime (20000082)	906	8.5
19	arts, culture, entertainment and media > arts and entertainment (20000002)	816	7.6
20	politics > international relations (20000638)	784	7.3

Table A.4 Top 20 IPTC categories by document coverage for all corpora. Statistics computed from the test split only.

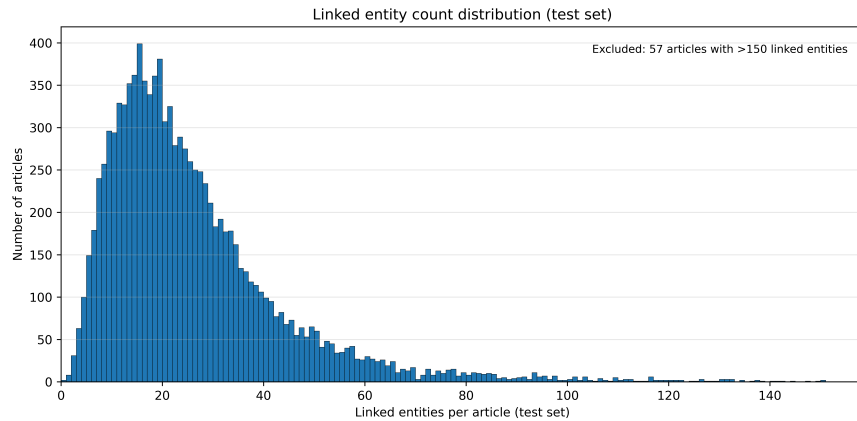


Figure A.1 Distribution of entity counts in the training dataset.

Corpus	Entities	Linked		Relevance	
	$\bar{x}$	$\bar{x}$	σ	$\bar{x}$	σ
cs_mafra_iptc	34.4	29.2	17.8	17.9	27.1
de_apa	22.6	17.9	17.3	28.2	30.7
de_dpa_iptc	31.0	28.5	31.7	19.6	27.1
de_eurosport_iptc	22.7	20.5	11.8	26.7	29.4
en_bbc_iptc	26.5	23.6	14.0	21.5	29.4
en_cnn_iptc	47.6	44.0	30.4	13.4	22.6
en_dw_iptc	57.7	53.2	38.3	11.4	20.6
en_eurosport_iptc	28.0	24.2	16.3	23.2	29.0
en_mediahuis_iptc	39.8	33.1	25.3	17.0	25.9
es_eurosport_iptc	23.0	17.0	13.2	29.6	28.2
fr_rts_instagram_iptc	24.1	21.5	8.0	23.9	31.5
fr_rts_iptc	33.2	29.9	17.9	18.5	27.7
nl_eurosport_iptc	25.2	21.4	18.5	22.1	28.9
nl_eventdna	7.3	6.5	2.6	56.0	37.7
nl_mediahuis_iptc	22.4	18.3	11.6	25.4	30.7
nl_noordhollandsdagblad	25.2	20.3	11.4	23.4	30.5
nl_nu_iptc	24.0	21.5	12.0	23.3	30.4
avg. (macro)	29.1	25.3	17.5	23.6	28.7
avg. (micro)	30.5	26.5	22.7	19.7	28.0

Table A.5 Per-corpus entity statistics per article: total entities, linked entities, and linked-entity relevance (mean and standard deviation). Statistics computed from the test split only.

Lang	Entities	Linked		Relevance	
	$\bar{x}$	$\bar{x}$	σ	$\bar{x}$	σ
cs	34.4	29.2	17.8	17.9	27.1
de	23.9	20.0	19.9	26.1	29.9
en	41.8	38.0	31.0	14.9	24.3
es	23.0	17.0	13.2	29.6	28.2
fr	32.6	29.3	17.5	18.8	28.0
nl	24.1	19.8	11.9	23.9	30.9
avg. (macro)	30.0	25.5	18.6	21.9	28.1
avg. (micro)	30.5	26.5	22.7	19.7	28.0

Table A.6 Per-language entity statistics per article: total entities, linked entities, and linked-entity relevance (mean and standard deviation). Statistics computed from the test split only.

A.2 Linked Entity Creation

The process of transforming raw text into structured, knowledge-base-linked entities is a multi-stage pipeline. Each stage refines the output of the previous one, progressively moving from surface-level pattern matching to semantically grounded, disambiguated entities with relevance scores. This section describes each stage.

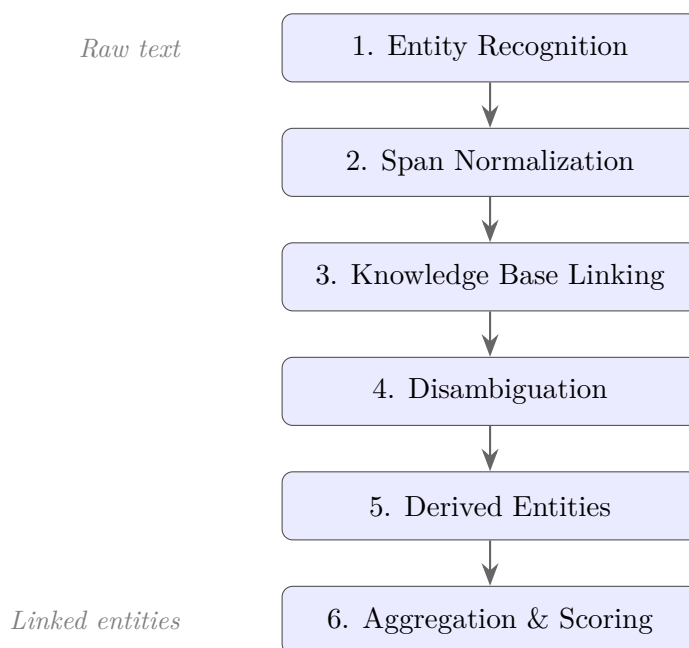


Figure A.2 Overview of the entity creation pipeline. Each stage transforms the intermediate representation, culminating in fully linked and scored entities.

A.2.1 Entity Recognition

The first stage identifies *spans*—contiguous sequences of tokens that potentially refer to named entities. Three complementary recognition methods operate in parallel, each with different strengths:

1. **Dictionary-based recognition.** Searchlists contain known entity names in their lemmatized forms, organized by language and entity type. When a token sequence matches a dictionary entry, the span inherits the entity type from the dictionary (e.g., person, organization, location).
2. **Statistical (machine learning) recognition.** Models trained on annotated corpora—typically using Conditional Random Fields (CRF) or similar sequence-labeling architectures—predict entity boundaries and types. These models generalize to previously unseen entities by learning contextual patterns (e.g., a capitalized word following “Mr.” is likely a person name). The trade-off is that they may produce false positives or assign incorrect types, especially for ambiguous or domain-specific names.
3. **Syntactic and rule-based recognition.** Heuristic rules exploit grammatical and orthographic cues. For example, capitalized out-of-vocabulary

nouns in certain syntactic positions are flagged as potential entities. Specialized sub-recognizers handle structured patterns such as email addresses, URLs, dates, and numeric expressions using regular expressions or dedicated parsers. Because these rules lack semantic knowledge, they typically assign only a generic type (e.g., `named_phrase`) rather than a specific category.

The outputs of all three methods are merged into a unified set of candidate spans. When multiple recognizers detect the same span, their results are aggregated—the span retains all proposed types and source annotations, which become useful signals during later disambiguation.

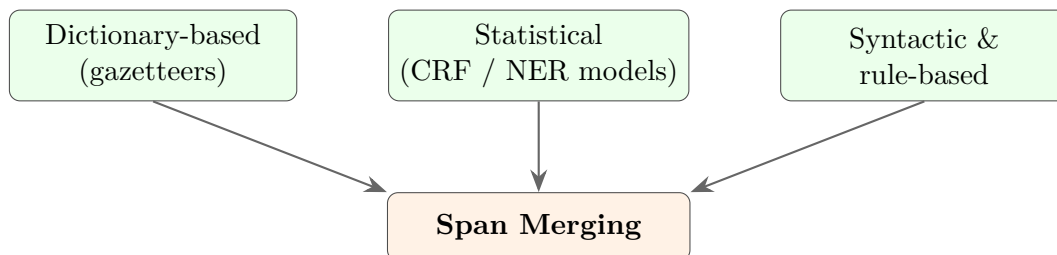


Figure A.3 Three parallel recognition methods feed into a unified span set. Each method contributes type hypotheses and source metadata.

A.2.2 Span Normalization

Entity names in running text appear in various inflected forms dictated by the grammar of the source language. For example, in Czech the organization name *Česká asociace pojišťoven* (Czech Insurance Association) might appear in the genitive case as *České asociace pojišťoven* or in the dative as *České asociaci pojišťoven*. Since knowledge base entries store names in a canonical (dictionary/nominative) form, each detected span must be normalized before linking can proceed.

This normalization step produces a *multi-word lemma* (MWL)—the base form of the entire multi-token span, not just individual token lemmas. Constructing the MWL requires morphological analysis that respects agreement patterns within the noun phrase. In morphologically rich languages, this step may generate multiple MWL hypotheses when the surface form is ambiguous, allowing downstream stages to consider all plausible normalizations.

A.2.3 Linking to the Knowledge Base

Once spans are normalized, the system attempts to link each one to an entry in the knowledge base (KB). The KB is a structured repository where each item represents a real-world entity and contains:

- A unique identifier (UID).
- One or more *labels*—alternative names or aliases for the entity, annotated with properties such as language, case class (e.g., always capitalized vs. common noun), and stability (how reliably the label identifies this specific entity).

- Semantic type (person, organization, location, etc.).
- Relations to other KB items (e.g., Paris $\rightarrow$ France).
- Popularity metrics derived from external sources (e.g., Wikipedia page views, inbound link counts).

A.2.4 Disambiguation

When the linking stage produces multiple candidate KB items for a single span, the system must select the most plausible one. This is achieved through a chain of scoring models, each contributing a positive or negative signal to each candidate’s cumulative score. The candidate with the highest final score is selected; if all candidates score below zero, the span is discarded as a false positive.

A.2.5 Derived Entities

After the disambiguation step, the system optionally appends *derived entities*. These are entities that are not explicitly mentioned in the text, but are closely connected to the detected entities through relations in the knowledge base.

In our dataset, we identify three types of derived entities:

- **Geographical entities:** For entities classified as locations, we add broader geographical regions. For example, if “Paris” is detected, we append “France”. There are 3,523 articles containing this type of derived entity.
- **Property-based entities:** These are added based on specific Wikidata properties (such as P176) linked to the originally detected entities. Only 1 article contains this type in our corpus.
- **Industry entities:** These represent broader business sectors, such as Finance, Logistics, or Mass Media. In our corpus there are 188 articles containing this type of derived entity.

Derived entities are flagged as such and typically receive lower relevance scores than directly mentioned entities (see Section A.2.6). They serve to enrich the semantic annotation, enabling downstream applications such as geographic aggregation or topic broadening.

A.2.6 Aggregation and Relevance Scoring

In the final stage, all mentions of the same entity (identified by their shared KB identifier) are aggregated into a single entity record. The system then computes a *relevance score* for each entity, quantifying how central the entity is to the article’s content.

The relevance score is determined by a weighted combination of several factors:

Term frequency. The number of times the entity is mentioned, weighted by the position of each mention. Mentions in the title carry the highest weight, followed by the lead paragraph, and then the body text. Derived entity mentions (not explicitly present in the text) receive a substantially reduced weight.

First occurrence position. Entities that appear early in the article, especially in the title or lead, receive higher scores. For body text, the score decreases with the distance from the beginning of the article.

Entity type. Different types of entities receive small type-specific bonuses. Persons and events tend to receive slightly higher bonuses than locations or products, reflecting the observation that person and event entities are more often central to news articles.

Knowledge base presence. Entities without a KB link (those with only a synthetic identifier) are penalized, as the lack of a KB match may indicate a recognition error or a marginal entity. In our thesis, we use only these entities.

Mention frequency threshold. Entities of generic type that appear less than a minimum number of times are penalized, reducing noise from incidental mentions.

Topic context. The overall topic of the article, determined by classification, influences the score of the entities. An entity that is typical for the article’s topic receives a boost, while one that is unusual for the topic may be penalized. For example, a news agency entity scores lower in a sports article than in a media-industry article.

Popularity. A small boost based on the entity’s general prominence in external knowledge sources (e.g., Wikipedia).

The raw weighted sum of these factors is then normalized using a logarithmic function:

$$s_{\text{norm}} = \log_2 \left(1 + \frac{s_{\text{raw}}}{D} \right) \quad (\text{A.1})$$

where s_{raw} is the entity’s raw score and D is the sum of all positive entity scores in the document. For long documents, D is computed only over entities with more than one mention, preventing single-mention entities from diluting the normalization. The normalized score is then scaled to an integer in the range $[1, 100]$.

In practice, scores above 80 indicate that the entity is a central theme of the article, scores between 20 and 80 correspond to entities that play a standard role in the content, and scores below 20 suggest peripheral or incidental mentions.

A.2.7 Influence on Disambiguation and Relevance

Early in the pipeline, before entity disambiguation begins, the system performs a preliminary topic classification of the article. This classification—often called *draft topics*—determines the broad thematic domain of the text (e.g., sports, politics, finance) using IPTC-aligned categories.

The draft topic influences entity processing in two ways:

1. **Disambiguation context.** Contextual rules stored in the knowledge base can reference the article’s topic. When the article is classified as a sports

text, disambiguation rules boost sports-related interpretations of ambiguous names. For instance, “Bayern Munich” is more likely to resolve to the football club than to the city when the article topic is sports.

2. **Relevance adjustment.** The relevance scoring stage (Section A.2.6) uses the article’s topic to adjust entity scores. Entities that are typical for the detected topic receive a boost, while entities that are atypical are penalized. This ensures that, for example, a news agency mentioned as a source attribution in a sports article does not receive an inflated relevance score.

A.3 Experiment 2: Attachments

Table A.7 lists per-corpus micro F1 improvements over the baseline (*Article Only (Tuned)*) on the development set. Each experiment group lists only corpora where micro F1 increased. Rows are sorted by $|\Delta F1|$ descending within each group. ΔFPR is experiment minus baseline; negative means fewer false positives.

Table A.7 Per-corpus micro F1 improvements over baseline (Dev). Per-corpus micro F1 improvements over the baseline.

Corpus	$\Delta F1$ (%)	ΔFPR (%)
<i>WP Entities (only)</i>		
Eurosport (ES)	+3.26	-0.02
Eurosport (NL)	+2.57	-0.00
Eurosport (DE)	+2.48	-0.01
Eurosport (EN)	+1.62	-0.00
<i>WP Entities PMM (only)</i>		
Eurosport (ES)	+2.42	-0.02
Eurosport (DE)	+1.49	-0.01
Eurosport (NL)	+1.42	+0.00
Eurosport (EN)	+1.12	-0.00
<i>Wikipedia2Vec (only)</i>		
Eurosport (ES)	+2.59	-0.02
Eurosport (DE)	+1.66	-0.01
Eurosport (EN)	+1.33	-0.00
Eurosport (NL)	+1.31	-0.00
<i>Wikidata Description (only)</i>		
Eurosport (ES)	+2.76	-0.02
Eurosport (DE)	+1.51	-0.01
Eurosport (NL)	+1.04	+0.00
Eurosport (EN)	+1.02	-0.00
<i>Wiki Intro (only)</i>		
Eurosport (ES)	+2.60	-0.02
Eurosport (DE)	+1.32	-0.01
Eurosport (NL)	+0.89	+0.00
Eurosport (EN)	+0.77	-0.00
<i>Wiki Article (only)</i>		
Eurosport (ES)	+2.47	-0.02
Eurosport (DE)	+1.82	-0.01
Eurosport (NL)	+1.59	+0.00
Eurosport (EN)	+1.02	-0.00
<i>WP Entities</i>		
Eurosport (ES)	+3.85	-0.02
Eurosport (NL)	+2.53	-0.01
Eurosport (DE)	+2.18	-0.01
EventDNA (NL)	+1.93	-0.00

continued from previous page

Corpus	$\Delta F1$ (%)	ΔFPR (%)
DPA (DE)	+1.85	-0.01
NoordHB(NL)	+1.73	-0.01
APA (DE)	+1.53	-0.00
Eurosport (EN)	+1.31	-0.00
Mediahuis (EN)	+1.18	+0.01
RTS (FR)	+1.16	+0.00
Mafra (CS)	+1.02	-0.01
BBC (EN)	+1.02	-0.01
CNN (EN)	+0.94	-0.00
Mediahuis (NL)	+0.86	+0.00
RTS Ing (FR)	+0.71	-0.00
DW (EN)	+0.67	+0.00
NU (NL)	+0.45	+0.01
<i>WP Entities PMM</i>		
Eurosport (ES)	+3.64	-0.02
Eurosport (NL)	+2.80	-0.01
DPA (DE)	+2.16	-0.02
Eurosport (DE)	+2.06	-0.01
NoordHB(NL)	+1.48	-0.01
APA (DE)	+1.44	-0.00
Eurosport (EN)	+1.25	-0.00
EventDNA (NL)	+1.22	+0.01
Mediahuis (EN)	+1.18	+0.00
Mafra (CS)	+0.95	-0.01
RTS (FR)	+0.86	+0.00
RTS Ing (FR)	+0.67	-0.01
BBC (EN)	+0.60	+0.00
CNN (EN)	+0.60	-0.01
DW (EN)	+0.33	-0.00
NU (NL)	+0.32	+0.01
<i>Wikipedia2Vec</i>		
Eurosport (ES)	+4.29	-0.03
Eurosport (NL)	+2.88	-0.01
DPA (DE)	+2.88	-0.03
Eurosport (DE)	+2.74	-0.02
EventDNA (NL)	+2.13	+0.01
NoordHB(NL)	+2.09	-0.01
Eurosport (EN)	+1.87	-0.01
APA (DE)	+1.74	-0.00
Mediahuis (EN)	+1.62	+0.00
RTS (FR)	+1.28	+0.00
RTS Ing (FR)	+1.02	-0.01
CNN (EN)	+0.85	-0.00
Mafra (CS)	+0.84	-0.01
BBC (EN)	+0.70	-0.00

continued from previous page

Corpus	$\Delta F1$ (%)	ΔFPR (%)
NU (NL)	+0.60	+0.00
DW (EN)	+0.55	+0.00
Mediahuis (NL)	+0.14	+0.00
<i>Wikidata Description</i>		
Eurosport (ES)	+3.86	-0.02
Eurosport (NL)	+2.56	-0.01
Eurosport (DE)	+2.12	-0.01
DPA (DE)	+1.92	-0.02
NoordHB(NL)	+1.62	-0.01
EventDNA (NL)	+1.59	+0.00
Eurosport (EN)	+1.49	-0.00
APA (DE)	+1.36	-0.01
Mafra (CS)	+1.19	-0.01
RTS (FR)	+0.96	-0.00
Mediahuis (NL)	+0.76	+0.00
CNN (EN)	+0.71	-0.01
BBC (EN)	+0.66	-0.00
Mediahuis (EN)	+0.66	+0.00
RTS Ing (FR)	+0.60	-0.01
NU (NL)	+0.50	-0.00
DW (EN)	+0.38	+0.00
<i>Wiki Intro</i>		
Eurosport (ES)	+3.50	-0.02
Eurosport (NL)	+2.50	-0.00
Eurosport (DE)	+2.01	-0.01
DPA (DE)	+1.86	-0.01
NoordHB(NL)	+1.59	-0.01
EventDNA (NL)	+1.45	+0.01
APA (DE)	+1.43	-0.00
Eurosport (EN)	+1.17	-0.00
Mafra (CS)	+0.85	-0.01
RTS (FR)	+0.84	+0.00
RTS Ing (FR)	+0.82	-0.00
Mediahuis (EN)	+0.81	+0.02
BBC (EN)	+0.59	-0.00
CNN (EN)	+0.53	+0.00
NU (NL)	+0.48	+0.01
DW (EN)	+0.39	+0.00
Mediahuis (NL)	+0.20	+0.01
<i>Wiki Article</i>		
Eurosport (ES)	+3.24	-0.02
Eurosport (NL)	+2.98	-0.01
Eurosport (DE)	+2.13	-0.01
DPA (DE)	+2.07	-0.02
EventDNA (NL)	+1.53	+0.01

continued from previous page

Corpus	$\Delta F1$ (%)	ΔFPR (%)
NoordHB(NL)	+1.52	-0.01
APA (DE)	+1.44	-0.00
Eurosport (EN)	+1.06	-0.00
RTS (FR)	+0.95	-0.00
Mediahuis (EN)	+0.94	+0.01
Mafra (CS)	+0.91	-0.00
BBC (EN)	+0.70	-0.00
RTS Ing (FR)	+0.67	-0.01
CNN (EN)	+0.60	+0.00
DW (EN)	+0.34	-0.00
NU (NL)	+0.23	+0.01

Table A.8 lists per-corpus micro F1 degradations vs. the baseline (*Article Only (Tuned)*) on the development set. Each experiment group lists only corpora where micro F1 decreased. Rows are sorted by $|\Delta F1|$ descending within each group. ΔFPR is experiment minus baseline; positive means more false positives.

Table A.8 Per-corpus micro F1 degradations vs baseline (Dev). Per-corpus micro F1 degradations vs. the baseline.

Corpus	$\Delta F1$ (%)	ΔFPR (%)
<i>WP Entities (only)</i>		
BBC (EN)	-7.11	+0.03
NU (NL)	-6.64	+0.04
Mediahuis (NL)	-6.18	+0.04
RTS Ing (FR)	-5.60	+0.03
DW (EN)	-5.59	+0.03
NoordHB(NL)	-5.10	+0.03
Mafra (CS)	-5.01	+0.01
CNN (EN)	-4.86	+0.03
Mediahuis (EN)	-4.58	+0.02
EventDNA (NL)	-4.55	+0.05
DPA (DE)	-3.83	-0.01
APA (DE)	-3.71	+0.01
RTS (FR)	-3.09	+0.02
<i>WP Entities PMM (only)</i>		
BBC (EN)	-8.68	+0.03
Mediahuis (NL)	-7.60	+0.04
NU (NL)	-7.55	+0.04
RTS Ing (FR)	-7.12	+0.03
DW (EN)	-6.74	+0.02
Mafra (CS)	-6.69	+0.01
EventDNA (NL)	-6.64	+0.06
NoordHB(NL)	-6.52	+0.03
CNN (EN)	-5.95	+0.02
Mediahuis (EN)	-5.72	+0.02

continued from previous page

Corpus	$\Delta F1$ (%)	ΔFPR (%)
DPA (DE)	-5.46	-0.00
APA (DE)	-5.03	+0.02
RTS (FR)	-4.03	+0.02
<i>Wikipedia2Vec (only)</i>		
BBC (EN)	-8.85	+0.03
Mediahuis (NL)	-7.34	+0.05
NU (NL)	-6.53	+0.05
RTS Ing (FR)	-5.87	+0.02
DW (EN)	-5.51	+0.02
NoordHB(NL)	-5.38	+0.03
Mafra (CS)	-5.38	+0.00
CNN (EN)	-5.05	+0.03
EventDNA (NL)	-4.91	+0.05
DPA (DE)	-4.69	-0.01
Mediahuis (EN)	-3.96	+0.01
APA (DE)	-3.67	+0.01
RTS (FR)	-3.43	+0.02
<i>Wikidata Description (only)</i>		
BBC (EN)	-9.13	+0.02
NU (NL)	-8.15	+0.04
RTS Ing (FR)	-7.16	+0.02
Mediahuis (NL)	-7.10	+0.04
DW (EN)	-7.00	+0.02
Mafra (CS)	-6.97	+0.01
EventDNA (NL)	-6.72	+0.06
NoordHB(NL)	-6.61	+0.03
CNN (EN)	-6.54	+0.02
Mediahuis (EN)	-5.77	+0.01
DPA (DE)	-5.74	+0.01
APA (DE)	-4.91	+0.01
RTS (FR)	-4.28	+0.02
<i>Wiki Intro (only)</i>		
BBC (EN)	-8.33	+0.02
NU (NL)	-7.54	+0.04
RTS Ing (FR)	-6.63	+0.02
DW (EN)	-6.46	+0.02
Mafra (CS)	-6.41	+0.01
NoordHB(NL)	-6.23	+0.03
CNN (EN)	-6.01	+0.03
Mediahuis (NL)	-5.70	+0.03
EventDNA (NL)	-5.66	+0.04
DPA (DE)	-5.51	+0.01
Mediahuis (EN)	-5.36	+0.01
APA (DE)	-4.75	+0.02

<i>continued from previous page</i>		
Corpus	$\Delta F1$ (%)	ΔFPR (%)
RTS (FR)	-3.84	+0.02
<i>Wiki Article (only)</i>		
BBC (EN)	-7.99	+0.02
NU (NL)	-7.49	+0.04
DW (EN)	-6.22	+0.02
Mediahuis (NL)	-6.21	+0.04
Mafra (CS)	-6.17	+0.01
RTS Ing (FR)	-6.16	+0.04
NoordHB(NL)	-5.99	+0.02
CNN (EN)	-5.73	+0.02
EventDNA (NL)	-5.26	+0.05
DPA (DE)	-5.02	+0.00
APA (DE)	-4.64	+0.01
Mediahuis (EN)	-4.33	+0.01
RTS (FR)	-3.70	+0.02
<i>WP Entities PMM</i>		
Mediahuis (NL)	-0.09	+0.01
<i>Wiki Article</i>		
Mediahuis (NL)	-0.31	+0.02

A.4 Comparison of Results on Our Data Split and the Old Data Split

Name	Our Data Split		Old Data Split	
	Micro F1 (%)	Macro F1 (%)	Micro F1 (%)	Macro F1 (%)
WP Entities (only)	66.3	49.5	65.9	50.0
WP Entities PMM (only)	65.0	46.4	64.5	47.6
Wikipedia2Vec (only)	65.7	48.8	65.3	49.4
Wikidata Description (only)	64.6	47.0	64.5	47.1
Wiki Intro (only)	65.4	47.6	64.7	48.3
Wiki Article (only)	65.7	47.7	65.1	49.1
WP Entities	72.1	58.9	72.0	58.8
WP Entities PMM	71.5	58.2	71.7	58.4
Wikipedia2Vec	72.0	59.2	71.8	58.3
Wikidata Description	71.7	58.5	71.6	58.1
Wiki Intro	71.8	58.6	71.7	58.2
Wiki Article	71.4	58.4	71.7	58.2
Base	70.6	56.9	70.4	56.4

Table A.9 Data split comparison on the test set. Comparison of model performance using the proposed (New) corpus-level chronological split versus the baseline (Old) data split. Each row reports the evaluation metrics on the test set. Bold marks the highest value in each column.

Table A.9 compares the performance of models trained and evaluated using the proposed corpus-level chronological split (New) and the original data split (Old). The purpose of this comparison is to determine whether the signal of model trained on the original split could artificially improve the quality of the entity representations available during evaluation, thereby affecting the measured performance of entity-enhanced classifiers.

Overall, the differences between the two data splits are very small. Across all evaluated models, the average absolute difference in Micro F1 is approximately 0.3 percentage points, while the average absolute difference in Macro F1 is approximately 0.5 percentage points. These results indicate that any performance gain caused by the original split is limited.

A useful reference is provided by the baseline model, which does not use any entity information and therefore should not benefit from improved entity quality. As shown in Table A.9, the baseline trained on the new split achieves a slightly higher performance (70.6% vs. 70.4% Micro F1 and 56.9% vs. 56.4% Macro F1). This suggests that the small differences observed between the two splits cannot be attributed solely to changes in entity quality.

Several factors may contribute to this slight improvement. First, the chronological split produces a somewhat different class distribution. In particular, some low-frequency categories contain fewer evaluation samples in the new test set. Since these long-tail categories are generally the most difficult to classify due to their limited training data as shown in results in Section 5.3.1, their reduced presence can slightly increase the overall evaluation metrics. Second, the observed differences may simply result from random variation in the sampled test articles.

Finally, the new split consists exclusively of recent news articles. These articles are generally easier for human annotators to label correctly because they are associated with contemporary events and well-known public figures, making the

article context easier to interpret. Similarly, the entity linking pipeline benefits from an up-to-date knowledge base, whose content closely matches the publication period of the evaluated articles. Consequently, entity linking quality can be expected to be slightly higher on recent articles even without any temporal leakage between the training and test sets.

Although these factors may introduce small differences between the evaluated splits, the experimental results demonstrate that their overall effect is limited. Therefore, the proposed chronological split is used throughout the remainder of this thesis, as it substantially reduces the possibility of temporal leakage in entities as we described when choosing the dataset split in Section 3.6.1 while maintaining comparable classification performance.